

DeepONet Framework for PCM-Energy Storage Systems

A thesis submitted in partial fulfilment
of the requirements for the award of the degree of

Master of Technology
In
Mechanical and Thermal Science Engineering

Submitted by

Asmit Balabantaray
22ME02016

Under the Supervision of

Dr. Anirban Bhattacharya
Dr. Manish Agrawal



School of Mechanical Sciences
Indian Institute of Technology Bhubaneswar
Argul, Jatni, Bhubaneswar – 752050
Sept 2026

यांत्रिकी विज्ञान विद्यापीठ/School of Mechanical Sciences
भारतीय प्रौद्योगिकी संस्थान भुवनेश्वर/Indian Institute of Technology Bhubaneswar
जटनी - 752050/Jatni-752050, खोर्धा/ Khordha, ओड़िशा, भारत/ Odisha, India



APPROVAL OF THE VIVA-VOCE BOARD

Date: 23 / 09 / 2026

Certified that the thesis entitled “DEEPONET FRAMEWORK FOR PCM-ENERGY STORAGE SYSTEMS” submitted by “ASMIT BALABANTARAY” (Roll No: 22ME02016), a postgraduate student of Mechanical and Thermal Science Engineering (ME), to the Indian Institute of Technology Bhubaneswar, for the award of the degree of Master of Technology, has been accepted by the external examiners and that the student has successfully defended the thesis in the viva-voce examination held today.

Dr. Anirban Bhattacharya
(Supervisor)

Dr. Prasenjit Rath
(Internal Examiner)

Dr. Amrit Bikram Sahu
(Internal Examiner)

Dr. Arun Kumar Pradhan
(Head of the School)

ACKNOWLEDGEMENTS

I take this opportunity to express my sincere gratitude to my supervisor, Dr. Anirban Bhattacharya, for his constant guidance, insightful feedback and sustained support throughout this work. His expertise shaped the direction of the project from its beginning as an undergraduate study through to its extension into the three-dimensional and cross-geometry problems presented here, and his patience during the long periods of dataset generation and model training is gratefully acknowledged.

I am equally grateful to Dr. Manish Agrawal of the Indian Institute of Technology Ropar for his close collaboration on the operator-learning aspects of this study. Many of the choices behind the data-generation scheme and the network architectures grew out of discussions with him.

I am thankful to the faculty and staff of the School of Mechanical Sciences, Indian Institute of Technology Bhubaneswar, for the academic environment and the computational resources that made this study possible, and for the constructive criticism offered during the presentations and reviews held over the course of the programme.

I would also like to thank my fellow research scholars and classmates for the many discussions that helped clarify my thinking on operator learning and on the numerical treatment of phase change.

Finally, I acknowledge the encouragement of my family and friends, without whose support this work would not have reached completion.

ASMIT BALABANTARAY

22ME02016

DECLARATION

I certify that,

1. The work contained in this thesis is original and has been done by me under the guidance and supervision of Dr. Anirban Bhattacharya and Dr. Manish Agrawal.
2. The work has not been submitted to any other institute for any Degree or Diploma.
3. I have followed the guidelines stipulated by the institute in preparing the Thesis.
4. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the institute.
5. Whenever I have used materials (data, theoretical analysis and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the References.

ASMIT BALABANTARAY

22ME02016

DeepONet Framework for PCM-Energy Storage Systems

Abstract

Phase change materials are widely used for thermal energy storage and for the thermal management of battery packs, because they absorb and release large quantities of latent heat at a nearly constant temperature. Predicting their transient behaviour requires the solution of a non-linear energy equation coupled to a moving solid–liquid interface, and although established numerical methods do this accurately, the cost of a single simulation makes repeated evaluation impractical for design exploration and optimisation. This thesis develops a deep operator network surrogate for transient melting and solidification in phase change material domains subjected to distributed and localised internal heat generation together with non-uniform boundary heating. Reference solutions are generated with an enthalpy-based melting and solidification model, and the training data span smooth Gaussian-process heat-source fields, battery-like sources of prismatic, circular and elliptical geometry, and hybrid combinations of the two. A pointwise DeepONet is developed first and then replaced by a grid-to-grid U-DeepONet, in which a convolutional U-Net encoder is fused with a temporal trunk network to return the complete transient temperature and liquid-fraction history in a single evaluation. Accuracy is found to be governed by how strongly the heat generation is concentrated rather than by the geometric complexity of the source arrangement. A cross-geometry study shows that a model trained only on prismatic sources predicts unseen circular and elliptical configurations without retraining, indicating that the formulation learns a largely local mapping rather than memorising source shapes. The framework is finally extended to three dimensions through a slice-based formulation, in which the volumetric fields are decomposed along three orthogonal directions and each plane is encoded together with a window of neighbouring planes to represent out-of-plane conduction. Applied to a prismatic battery heat source, the surrogate reproduces the three-dimensional fields accurately on every plane intersecting the battery, at an evaluation cost two to three orders of magnitude below that of the numerical solver.

Keywords

Phase change material; Deep operator network; U-DeepONet; Operator learning; Enthalpy-based melting and solidification model; Gaussian process regression; Battery thermal management; Surrogate modelling

CONTENTS

Contents	Page No.
CHAPTER 1: INTRODUCTION	15-20
1.1 Theoretical background	
1.2 History of development	
1.3 Applications	
1.4 Advantages and disadvantages	
1.5 Scope and Organisation of the Thesis	
CHAPTER 2: LITERATURE REVIEW & FORMULATION	21-28
2.1 Literature review	
2.2 Literature Gap	
2.3 Objective Formulation	
2.4 Methodology / Work Plan	
CHAPTER 3: EXPERIMENTAL DETAILS/ MODEL DEVELOPMENT	29-68
3.1 Physical Model for PCM System	
3.2 Data Generation with Gaussian Process Regression	
3.3 DeepONet Pointwise Surrogate Development	
3.4 Limitations of Pointwise MLP-Based DeepONet	
3.5 Extension to 3D PCM systems	
3.6 Cross-Geometry Training Configurations	
CHAPTER 4: RESULTS & DISCUSSION	69-142
4.1 Data Characteristics and Heating Modes	
4.2 Model Performance and Validation	
4.3 1D and 2D Prediction Plots and Inference	
4.4 Slice-based 3D U-DeepONet results	
4.5 Cross-Geometry Generalisation	

4.6 Computational Cost

4.7 General Discussion

CHAPTER 5: CONCLUSION AND FUTURE SCOPE 143-147

5.1 Conclusion

5.2 Future Scope

REFERENCES 149-151

LIST OF FIGURES

Fig.No.	Caption	Page No.
Fig. 1	Schematic of PCM–metal foam composite showing heat conduction paths through foam ligaments and PCM pores	30
Fig. 2	Temperature maps: (a) 1D-Temperature functions generated using Gaussian Process Regression (GPR); (b) Two-dimensional heat-source field generated using RBF-kernel GPR, with the boundary constrained to Q equals zero; (c) RBF-GPR heat-source field with heterogeneous boundary constraints	34
Fig. 3	Deep Operator Network (DeepONet) framework with branch net (global conditions), trunk net (local-coordinates), and head (temperature and liquid fraction outputs)	41
Fig. 4	U-DeepONet architecture showing the branch and the trunk networks.	44
Fig. 5	Heat source at left and liquid fraction domain at a given time snapshot in 3D at right having 2D gaussian boundary conditions	54
Fig. 6	Slice extraction strategy for the proposed slice-based 3D U-DeepONet framework based on the decomposition of the three-dimensional PCM domain using the XY, XZ, and YZ-planes	59
Fig. 7	Encoding boundary conditions for a 2D slice with boundary-value and boundary distance channels.	60
Fig. 8	Representation of the multi-channel input tensor for the slice-based 3D U-DeepONet framework.	61
Fig. 9	Temperature maps when a constant heat source is applied everywhere inside the PCM block for mushy condition.	69
Fig. 10	Temperature maps with boundary heating where all sides are held hot at 350 K.	70
Fig. 11	Temperature maps with a randomly varying heat source generated via GPR.	71
Fig. 12	Temperature maps for volumetric heat source scaled by 5×10^4 .	73

Fig.No.	Caption	Page No.
Fig. 13	Boundary Temperature cases for the samples shown; (a) constant 350 K on all sides, (b) constant 350 K on three sides and right-side boundary conditions given using gaussian profile, varying from 380 K to 410 K	75
Fig. 14	Distribution of mean temperature across all 200 simulation cases for each time snapshot. Each curve represents the spread of mean temperatures at a specific time	78
Fig. 15	Distribution of maximum temperature across cases at different time snapshots.	78
Fig. 16	Distribution of mean liquid fraction across all 200 cases at each time snapshot.	79
Fig. 17	Volumetric heat source, temperature field and liquid fraction maps for prismatic battery as heat source	80
Fig. 18	Volumetric heat source, temperature field and liquid fraction maps for rotated prismatic battery as heat source.	81
Fig. 19	Generalised Volumetric heat source, temperature field and liquid fraction maps for prismatic battery, circular and elliptic battery as heat sources along with rotation.	81
Fig. 20	Training and validation loss history of the 1D-DeepONet model, showing convergence with epochs	82
Fig. 21	Training MSE vs Epoch for the 2D Temperature DeepONet.	83
Fig. 22	Training RMSE vs Epoch for the 2D Liquid Fraction U-DeepONet for different training set sizes.	85
Fig. 23	Best Validation RMSE vs Training set size for the 2D Liquid Fraction U-DeepONet as per Table 4	87
Fig. 24	True and Predicted liquid fraction fields for a given set of conditions.	88
Fig. 25	Training mean squared error of the U-DeepONet for temperature prediction over 60 epochs, for different numbers of training simulations.	89

Fig.No.	Caption	Page No.
Fig. 26	Validation mean squared error for temperature prediction, for the same training-set sizes as Fig. 25.	90
Fig. 27	Training mean squared error of the U-DeepONet for liquid-fraction prediction over 60 epochs, for different numbers of training simulations.	90
Fig. 28	Validation mean squared error for liquid-fraction prediction, for the same training-set sizes as Fig. 27.	91
Fig. 29	Comparison of predicted and ground-truth 1D temperature fields for four representative test samples.	93
Fig. 30	Comparison of true and predicted temperature fields in U-DeepONet model	97
Fig. 31	Case-1 of comparison of predicted and ground-truth 2D liquid fraction fields for (100, 500, 900, 1300, 1600, 2000 and 3500s).	98
Fig. 32	Training and Validation (normalized) RMSE history for 3D U-DeepONet Temperature Surrogate model	101
Fig. 33	Training and Validation (physical) RMSE history for 3D U-DeepONet Temperature Surrogate model	101
Fig. 34	2D gaussian boundary conditions across left, right, front, back, bottom and top faces as per the used orientation for prediction of temperature plots across different slices	102
Fig. 35	Comparison of predicted and ground-truth 3D slice-based temperature fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s).	104
Fig. 36	Comparison of predicted and ground-truth 3D slice-based temperature fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s).	106
Fig. 37	2D gaussian boundary conditions across left, right, front, back, bottom and top faces as per the used orientation for prediction of liquid fraction plots across different slices	108

Fig.No.	Caption	Page No.
Fig. 38	Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s).	110
Fig. 39	Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s).	112
Fig. 40	Two-dimensional temperature fields imposed on the six bounding faces of the domain for the representative three-dimensional prismatic battery test case.	115
Fig. 41	Reference and predicted three-dimensional slice-based temperature fields for x-normal plane 4 (yz-plane), with $R^2 = 0.9672$ over the full space–time field of the plane.	116
Fig. 42	Reference and predicted three-dimensional slice-based temperature fields for y-normal plane 5 (xz-plane), with $R^2 = 0.9651$ over the full space–time field of the plane.	119
Fig. 43	Reference and predicted three-dimensional slice-based temperature fields for z-normal plane 4 (xy-plane), with $R^2 = 0.9879$ over the full space–time field of the plane.	122
Fig. 44	Reference and predicted three-dimensional slice-based liquid-fraction fields for x-normal plane 4 (yz-plane), with $R^2 = 0.9384$ over the full space–time field of the plane.	126
Fig. 45	Reference and predicted three-dimensional slice-based liquid-fraction fields for y-normal plane 5 (xz-plane), with $R^2 = 0.9469$ over the full space–time field of the plane.	129
Fig. 46	Reference and predicted three-dimensional slice-based liquid-fraction fields for z-normal plane 4 (xy-plane), with $R^2 = 0.9779$ over the full space–time field of the plane.	132
Fig. 47	Cross-geometry generalisation to unseen circular heat sources, predicted by the model trained exclusively on prismatic sources.	137
Fig. 48	Cross-geometry generalisation to unseen elliptical heat sources of differing aspect ratio and orientation, predicted by the model trained exclusively on prismatic sources.	138

LIST OF TABLES

Table.No.	Caption	Page No.
Table. 1	U-DeepONet architecture.	49
Table. 2	Slice-Based 3D U-DeepONet architecture.	52
Table. 3	Parameters of the three-dimensional melting and solidification model and of the generated dataset.	67
Table. 4	Best Validation RMSE on normalized data for different training set sizes which were trained on U-DeepONet model architecture	86
Table. 5	Best training and validation mean squared error obtained by the U-DeepONet for liquid-fraction prediction over 60 epochs, for different numbers of training simulations.	92
Table. 6	Comparison of observed stats of Pointwise and U-DeepONet model architecture	96
Table. 7	Slice-wise temperature prediction accuracy of the slice-based three-dimensional U-DeepONet for the prismatic battery test case.	125
Table. 8	Slice-wise liquid-fraction prediction accuracy of the slice-based three-dimensional U-DeepONet for the same planes.	135
Table. 9	Cross-geometry generalisation performance of the U-DeepONet for unseen circular and elliptical heat-source configurations, comparing a model trained only on prismatic sources with one trained on a geometrically generalised dataset.	139
Table. 10	Evaluation cost of the numerical solver and of the U-DeepONet surrogate in two and three dimensions.	140

CHAPTER 1:

INTRODUCTION

1.1 Theoretical Background

1.1.1 Heat transfer and PCM physics

Phase change materials (PCMs) store and release heat by melting and solidifying. They provide high energy density because they absorb latent heat during phase transition, while their temperature remains nearly constant. This makes them useful for thermal energy storage, since they can regulate temperature and store large amounts of energy in small volumes. However, most common PCMs such as paraffin or salt hydrates have very low thermal conductivity, typically around 0.2–0.5 W/m·K. This slows down charging and discharging, since heat cannot spread quickly through the material.

To improve performance, researchers have embedded conductive additives such as open-cell metal foams. These foams are porous metallic structures with high conductivity and large surface area. When a PCM fills the pores, the metallic network provides short conduction paths and distributes heat more uniformly. This increases the effective conductivity of the system by several times, although it reduces the total PCM volume and can suppress natural convection in the liquid phase. The trade-off is usually favorable in applications where quick charging and discharging are more important than the maximum storage capacity ^[1].

1.1.2 Surrogate modeling and DeepONet background

The physics of PCM melting and freezing involves nonlinear partial differential equations that describe conduction, convection and phase change. Traditional numerical methods such as finite volume or finite element solvers can simulate these processes, but they are computationally expensive. Each design or operating condition requires a new simulation, which is impractical for optimization or real-time use.

Surrogate modeling with machine learning offers an alternative. Deep Operator Networks (DeepONets) are a recent development that learn mappings between input and output functions rather than only single input–output pairs. In the PCM context, the input could be a heat source profile and design parameters, while the output is the temperature or liquid fraction field. The DeepONet architecture has two components: a branch network that encodes the global inputs such as heat source and geometry, and a trunk network that encodes the local spatial or temporal coordinates. Their outputs are combined to predict the field at any queried location. This makes DeepONet fast once trained, suitable for design sweeps and optimization [2].

1.1.3 Localised heating and battery thermal management

The heat sources considered in most surrogate studies of conduction are smooth and distributed. Practical thermal management problems seldom are. In a battery pack the heat is generated inside discrete cells occupying a small fraction of the enclosure, at rates an order of magnitude above anything in the surrounding material, so the temperature field is governed by steep gradients at the cell perimeters and by the interaction between neighbouring cells. When the enclosure is filled with a PCM, melt fronts nucleate around each cell and subsequently merge, giving a moving interface whose topology changes during the transient.

Such configurations combine several difficulties at once: a nonlinear energy equation, a diffuse and moving solid–liquid interface, a discontinuous input field, and a strong dependence on the geometry, position and orientation of the sources. They are also the configurations of most engineering interest, and they motivate the two questions this thesis adds to the underlying surrogate problem: whether an operator trained on one family of source shapes transfers to shapes it has never seen, and whether the approach can be carried from a plane into a genuine three-dimensional enclosure at acceptable cost.

1.2 History of Development

1.2.1 PCM-based systems

Research on PCMs for solar heating and building thermal control dates back to the mid-20th century. By the early 2000s, researchers began incorporating metal foams into PCMs to address their low thermal conductivity. Studies by Siahpush et al. and Zhao et al. demonstrated that embedding copper or aluminum foams into paraffin increased conductivity from about 0.4 to 3 W/m·K, reducing charging and discharging times significantly. Later work by Xiao et al. and Lafdi et al. showed how pore size and porosity influence performance, with denser foams providing faster conduction but reducing PCM volume. Modeling approaches developed in this period included volume-averaged models, which capture bulk behavior at low computational cost, and pore-scale simulations, which resolve foam geometry in detail but are computationally heavy ^[1].

1.2.2 Machine learning and neural operators

Parallel to material advances, machine learning methods for physical modeling evolved rapidly. Physics-informed neural networks (PINNs) introduced by Raissi et al. (2019) incorporated governing equations into the loss function, reducing dependence on data but requiring complex training ^[3]. Li et al. (2021) developed the Fourier Neural Operator (FNO), which uses Fourier layers to capture global correlations and solve parametric PDEs efficiently^[4].

Lu et al. (2021) proposed DeepONet, based on the operator approximation theorem, showing that neural networks can approximate nonlinear operators. Its branch–trunk structure provided a practical way to learn mappings from boundary or source functions to solution fields ^[2]. Despite widespread applications in fluid mechanics and material science, DeepONet has not yet been widely applied to PCM–foam thermal storage. This gap motivates the present work.

1.2.3 Grid-based operator learning

A limitation of the original branch–trunk construction became apparent as it was applied to spatially structured problems. Encoding the input field as a flattened vector discards the relationship between neighbouring grid points, so a fully connected branch network must learn spatial continuity through dense matrix multiplications instead of receiving it as an architectural prior. For heat conduction, where the local gradient is governed by the immediate neighbourhood of each point, this discards the strongest available prior.

Encoder–decoder operators address this directly. The U-Net of Ronneberger et al. (2015) ^[16], developed for biomedical image segmentation, extract features at several scales by successive downsampling and restores resolution by upsampling, with skip connections that preserve fine detail. Diab and Al-Kobaisi placed such an encoder in the branch position of a deep operator network, producing the U-DeepONet ^[15]. The resulting grid-to-grid formulation treats each simulation as a single training sample rather than as a collection of space–time points, which removes the need for point sampling and supplies the spatial inductive bias that localised gradients and sharp moving interfaces require. This architecture is the one developed and extended in the present work.

1.3 Applications

Deep Operator Networks (DeepONets) are being used in many areas, where solving physics equations directly is slow. In fluid mechanics, they can learn the link between inputs like boundary conditions and outputs like velocity or temperature fields, avoiding repeated Navier–Stokes simulations ^[2]. They also extend the idea of physics-informed neural networks (PINNs) to provide fast, mesh-free solutions for forward and inverse problems ^[2,3]. For systems with changing parameters or uncertainty, DeepONets have been applied to parametric PDEs and even hysteretic systems, offering quick predictions that are still accurate ^[4,11]. They are also useful when data is limited, since special data-generation schemes can be used to train them as surrogates for engineering systems ^[12].

Applications closer to the present problem have begun to appear. Operator surrogates have been demonstrated for parametric heat conduction with distributed sources ^[17], for the Stefan problem and other moving-interface formulations ^[18], for phase-field solidification

and gradient-flow dynamics ^{[19] [20]}, and for melting in porous media ^[21]. Taken together these establish that the transient behaviour of a phase-change system is learnable as an operator, which is the premise on which this work proceeds.

1.4 Advantages and Disadvantages of DeepONet for PCM

Advantages

DeepONet provides very fast predictions after training, compared to CFD or FEM solvers. It outputs full temperature or liquid fraction fields at any point in the domain and is not tied to a fixed mesh. It can generalize across different input conditions within the training range, making it suitable for optimization and parametric studies ^[2].

Disadvantages

The approach requires a representative training dataset, which may involve many high-fidelity simulations. Sensor placement and encoding choices affect accuracy, and predictions outside the training distribution may not be reliable. DeepONet also functions as a black box, offering less interpretability than traditional models ^[2-4].

Two of these limitations are addressed directly in this thesis. The cost of the training dataset is quantified rather than assumed, through a sensitivity study that identifies the number of high-fidelity simulations beyond which further data yields diminishing returns. The reliability of predictions outside the training distribution is examined through a cross-geometry study, in which a model trained on one family of source shapes is tested on shapes absent from its training data. The grid-to-grid formulation also removes the sensor-placement question entirely, since the input field is presented to the network whole rather than sampled at chosen points.

1.5 Scope and Organisation of the Thesis

This thesis develops a deep operator network surrogate for transient melting and solidification in two- and three-dimensional PCM domains subjected to distributed and localised internal heat generation together with non-uniform boundary heating. The work continues a study begun at the undergraduate level, in which the pointwise and grid-to-grid architectures were developed and validated for the two-dimensional problem. Three additions distinguish the present work: a substantially enlarged and geometrically diversified training dataset with a systematic study of the sensitivity to its size, a cross-geometry generalisation study that tests the operator on source shapes absent from its training data, and an extension of the framework to three dimensions through a slice-based formulation applied to a prismatic battery heat-source configuration.

Chapter 2 reviews the literature and states the objectives and work plan. Chapter 3 develops the models, from the enthalpy-based formulation and the Gaussian-process data generation scheme through the pointwise DeepONet and the grid-to-grid U-DeepONet to the three-dimensional slice-based architecture. Chapter 4 presents the dataset characteristics, the model performance and data-sensitivity study, the two- and three-dimensional prediction results, the cross-geometry generalisation study and the computational cost. Chapter 5 states the conclusions and the directions in which the work can be taken further.

CHAPTER 2:

LITERATURE REVIEW & FORMULATION

2.1 Literature Review

2.1.1 PCM–Problem Studies

Research on PCM–composites has consistently highlighted the benefit of improved heat transfer. Krishnan et al. (2005) developed an early volume-averaged model and showed that metal foams accelerate PCM melting by providing high-conductivity pathways ^[5]. Yang & Garimella (2010) extended this with improved treatment of PCM volume change ^[6]. Ren et al. (2017) carried out pore-scale simulations to capture melting dynamics at the pore level, including the effects of porosity and pore size ^[7]. Abishek et al. (2018) confirmed experimentally and numerically that the surface area of the foam is a key parameter in determining melt rate ^[8]. These studies form the physical basis for PCM–foam system modeling.

Subsequent work refined the description of the composite at finer scales. Ren, Chan and Ding ^[7] performed pore-scale simulations of melting in open-cell foams and resolved the local interaction between the ligaments and the advancing front, which volume-averaged models represent only in an effective sense. Abishek, Verma and Singh ^[8] quantified the influence of foam surface area on the melting rate, and Chibani and co-workers ^[14] examined additively manufactured composite structures in which porosity and pore size are controlled directly. Across this literature the physical conclusions are consistent, and the methodological point relevant here is a different one: every result was obtained from a high-fidelity transient simulation, and the cost of those simulations is what limits how much of the design space can be explored.

2.1.2 Data-Driven Models for PCM

Machine learning approaches have also been tested for phase change problems. Prasad et al. (2021) used a convolutional neural network (CNN) to predict melting dynamics from

image-like inputs, though it required large datasets ^[9]. Zhou et al. (2022) developed an autoencoder–LSTM model to predict PCM heating behavior, which performed well but was highly data-intensive ^[10]. These models demonstrated potential but highlighted the need for more data-efficient and generalizable approaches.

Both of these approaches share a structural limitation. A convolutional or recurrent network maps a fixed input discretisation to a fixed output discretisation, so a model trained on one grid cannot be queried on another, and neither formulation learns the solution operator itself: it learns one particular input–output correspondence. The physics-informed neural network of Raissi and co-workers ^[3] reduces the data requirement by penalising the residual of the governing equation at collocation points, but the object it produces solves a single problem instance rather than a family of them. For a surrogate that must answer many different heating and boundary conditions, this is the wrong object to learn.

2.1.3 Operator Learning Models

Recent advances in neural operators offer a new path. Raissi et al. (2019) introduced physics-informed neural networks (PINNs), embedding PDE residuals in the loss to reduce data need ^[3]. Li et al. (2021) proposed Fourier Neural Operators (FNOs), which use Fourier layers to capture long-range correlations and efficiently solve PDE families ^[4].

Lu et al. (2021) introduced Deep Operator Networks (DeepONets), which directly learn mappings between input and output functions, using a branch–trunk architecture ^[2]. Subhayan et al. (2024) further extended DeepONet to handle hysteretic systems under uncertainty with a bi-fidelity approach ^[11].

The theoretical basis for this class of model predates it by two decades. Chen and Chen ^[22] proved that continuous non-linear operators between Banach spaces can be approximated to arbitrary accuracy by feed-forward architectures with non-polynomial activations under suitable compactness assumptions. The result remained largely theoretical until it became the foundation of modern operator learning, first in the original DeepONet formulation ^[23] and then in its expanded form ^[2].

Several architectures appeared in quick succession alongside it. Li and co-workers introduced the graph kernel network ^[24], which approximates the kernel integral of a partial differential operator using graph neural networks and was among the first formulations able to generalise across discretisations of the input function. The multipole graph neural operator ^[25] followed, incorporating multi-level long-range interactions at almost linear computational complexity. The Fourier neural operator ^[4] parametrises the integral kernel directly in the Fourier domain, so that global feature propagation is achieved through the fast Fourier transform, and reached state-of-the-art accuracy on benchmark problems including Darcy flow and the incompressible Navier–Stokes equations.

The theory kept pace with the architectures. Kovachki, Lanthaler and Mishra ^[26] established the universality of Fourier neural operators and their approximation properties in Sobolev spaces, derived explicit error and complexity bounds, and generalised the various constructions into a unified neural-operator framework with a proof of discretisation invariance. Deng and co-workers ^[27] studied the approximation rates of deep operator networks for Hölder continuous operators and for solution operators of advection–diffusion–reaction systems, and showed that structured solution operators can escape the curse of dimensionality under suitable assumptions, a result of direct relevance to surrogate modelling of governed physical systems.

2.1.4 Applications of Surrogates in Thermal Systems

While neural operators have been applied to fluid mechanics and structural mechanics, their use in PCM–foam thermal storage is still missing. Some works have proposed surrogate data generation for training operators. For instance, Choubey et al. (2023) introduced a novel data generation scheme for training DeepONet in surrogate modeling ^[12].

Gaussian Process Regression (GPR) has also been used as a tool for smooth field generation, offering an alternative to standard random fields when sampling temperature or heat source inputs ^[13]. These methods show how data generation and operator learning can combine to enable efficient surrogate modelling.

A large family of modifications to the basic architecture has since been proposed, each addressing a particular weakness. Physics-informed deep operator networks ^[28] incorporate the governing equations into the training objective and make operator learning feasible in

limited-data regimes; the physics-informed neural operator ^[29] combines operator learning with optimisation at test time for transient systems. Multi-fidelity formulations ^[30] couple cheap low-fidelity data with sparse high-fidelity data to reduce the number of expensive simulations required. Bayesian variants ^[31] supply uncertainty estimates and improve robustness to noisy training data. Derivative-enhanced networks ^[32] and Gaussian-process-assisted operators ^[33] improve accuracy and stability, while reduced-basis and principal-component formulations ^[34] offer efficiency for problems with low-rank solution structure, although they are less suited to strongly non-linear problems with moving interfaces or changing topology.

2.1.5 Grid-Based Operator Learning and the U-DeepONet

The pointwise branch–trunk construction encodes the input field as a flattened vector, which is a poor representation for a problem whose physics is local. Conduction is governed by the immediate neighbourhood of each point, and a fully connected branch must learn spatial continuity through dense matrix multiplications instead of receiving it as an architectural prior. Encoder–decoder operators address this directly. The U-Net of Ronneberger, Fischer and Brox ^[16], originally developed for biomedical image segmentation, extract features at multiple scales by successive downsampling and restores resolution by upsampling, with skip connections that preserve fine detail across the bottleneck.

Diab and Al-Kobaisi ^[15] placed such an encoder in the branch position of a deep operator network, producing the U-DeepONet. The resulting grid-to-grid formulation treats each simulation as a single training sample rather than as a collection of space–time points, which removes the need for point sampling and supplies the spatial inductive bias that localised gradients and sharp moving interfaces require. The authors reported improved data efficiency, faster training and higher accuracy than several competing neural-operator architectures including U-FNO and Fourier-MIONet. These are precisely the properties needed for phase change with discrete internal sources, and the U-DeepONet is therefore the architecture developed in this work.

2.1.6 Operator Learning for Phase-Change Problems

Applications of operator learning to moving-interface problems are comparatively recent. Wang and co-workers ^[18] addressed the Stefan problem with physics-informed networks and documented the difficulties that free-boundary motion presents to neural solvers. Li, Bazant and Zhu ^[19] proposed the phase-field DeepONet for the Allen–Cahn and Cahn–Hilliard equations, treating the neural operator as an explicit time-stepping scheme for gradient-flow dynamics. Ciesielski and co-workers ^[20] used deep operator approximations for phase-field solidification microstructure evolution and reported low cumulative error over long predictions. Mao and co-workers ^[21] applied Fourier neural operators to the melting of gallium in porous media, and a recent multi-head physics-informed formulation ^[35] has extended operator learning to porous composite storage media across multi-dimensional parameter spaces.

In the neighbouring problem of parametric heat conduction without phase change, Koric and co-workers ^[17] demonstrated deep operator surrogates for parametric heat sources. What none of these studies addresses is an enthalpy-based melting and solidification model driven by localised, battery-representative volumetric sources, which is the configuration of most engineering interest and the one treated here.

2.1.7 Geometry Generalisation and Three-Dimensional Operator Learning

A surrogate is useful only to the extent that it remains reliable for inputs it has not seen. Two limitations are recognised in the literature. The first concerns the geometry of the domain: Fourier-based operators assume a structured grid, and the geometry-informed neural operator ^[36] was introduced to extend operator learning to unstructured meshes and irregular domains. The second concerns the input function itself. Extrapolation beyond the training distribution remains unreliable, particularly for systems with strongly localised source terms, discontinuities and moving interfaces, and physics-assisted and sparse-data-assisted strategies have been proposed to make it more dependable ^[37].

The question examined in this thesis sits between the two. The domain geometry is fixed and the grid is structured; what changes is the shape of the region within the domain in which heat is generated. Whether an operator trained on one class of source shapes transfers to another has not, to the author's knowledge, been examined for phase-change problems,

and it bears directly on how a fixed simulation budget should be spent when building a training dataset.

Extension to three dimensions raises a further difficulty that is practical rather than conceptual. Practical storage units and battery packs conduct heat in all three directions, so that melt fronts advance non-uniformly and corner effects, side-wall heating and the interaction of internal hot spots cannot be captured within a single plane. Learning the operator directly on the volumetric field is severely constrained by cost, however, since both the stored data and the parameter count and memory footprint of volumetric convolutional encoders grow rapidly with resolution. A formulation that retains the essential three-dimensional behaviour at two-dimensional cost has not been demonstrated for this class of problem.

2.2 Literature Gap

From the reviewed studies, three gaps are clear:

- PCM–foam modeling has been widely explored, but mostly with volume-averaged or pore-scale CFD approaches, which are accurate but computationally expensive.
- Machine learning has been applied to phase change, but the models so far (CNN, LSTM, Autoencoders) are data-hungry and lack generalization.
- Neural operator methods like DeepONet and FNO have proven effective in other domains, but limited study has been done in the field of operator learning for complex heating, boundaries, interfaces, temperature and liquid-fraction fields.

Two further gaps follow from the work reviewed above and are addressed by the present study. Neural operators have proved effective in fluid and solid mechanics and in parametric heat conduction, but the behaviour of such an operator when presented with source geometries absent from its training data has not been established for phase-change problems, and that behaviour determines how a fixed simulation budget should be allocated across source shapes.

Operator-learning treatments of phase change are also almost entirely two-dimensional, and the direct volumetric extension is limited by the cost of three-dimensional convolutional encoders. A formulation that retains the essential three-dimensional behaviour while keeping the convolutions two-dimensional has not been demonstrated for melting and solidification with localised internal heating.

2.3 Objective Formulation

- Develop a DeepONet surrogate model to map global heating and foam design conditions to temperature and liquid fraction fields in PCM–foam composites.
- Demonstrate how operator learning can replace slow CFD/FEM solvers with fast surrogates.
- Validate DeepONet predictions against simulation data and explore its use for parametric studies and optimization.
- Quantify the sensitivity of the surrogate to the size of the training dataset, and establish the point beyond which additional high-fidelity simulations yield diminishing returns.
- Examine cross-geometry generalisation by testing a model trained exclusively on prismatic battery sources against unseen circular and elliptical configurations, and by comparing it against a model trained on a geometrically generalised dataset, for both the temperature and the liquid-fraction fields.
- Extend the framework to three dimensions by developing a three-dimensional melting and solidification solver with a prismatic battery heat source, and by predicting the resulting volumetric fields with a slice-based U-DeepONet, evaluating both the slice-wise accuracy and the inference cost.

2.4 Methodology / Work Plan

- *Literature Review*: Survey PCM composites, and operator learning methods.
- *Data Generation*: Use Gaussian Process Regression (GPR) to create smooth heat source inputs; generate PCM melting data using enthalpy-based numerical models.
- *Model Development*: Implement DeepONet with branch–trunk architecture, trained on generated datasets.
- *Validation*: Compare DeepONet outputs with numerical simulation results.
- *Applications*: Test surrogate model for parametric sweeps and optimization of geometry and heating protocols

CHAPTER 3:

MODEL DEVELOPMENT

3.1 Physical Model for PCM–System

The study is based on the enthalpy–porosity method for modeling melting/solidification in PCM–composites [5]. The foam provides high conductivity paths and modifies the effective thermal properties of the system. The governing equations are:

- *Energy equation (enthalpy form):*

$$H = [C_{p,pcm}(1 - \phi) + C_{p,m}\phi]T + (1 - \phi)f_l L \quad (\text{Eqn. 1})$$

where H is the enthalpy of the composite, $C_{p,pcm}$ is the specific heat of PCM, ϕ is the porosity of the foam, T is the local temperature, f_l is the liquid fraction (ranging from 0 for solid to 1 for liquid), and L is the latent heat of fusion.

- *Liquid fraction update:*

$$f_{l,new} = \frac{H - [C_{p,pcm}(1 - \phi) + C_{p,m}\phi]T_m}{(1 - \phi)L} \quad (\text{Eqn. 2})$$

where $f_{l,new}$ is the updated melt fraction and T_m is the melting temperature of the PCM.

- *Heat conduction with source term:*

$$\frac{\partial H}{\partial t} = k_{avg} \nabla^2 T + Q \quad (\text{Eqn.3})$$

where $\frac{\partial H}{\partial t}$ is the rate of enthalpy change with time, k_{avg} is the effective thermal conductivity of the PCM–composite, $\nabla^2 T$ is the Laplacian of temperature field representing spatial diffusion, and Q is the volumetric heat source. Boundary conditions are specified temperatures at domain edges, with transient update by FTCS scheme.

This formulation captures the coupling between temperature evolution and liquid fraction mapping, which forms the training dataset for surrogate development.

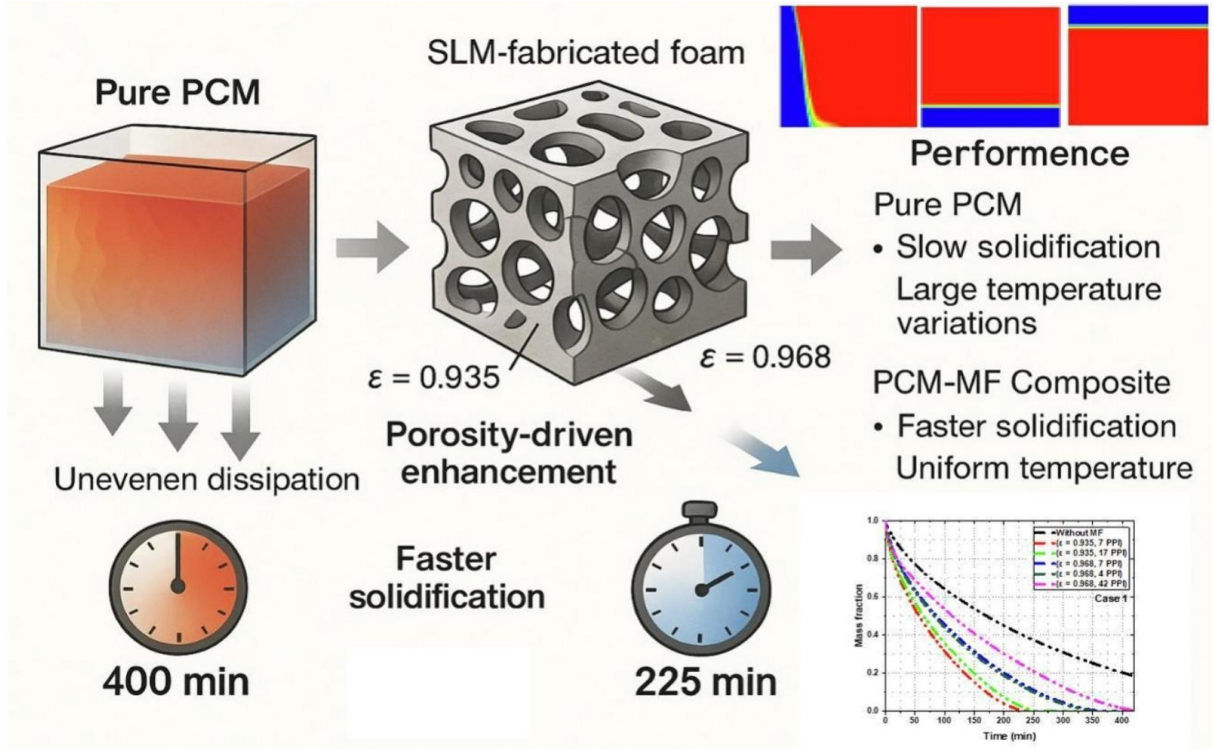


Fig. 1: Schematic of PCM–metal foam composite showing heat conduction paths through foam ligaments and PCM pores

Now, we present the algorithm using the same notations and terms as before:

3.1.1 Algorithm

1. Initialize the enthalpy field:

$$H = [C_{p,pcm}(1 - \phi) + C_{p,m}\phi]T + (1 - \phi)f_lL \quad (\text{Eqn. 4})$$

2. Enforce Boundary Conditions:

Set boundary temperature:

$$T = T_b$$

Compute boundary enthalpy:

$$H = C_{p,pcm}T_b + f_lL_{lat} \quad (\text{Eqn. 5})$$

3. Compute Temperature-spatial derivatives:

2D-Laplacian computation:

$$\nabla^2 T = \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} \quad (\text{Eqn. 6})$$

4. Update Enthalpy:

Explicit FTCS Scheme:

$$H^{new} = H^{old} + \Delta t \left[\frac{k_{avg}}{\rho_{avg}} \nabla^2 T + \frac{Q}{\rho_{avg}} \right] \quad (\text{Eqn. 7})$$

5. Calculating Liquid Fraction:

$$f_{l,new} = \frac{H - [C_{p,pcm}(1-\phi) + C_{p,m}\phi]T_m}{(1-\phi)L} \quad (\text{Eqn. 8})$$

6. Phase Change Rules:

Case-1: Mushy Zone: $0 < f_{l,new} < 1$

$$T = T_m$$

$$f_l = f_{l,new}$$

Case-2: Solid Phase: $f_{l,new} \leq 0$

$$f_l = 0 \text{ (no liquid fraction)}$$

$$T = \frac{H}{C_{p,pcm}(1-\phi) + C_{p,m}\phi} \quad (\text{Eqn. 9})$$

Case-2: Liquid Phase: $f_{l,new} \geq 1$

$$f_l = 1 \text{ (Complete liquid)}$$

$$T = \frac{H - (1-\phi)L}{C_{p,pcm}(1-\phi) + C_{p,m}\phi} \quad (\text{Eqn. 10})$$

3.2 Data Generation with Gaussian Process Regression (GPR)

3.2.1 Kernel function

Training DeepONet requires diverse heat source functions as input. Instead of arbitrary waveforms, Gaussian Process Regression (GPR) is used to generate smooth, spatially correlated heat sources ^[13].

- **Kernel function:** Squared exponential (RBF) kernel controls smoothness:

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x-x')^2}{2l^2}\right) \quad (\text{Eqn. 11})$$

- Parameters: l (length scale) controls smoothness of heat source; σ controls amplitude.
- Larger l yields smoother fields, while larger σ increases amplitude.
- $k(x, x')$ is the covariance between points x and x'
- This ensures random but physically plausible input heat profiles, forming a representative training set.

3.2.2 Kernel matrices

For grid points X and boundary (anchor) points X_b , the following covariance matrices between points are built:

$$K_{xx} = K(X, X), K_{xb} = K(X, X_b), K_{bb} = K(X_b, X_b) \quad (\text{Eqn. 12})$$

3.2.3 Posterior mean

The GPR mean field that satisfies boundary conditions (e.g. fixed $T = 0$ at domain ends) is:

$$T_{mean} = K_{xb} K_{bb}^{-1} T_b \quad (\text{Eqn. 13})$$

where T_b are prescribed boundary values. T_{mean} is the posterior mean field, basically the best guess temperature distribution inside the domain, given the boundary or anchor values. In other words, T_{mean} is the smooth temperature field predicted by Gaussian Process

Regression that exactly matches the boundary conditions and smoothly extends them throughout the domain.

3.2.4 Posterior covariance

The uncertainty after conditioning on boundaries is:

$$K_{post} = K_{xx} - K_{xb}K_{bb}^{-1}K_{xb}^T \quad (\text{Eqn. 14})$$

where K_{post} stands for posterior covariance matrix.

3.2.5 Field realization

A sample but realistic field is generated by adding fluctuations using:

$$T = T_{mean} + L_{post}z \quad (\text{Eqn. 15})$$

where L_{post} is the Cholesky factor of K_{post} and z is a standard Gaussian random vector.

This produces smooth, spatially correlated heat source functions that both match physical boundary conditions and introduce variability for training DeepONet.

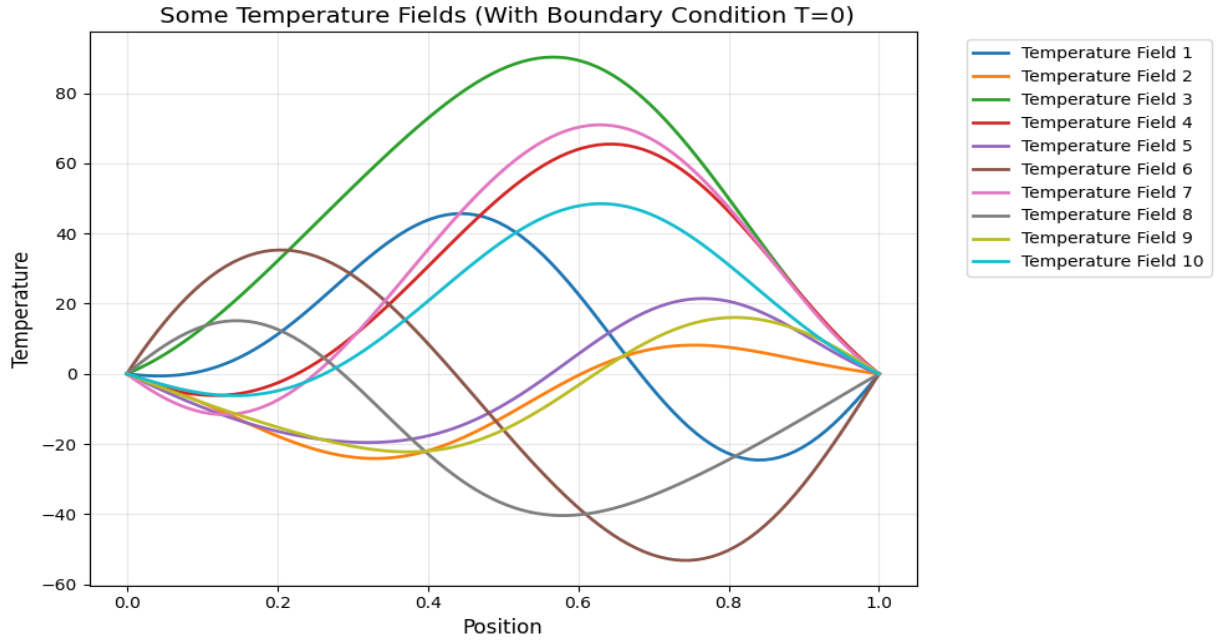


Fig. 2(a): Temperature functions generated using Gaussian Process Regression (GPR) in 1D. Smoothness controlled by kernel length scale l and variance σ

To adapt the 1D GPR scheme for practical PCM–simulations like in Fig. 2(a), the Gaussian Process was further developed on a two-dimensional grid. The same squared-exponential (RBF) kernel is used, but it is now applied to (x, y) coordinates to create smooth heat-source fields that are spatially correlated.

Fig. 2(b) and 2(c) show examples of these kinds of 2D fields. In Fig. 2(b), the boundary is held at $Q = 0$, causing the GP to produce smooth interior variations that taper to zero along the edges. Fig. 2(c) shows a more heterogeneous case where the boundary values vary, producing stronger gradients and multiple hot-spot regions. These 2D GPR fields form the diverse input space required to train the DeepONet surrogate on realistic heating conditions.

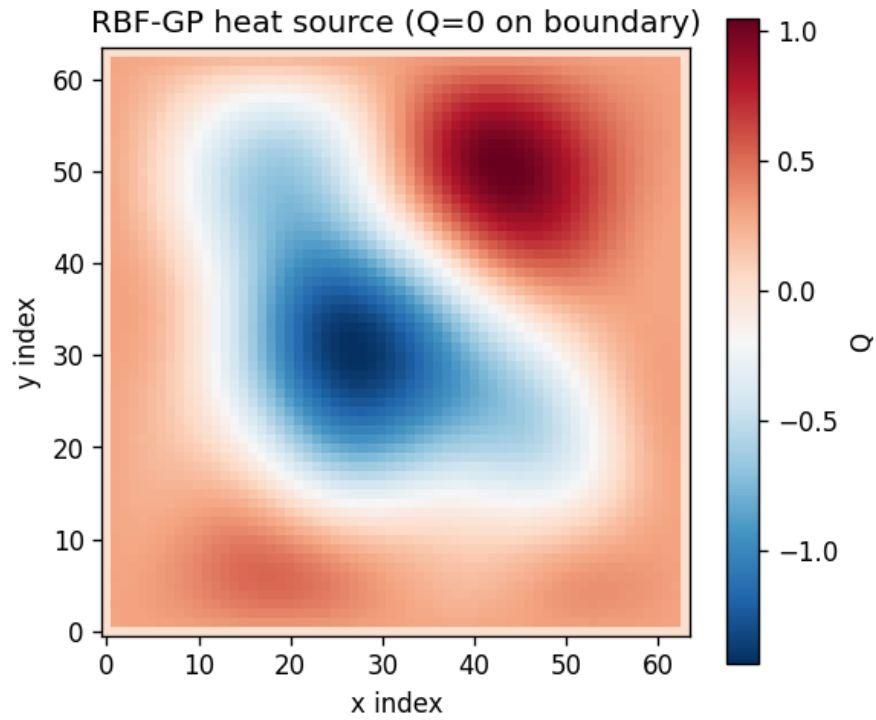


Fig. 2(b): Two-dimensional heat-source field generated using RBF-kernel Gaussian Process Regression (GPR), with the boundary constrained to $Q = 0$.

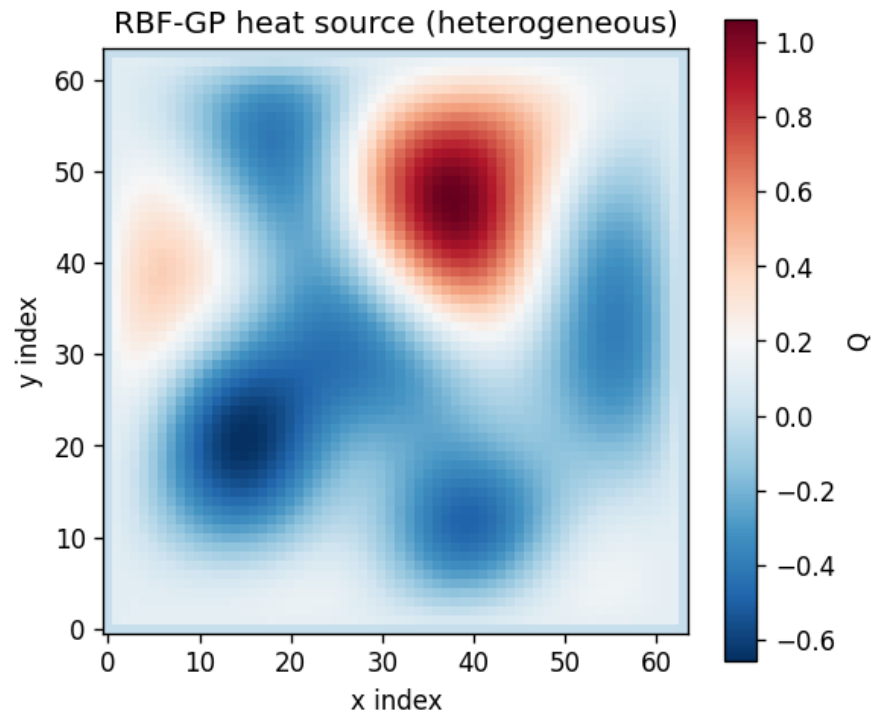


Fig. 2(c): RBF-GPR heat-source field with heterogeneous boundary constraints. Spatial variation is stronger compared to Fig. 2(b), producing multiple local hot-spots and cold pockets.

3.3 DeepONet Pointwise Surrogate Development

Following ^[12], the DeepONet surrogate is built with branch and trunk networks:

- Branch net encodes heat source functions and design parameters.
- Trunk net encodes query coordinates (x, y, t) .
- Outputs are combined to give local temperature.

3.3.1 1D-Architecture

- Branch net: 2 hidden layers, 128 neurons each.
- Trunk net: same structure.
- Activation: ReLU.
- Head: inner product + bias.

3.3.2 Training details

- Data: 500 heat source–temperature pairs.
- Batch size: 512, epochs: 200.
- Optimizer: Adam, learning rate 0.001.
- Loss: mean squared error (MSE).

3.3.3 Inference

This simple form was sufficient for learning 1D temperature profiles generated using Gaussian Process Regression (GPR). The outputs were smooth and accurate, demonstrating the feasibility of operator learning for PCM systems.

3.3.4 Extension to 2D Architecture

To model full two-dimensional transient PCM–dynamics, the DeepONet architecture is extended considerably. The 2D system depends not only on the spatially varying heat source but also on boundary heating, geometry, and the evolution of temperature over time. Therefore, a richer neural operator structure is required.

(a) Temperature Surrogate: 4-Network DeepONet

The temperature model predicts

$$T(x, y, t) = \mathcal{G}(Q(x, y), \text{BC}; x, y, t), \quad (\text{Eqn. 16})$$

where $Q(x, y)$ is the heat-source field, BC is a vector describing boundary temperatures and (x, y, t) are the spatial and temporal coordinates of the given domain. The full 4-stream architecture consists of:

Branch Networks

1. Q-Branch:

- Inputs: flattened heat-source field $Q(x, y)$
- Purpose: encodes global heating pattern
- Structure: 2 hidden layers (256 neurons each, GELU activation)

2. BC-Branch:

- Inputs: boundary-condition feature vector (Gaussian parameters, wall temperatures, etc.)
- Purpose: captures global boundary constraints
- Structure: similar MLP with 128–256 neurons

Trunk Networks

3. XY-Trunk:

- Inputs: spatial coordinates (x, y) , normalized to $[0,1]$
- Purpose: encodes spatial variation
- Structure: 2 hidden layers (256 neurons each)

4. Time-Trunk:

- Inputs: time coordinate t , scaled to $[0,1]$
- Purpose: encodes temporal evolution of melting
- Structure: 2 hidden layers (128 neurons each)

Head (Feature Fusion)

The branch and trunk latent vectors are combined using two parallel mechanisms:

- Additive path: addition of latent vectors between branch and trunk
- Multiplicative path: dot product of vectors between branch and trunk

Weighted combination:

$$T = \alpha_{\text{add}} T_{\text{add}} + \alpha_{\text{prod}} T_{\text{prod}} + b. \quad (\text{Eqn. 17})$$

where T_{add} is the addition of the fixed dimension latent vectors from 4 networks, T_{prod} is the dot product of the latent vectors from 4 networks, α_{add} and α_{prod} are the respective sensitivity weights for the weighting of both combined vectors.

(b) Liquid-Fraction Surrogate: 5-Network DeepONet

The liquid-fraction model predicts

$$f(x, y, t) = \mathcal{H}(Q, BC, T_{\text{snapshots}}; x, y, t), \quad (\text{Eqn. 18})$$

where $T_{\text{snapshots}}$ is the full set of temperature snapshots for that case and $f(x, y, t)$ is the liquid fraction value at the given (x, y, t) coordinates. It extends the temperature architecture by adding a third branch:

Temperature-Snapshot Branch

- Inputs: concatenated temperature fields from all save-times
- Purpose: provides a learned summary of the thermal history
- Structure: deeper MLP (2 layers, 256 neurons each)
- Output: latent vector that gives the phase-change prediction

The trunk XY and trunk-time networks are the same as in the temperature model. The head uses the same additive + multiplicative fusion. This hybrid head significantly improves accuracy for nonlinear phase-change physics.

(c) Training Procedure

- Dataset: 200 complete PCM-simulations
- Temperature model: uses heat-source + BC + coordinates
- Liquid-fraction model: uses heat-source + BC + temperature snapshots + coordinates

For each training iteration, a random set of spatial and temporal points (x, y, t) is selected pooled point sampling. This exposes the network to many micro-regions of the domain in every batch.

Training details

- Batch points: 16,384 per iteration
- Optimizer: Adam
- Learning rate: 1×10^{-3} or 1×10^{-4}
- Loss: mean squared error (MSE) on normalized targets
- Metrics: root mean squared error (RMSE) in Kelvin (temperature) and absolute fraction units (liquid fraction)

3.3.5 Output and Inference

After training, both DeepONet models can produce full temperature and liquid-fraction fields on a 2D grid for any new heat-source or boundary condition combination:

- Temperature prediction: milliseconds
- Liquid fraction prediction: milliseconds
- Speed-up over CFD/FEM: several orders of magnitude
- Mesh-free: predictions at arbitrary spatial or temporal locations

This enables rapid design studies, optimization, and uncertainty propagation for PCM–energy-storage systems.

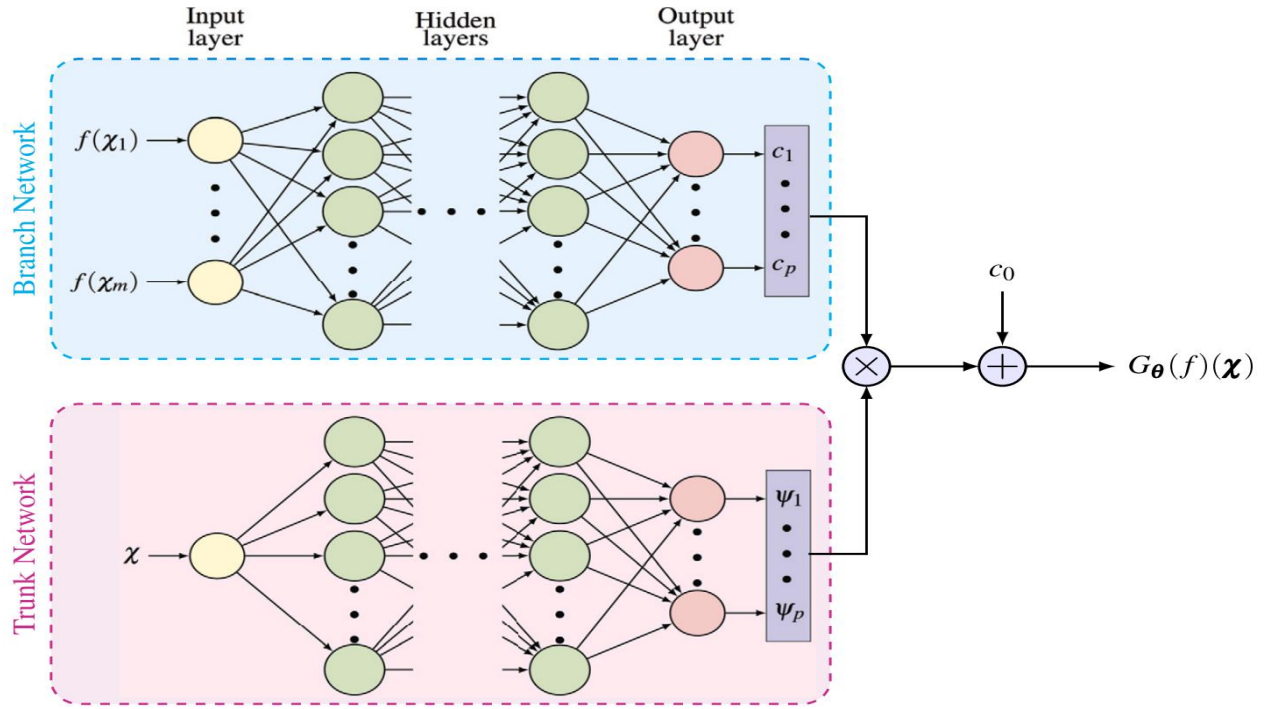


Fig. 3: Deep Operator Network (DeepONet) framework with branch net (global conditions), trunk net (local-coordinates), and head (temperature and liquid fraction outputs) ^[11]

3.4 Limitations of Pointwise MLP-Based DeepONet and Introduction of U-DeepONet

Even though the current DeepONet architecture provided acceptable results, but in the process a significant number of limitations were noticed for the above model based on pointwise pooling and sampling.

3.4.1 Limitations of Pooled Pointwise MLP DeepONet

In the previous formulation, the two-dimensional transient PCM problem was addressed via a pooled pointwise sampling strategy. In this case, all spatial points at a given time step were treated as separate training points. For a 40×40 grid and 9-time snapshots for a single case, one simulation provided:

$$40 \times 40 \times 9 = 14,400$$

space–time training points. For 200 simulations, this resulted in 2.88×10^6 training samples. So, even though we generated data for a mere 200 cases, for the model the data points were in millions. This formulation allowed the DeepONet architecture to approximate the operator mapping:

$$(Q, BC, x, y, t) \rightarrow T(x, y, t)$$

however, it also imposed the following limitations:

Loss of Spatial Correlation Structure

The pooled pointwise formulation treats spatial fields as vectors and feeds these into the fully connected branch networks. This results in:

- The loss of spatial relationships between neighboring points on the grid.
- The failure to exploit convolutional locality and translation invariance.
- The requirement for the network to learn spatial continuity via dense matrix multiplications.

For a physically structured problem such as heat diffusion and phase change, gradients and conduction paths dominate. So, fully connected MLPs fail to exploit the spatial inductive bias that convolutional architectures offer.

Large Effective Dataset Size and Training Time

Each space-time point is a data point for training. Therefore, there are hundreds of iterations for each epoch. The batch size is also large, 16,384 points are pooled and thus the training is computationally expensive. The physics occurs on a grid but is not treated as a picture by the model. Instead, it treats each coordinate as a separate data point. This results in a higher memory cost and slower convergence. The model also learns the same spatial features repeatedly, which is not desired.

Difficulty Capturing Sharp Melting Fronts

The phase change problem also includes mushy regions and sharp liquid-solid interfaces. Additionally, hot spots are created by uneven heating. The MLP is likely to smooth out features. The reason for this smoothing is that the dense transformation is a global operation. The MLP does not have hierarchical feature extraction. Therefore, it does not handle spatial discontinuities well unless it is a deep network. The smoothing effect is observed in the liquid fraction prediction.

3.4.2 Motivation for U-DeepONet

To address the challenges, the grid-to-grid operator learning method was developed by integrating the U-Net architecture ^[15] into the Branch network architecture. This is based on the following motivations:

- PCM systems are fundamentally spatial and heat conduction is locally diffusive.
- More generalised data like battery geometries are locally heterogeneous.
- Convolutional networks naturally capture spatial melting fronts which are sharp in nature, much better.

U-Net, originally conceived for biomedical image segmentation, has the desirable properties of multi-scale feature extraction, downsampling-then-upsampling, skip connections, and parameter sharing, which are all beneficial for solving physics problems with strong spatial dependencies.

3.4.3 U-DeepONet: Grid-to-Grid Operator Learning

In the context of the study conducted by Diab.et.al., on his work on U-DeepONet ^[15], Each PCM simulation is treated as one training sample, not millions of space-time points.

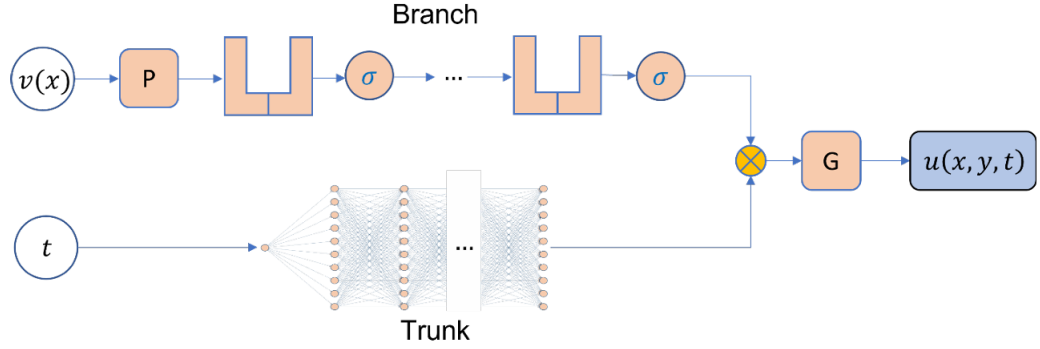


Fig. 4: U-DeepONet architecture. The figure shows the branch and the trunk networks. The trunk is a feed forward neural network, where t is the time input. In the branch, P is a linear layer, followed by U-Net blocks; σ is an activation function, and G is a shallow neural network. ^[15]

Input Structure

The branch network is provided with a static tensor denoted by:

$$X_{\text{static}} \in \mathbb{R}^{C \times p \times q}$$

This essentially means that there are C input channels over a $p \times q$ spatial grid. For our model a 13-channel tensor is used which contains all the information with respect to the spatial location of the PCM. As given in Table-1, the individual channels of this tensor include:

- $Q(x, y)$ - volumetric heat-source field
- Battery mask - indicating battery location with respect to PCM
- X -coordinate map
- Y -coordinate map
- Channels from 5-12 are individual boundary value-mask pairs with respect to the left, right, bottom, and top boundaries.
- Constant bias channel - all ones

The full input tensor can be written compactly as

$$X_{\text{static}} = [Q, M_{\text{bat}}, x, y, BC_{\text{val}}^{\text{left}}, BC_{\text{mask}}^{\text{left}}, \dots, 1]$$

Each channel has a clear physical meaning. The heat-source field drives internal heating, the boundary value–mask encoding ensures correct enforcement of boundary conditions, and the coordinate maps allow the network to remain spatially aware. The bias channel improves training stability. Unlike the previous MLP-based approach, no flattening is performed. The spatial grid is preserved exactly in its two-dimensional form.

Branch Network: U-Net Based Spatial Encoder

To define the branch network, we use the U-Net architecture and treat the entire spatial tensor

$$X_{\text{static}} \in \mathbb{R}^{13 \times 40 \times 40}$$

as a multi-channel image. In this context, the number of physical channels is 13, and the spatial grid is 40 x 40. Each channel represents a physically meaningful field such as the heat source, mask, coordinates, boundary maps, and bias terms.

Let the input tensor for one case be denoted as

$$F^{(0)} = X_{\text{static}}.$$

The encoder is constructed from stacked convolutional blocks that apply convolution operations to extract spatial features and apply non-linear activation functions. The activation function used in this branch network is the ReLU activation function. The general form of the convolutional layer is given by:

$$F^{(l+1)} = \sigma(W^{(l)} * F^{(l)} + b^{(l)}), \quad (\text{Eqn. 19})$$

where $F^{(l)}$ is the feature map at layer l , $W^{(l)}$ is the convolution kernel, $*$ denotes convolution operation, $b^{(l)}$ is the bias term, $\sigma(\cdot)$ is the ReLU activation.

To increase the receptive field of the network and capture the spatial structure of the input tensor at different scales, downsampling is used and given by the expression:

$$F^{(l+1)} = \text{Down}(F^{(l)}), \quad (\text{Eqn. 20})$$

where $\text{Down}(\cdot)$ represents max-pooling or strided convolution.

This gradual decrease in spatial resolution allows the network to learn the global heating patterns, melting zones, and the geometry influence.

At the bottleneck layer, the feature map has lower spatial resolution but higher channel depth. This represents a compressed spatial encoding of the full heat-source and boundary information.

Next, the decoder section scales up the features:

$$\tilde{F}^{(l)} = \text{Up}(F^{(l+1)}) + F^{(l)}, \quad (\text{Eqn. 21})$$

Where, $\text{Up}(\cdot)$ represents interpolation or transposed convolution, $F^{(l)}$ is the corresponding encoder feature map (skip connection). The skip connections ensure the preservation of fine-scale spatial features such as sharp melting fronts or hot-spots.

Finally, the branch section of the decoder produces the spatial embedding:

$$Z(x, y) \in \mathbb{R}^{d \times 40 \times 40},$$

Where, d is the latent feature dimension, $Z_k(x, y)$ represents the k^{th} learned spatial feature map. This spatial embedding captures the spatial interactions between heat source, boundary conditions, and geometry.

Trunk Network: Temporal Encoding

The trunk network encodes the time coordinate:

$$t \in \mathbb{R}.$$

For the batch of time snapshots, the time coordinate is given by

$$t = [t_1, t_2, \dots, t_{N_t}],$$

where $N_t = 24$, referring to 24 time – snapshots in the dataset.

In the trunk, the multi-layer perceptron (MLP) with sinusoidal activation is used, which is implemented as sin in the model. This allows for an efficient means of determining the evolution of things with time:

$$\tau(t) = \phi_{\text{trunk}}(t), \quad (\text{Eqn. 22})$$

with

$$\tau(t) \in \mathbb{R}^d.$$

In expanded form,

$$h^{(1)} = \sin(W^{(1)}t + b^{(1)}), \quad (\text{Eqn. 23})$$

$$h^{(2)} = \sin(W^{(2)}h^{(1)} + b^{(2)}), \quad (\text{Eqn. 24})$$

$$\tau(t) = W^{(3)}h^{(2)} + b^{(3)}. \quad (\text{Eqn. 25})$$

Here, $W^{(i)}$ are weight matrices, $b^{(i)}$ are corresponding bias vectors, $\tau(t)$ is the latent temporal vector of dimension d . This vector represents the evolution of the system with time, given the latent spatial state.

Fusion Mechanism: Grid-Wise Operator Evaluation

In the U-DeepONet, the spatial embedding in the branch is combined with the temporal embedding in the trunk via the feature-wise inner product:

$$\hat{T}(x, y, t) = \sum_{k=1}^d Z_k(x, y) \tau_k(t) + b, \quad (\text{Eqn. 26})$$

Where, $Z_k(x, y)$ is the k^{th} spatial feature at location (x, y) , $\tau_k(t)$ is the k^{th} temporal coefficient, b is a learned scalar bias.

The input temperature field is predicted as the sum of the product of the spatial features and the corresponding coefficients in the trunk, along with the learned scalar bias b .

In compact operator form,

$$\hat{T}(\cdot, t) = \langle Z(\cdot), \tau(t) \rangle. \quad (\text{Eqn. 27})$$

What This Model Does:

- Features are learned once from the full grid,
- Temporal evolution is captured by the coefficients in the trunk.
- Entire temperature field is produced in one forward pass.

Thus, the model performs a grid-to-grid mapping:

$$\mathcal{G}: X_{\text{static}} \rightarrow T(x, y, t),$$

Where the model is predicting the full spatial field simultaneously for each time snapshot.

In Table 1, the implemented 2D U-DeepONet architecture is shown with corresponding input structure, branch network for spatial inputs, trunk network for time inputs, trunk-branch operator product and projection head respectively.

Table 1: U-DeepONet architecture ^[15]. “Padding” denotes a padding operation. “UNet2D denotes a 2D U-Net. “ReLU” denotes the rectified linear unit. “Sin” denotes sinusoidal activation. “C” denotes the batch size. Network architecture for temperature field and liquid fraction field prediction

Part	Operation	Output Shape
Input	–	((C, 13, 40, 40))
Padding	Zero Padding (to preserve resolution)	((C, 13, 40, 40))
Trunk Input	–	((C, 24))
Branch Input	–	((C, 13, 40, 40))
Part	Operation	Output Shape
Lifting Layer	Conv2D ($13 \rightarrow 64$, kernel = 1)	((C, 64, 40, 40))
U-Net 1	UNet2D / ReLU	((C, 64, 40, 40))
U-Net 2	UNet2D / ReLU	((C, 64, 40, 40))
U-Net 3	UNet2D / ReLU	((C, 64, 40, 40))
Activation	ReLU	((C, 64, 40, 40))
Part	Operation	Output Shape
Trunk Net	FNN ($1 \rightarrow 128 \rightarrow 128 \rightarrow 64$) / Sin	((C, 24, 64))
Part	Operation	Output Shape
Trunk–Branch Product	Pointwise/Elementwise Multiply (Hadamard)	((C, 64, 24, 40, 40))
Part	Operation	Output Shape
Projection 1	Conv3D ($64 \rightarrow 64$, kernel = 1) / ReLU	((C, 64, 24, 40, 40))
Projection 2	Conv3D ($64 \rightarrow 1$, kernel = 1)	((C, 1, 24, 40, 40))
Final Reshape	Squeeze Channel Dimension	((C, 24, 40, 40))

After passing through all stages, we get the final output in the form of

$$T_{\text{pred}} \in \mathbb{R}^{C \times 24 \times 40 \times 40}$$

Where T_{pred} is the predicted temperature field obtained. Same architecture with heat source as inputs was also employed for prediction of liquid fraction field.

3.4.4 Training Strategy and Implementation Details

Now that the U-DeepONet architecture has been specified, the training strategy and implementation details will be discussed. Due to the grid-to-grid formulation, the training process is much simpler compared to the previous pooled pointwise method.

Dataset Organization

Each PCM simulation is represented by one training example.

The entire dataset includes:

- 200 full 2D PCM-simulations
- Grid resolution: 40×40
- Time frame snapshots: $N_t = 24$
- Static input tensor:

$$X_{\text{static}} \in \mathbb{R}^{13 \times 40 \times 40}$$

- Target temperature tensor:

$$Y_T \in \mathbb{R}^{24 \times 40 \times 40}$$

The dataset is split into:

- Training set
- Validation set
- Test set

with a common split of 80:10:10 cases.

Because each example is represented as a structured tensor, not a flattened point set, batching becomes very simple. With a batch size of $B=16$, 16 full simulations are processed in each iteration.

Normalization Strategy

To facilitate stable training and faster convergence, all continuous fields are normalized before being passed to the network.

Normalization for any scalar field u is done as:

$$\tilde{u} = \frac{u - \mu_u}{\sigma_u}, \quad (\text{Eqn. 28})$$

where μ_u and σ_u denote the mean and standard deviation computed over the training dataset.

In particular:

- Heat source field Q
- Boundary temperature values
- Target temperature T

are normalized.

This removes any scale differences among channels (e.g., between coordinate maps and heat source values) and ensures that the gradients remain nicely behaved during backpropagation.

Loss Function

The temperature model is trained with the mean squared error (MSE) loss function between the predicted and actual temperature fields and same for liquid fraction fields as well:

$$\mathcal{L}_T = \frac{1}{N} \sum_{i=1}^N \| T_{\text{pred}}^{(i)} - T_{\text{true}}^{(i)} \|_2^2, \quad (\text{Eqn. 29})$$

where N denotes the number of samples in the batch.

The loss function is calculated over all spatial and temporal points simultaneously due to the output being a full 3D tensor (t, x, y) . This enables the network to learn the entire transient process of the PCM domain in a single step.

To monitor the convergence, the root mean squared error (RMSE) in Kelvin is also calculated as:

$$\text{RMSE} = \sqrt{\mathcal{L}_T}, \quad (\text{Eqn. 30})$$

which is a physically meaningful quantity for measuring the accuracy of the predictions.

Optimization and Training Details

The temperature model is trained using the Adam optimizer with a learning rate of 10^{-4} .

The training parameters are as follows:

- Batch size: 16
- Epochs: 60
- Branch activation: ReLU
- Trunk activation: Sin
- Latent width: 64

Since the U-DeepONet considers each simulation as one sample, the number of iterations per epoch becomes:

$$\frac{200}{16} \approx 12-13$$

This is significantly lower compared to the previous pooled-point model, which took hundreds of iterations per epoch. The entire training process takes approximately 20 minutes to complete while ensuring stable convergence.

3.5 Extension to 3D PCM Systems

The 2D U-DeepONet model is good at predicting temperature and liquid-fraction fields. When we use PCM thermal energy storage in real life we need to use 3D modelling. This is because the battery pack and the latent heat storage module and the PCM composite all exchange heat in three dimensions. So, we need to make a surrogate-learning model based on the U-DeepONet architecture. This will help us get accurate results and make it more useful for engineering.

When we deal with PCM energy storage in 3D the process of heat diffusion and the melting front development becomes more complicated. This is because heat conduction happens in all directions, which creates a melting front with different temperature gradients. Also, the effects of the walls and the areas with heat become more important in 3D cases. We cannot always use 2D models to describe these effects.

Making a 3D model is not easy. If we make a 3D model that changes over time we have to deal with a lot of data. We have to look at the temperature T and the liquid fraction f at every point in space and time:

$$T(x, y, z, t) \text{ and } f(x, y, z, t)$$

where x, y, z, t refer to each axis and time respectively. This creates an amount of data and makes the neural network training very expensive. The usual way of using neural networks with Conv3D needs a lot of GPU memory and time. This is because we have to process the 3D data at the same time.

The solution we propose is to use a slice-based approach to make a U-DeepONet surrogate model. We try to simplify the problem by adopting a new approach of trying to learn the operators directly from the 3D PCM fields. We do this by slicing the fields in the x, y and z directions. Then we train 2D networks for each of these fields. We can treat these fields like pictures with channels and use them in the 2D U-net architecture with the temporal trunk that we have already developed.

This approach is a way to learn 3D operators, in PCM thermal energy storage applications. It uses convolutional networks in an efficient way.

3.5.1 Motivation for 3D Operator Learning

The need for extending 2D operator learning to three dimensional is because practical PCM thermal systems are really three dimensional. Most of the time PCM energy storage systems, like battery thermal management systems, cylindrical energy storage modules and metal foam latent heat storage systems have heat transfer that happens in three dimensions.

When we use 2D models, we make things simpler by saying heat transfer only happens in one plane. This makes it easier to do the math and make models. It does not show what really happens. In 3D shapes, heat moves in all three directions so the melting fronts do not move uniformly across the whole PCM area.

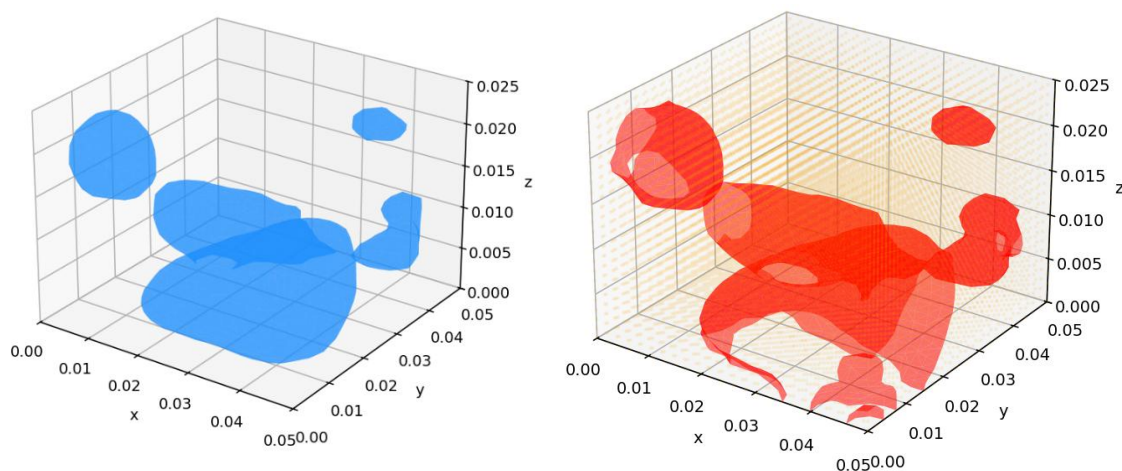


Fig. 5: Heat source at left and liquid fraction domain at a given time snapshot in 3D at right having 2D gaussian boundary conditions

What is more interesting is sometimes the melting process can be different in different directions because of how heat is transferred from inside the material and from the edges. Things like corner effects, side wall heating and how hot spots interact with each other can change how the temperature field evolves. This is especially important in PCM systems

that are powered by batteries. Real batteries are 3D and produce heat inside this space. So, depending on where the heat is produced, the thermal behaviour of the PCM depends on the 3D shape of the system. So, to get a simulation of PCM systems using an operator learning approach we need to:

- I. model how heat really moves
- II. account for the different ways the melt fronts move
- III. show what the battery and PCM look like
- IV. and make the surrogate models more accurate.

The problem is that learning 3D fields that change over time take a lot of computation which makes it hard to simulate and train. Here we have tried to propose a solution to this problem which is explained in the later subsections.

3.5.2 Challenges in Full 3D Operator Learning

Learning in 3D is great because it gives us better physical results. However, learning 3D Neural Operator data is quite hard to do. It is computationally expensive and also difficult to implement.

If we want to store the data for a specific 3D Neural Operator field we need to keep it in a special format:

$$[Nt, Nx, Ny, Nz]$$

Here Nt is the number of time-snapshots we take and Nx, Ny, Nz are the number of points in each x, y, z directions respectively.

As said earlier, compared to 2D, for storing 3D data we need a lot of points. For example, if we increase the grid size from (40×40) to $(40 \times 40 \times 40)$ we need 40 times more points. So, we have three problems:

- I. storing 3D Neural Operator data is expensive
- II. making training data takes a lot longer
- III. training the neural network needs a lot more computer memory.

Using 3D convolutional neural networks makes it even worse. These networks have a lot of parameters. Need to do complex operations, on all layers. This means:

- I. training takes longer
- II. we need memory to train
- III. the network takes longer to learn

3D Neural Operator systems have moving liquid boundaries and temperature changes. To model these systems accurately, we need networks and more details which makes it even more expensive.

To solve these problems, we propose a slice-based learning method, which can be an effective solution. We divide the 3D data into 2D slices. This way we keep the temperature and liquid fraction data structure intact and need less memory. The slice-based approach has benefits:

- I. it works with efficient 2D convolutional architectures
- II. it needs less computer memory
- III. it is less computationally intensive
- IV. it captures the front structures
- V. and it learns 3D thermal behaviour in multiple directions

This approach is better because it makes 3D Neural Operator learning easier and less expensive and can serve as a potential stepping stone to address this problem.

3.5.3 Slice-Based 3D U-DeepONet Framework

Rather than learning the operator from the volumetric tensor $[Nt, Nx, Ny, Nz]$, our approach learns the operator on spatial representations and still captures the essential 3D thermal behaviour.

Table 2: Slice-Based 3D U-DeepONet architecture ^[15]. “Padding” denotes a padding operation. “UNet2D” denotes a two-dimensional U-Net. “ReLU” denotes the rectified linear unit. “Sin” denotes sinusoidal activation. “B” denotes the batch size. “S” denotes the number of neighbouring slices used for spatial fusion. “T” denotes the number of temporal snapshots. Network architecture for transient three-dimensional temperature field and liquid-fraction field prediction using slice-based operator learning.

Part	Operation	Output Shape
Input	–	(B, S, 17, H, W)
Time Input	–	(B, T)
Slice-wise Processing	Process one slice at a time	(B, 17, H, W)
Part	Operation	Output Shape
Lifting Layer	Conv2D ($17 \rightarrow 64$, kernel = 1)	(B, 64, H, W)
U-Net 1	UNet2D / ReLU	(B, 64, H, W)
U-Net 2	UNet2D / ReLU	(B, 64, H, W)
U-Net 3	UNet2D / ReLU	(B, 64, H, W)
Activation	ReLU	(B, 64, H, W)
Part	Operation	Output Shape
Repeat for all slices	Shared encoder for all neighbouring slices	(B, S, 64, H, W)
Spatial Fusion	Weighted spatial fusion of slice features	(B, 64, H, W)
Part	Operation	Output Shape
Trunk Net	FNN ($1 \rightarrow 128 \rightarrow 128 \rightarrow 64$) / Sin	(B, T, 64)

Part	Operation	Output Shape
Expand Spatial Features	Copy spatial features across all times	(B, T, 64, H, W)
Expand Temporal Features	Broadcast temporal embeddings spatially	(B, T, 64, H, W)
Part	Operation	Output Shape
Trunk–Branch Fusion	Convolutional fusion of spatial and temporal features	(B, T, 128, H, W)
Part	Operation	Output Shape
Projection 1	Conv3D (128 $\rightarrow$ 64, kernel = 1) / ReLU	(B, T, 64, H, W)
Projection 2	Conv3D (64 $\rightarrow$ 1, kernel = 1)	(B, T, 1, H, W)
Final Reshape	Squeeze channel dimension	(B, T, H, W)

For each simulation case the transient PCM solver gives us:

- I. volumetric heat source fields,
- II. transient temperature fields,
- III. transient liquid fraction fields.

Mathematically these physical fields can be represented like this:

- I. $Q(x, y, z)$ for heat source maps
- II. $T(x, y, z, t)$ for temperature maps
- III. $f(x, y, z, t)$ for liquid fraction maps

Where, Q is the heat source distribution, T is the temperature field, f is the liquid-fraction field. We get these volumetric fields by solving numerically using the enthalpy-based PCM solver for boundary conditions and heat sources.

The 3D fields are then cut into 2D slices along all 3 axes:

- I. x – normal slices (yz plane)
- II. y – normal slices (xz plane)
- III. z – normal slices (xy plane)

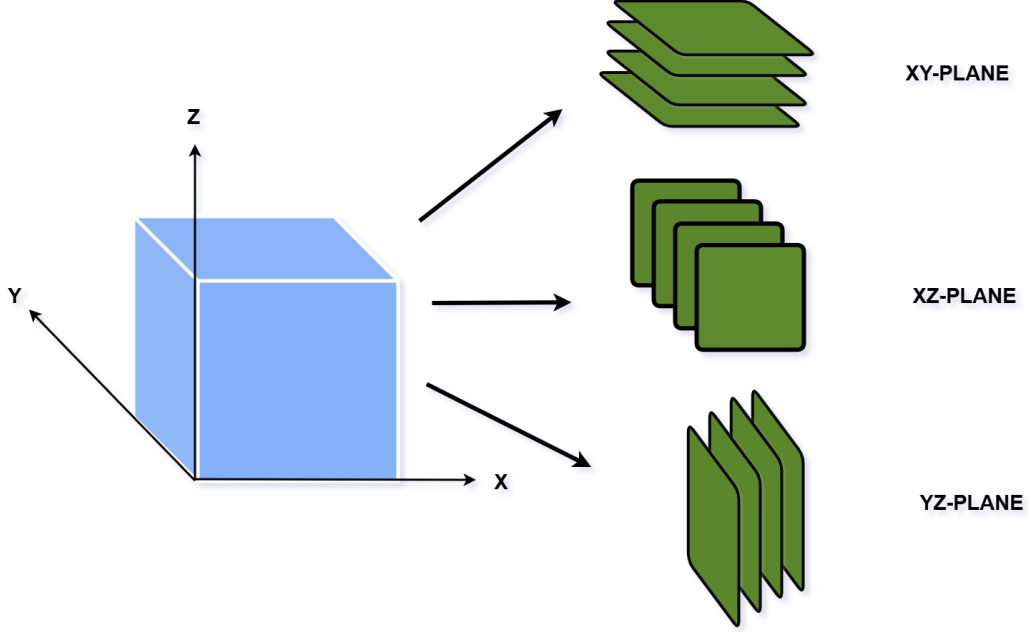


Fig. 6: Slice extraction strategy for the proposed slice-based 3D U-DeepONet framework based on the decomposition of the three-dimensional PCM domain using the XY, XZ, and YZ-planes

So, each 3D simulation case gives a set of transient slice samples with different spatial orientations and time snapshots. For example, an x -slice is a snapshot with a fixed x value while temperature and liquid fraction change within the y - z plane. Similarly, a y -slice shows changes in the x - z plane, a z -slice shows changes in the x - y plane. This way we can let the neural network see the thermal phenomena from different spatial perspectives.

Each 2D slice we get is represented as a multi-channel tensor:

$$X_{slice} \in R^{C_{in} \times H \times W}$$

where, C_{in} is number of input channels, H is height of the slice and W is width of the slice.

The input tensor has a total of 17 channels:

- I. volumetric heat source as channel-1,
- II. coordinate encodings for 3 axes with channels 2,3 and 4,
- III. 2D gaussian boundary faces values as channels 5,7,9,11,13 and 15,
- IV. Distance maps which tell how far a point/pixel of the slice sample is from each of the 6 boundary faces as channels 6,8,10,12,14 and 16. It is also shown in figure 7 for 4 distance channels. Rest 2 channels are constant for every point on the slice sample.
- V. Bias as channel 17.

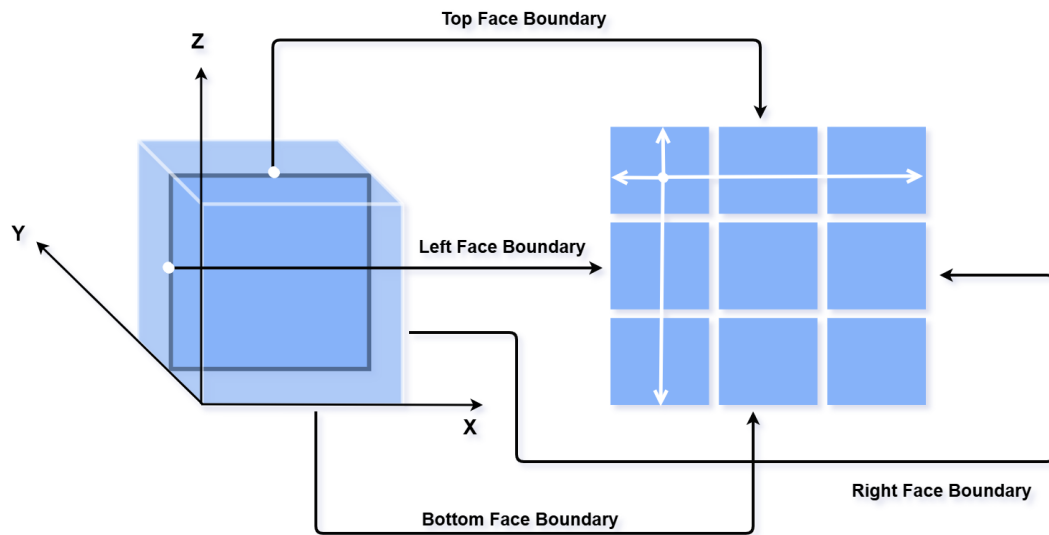


Fig. 7: Encoding boundary conditions for a 2D slice with boundary-value and boundary distance channels. Boundary distance channels on a given two-dimensional slice whose value depends on how far a point/pixel of the given slice sample is from each of the 4 boundary faces. It is based on orthogonal Euclidean distance

The orientation used in figure 7 would be used for storing the data as well as for implementation of the model. Here, considering the x, y, z axes meeting point to be origin, then in this orientation the XZ plane consisting the origin is the front face and its opposite XZ plane is considered to be the back face.

Unlike image processing, where channels are colours, our channels represent real physical quantities as channels that need to be learned. It is shown in figure 8.

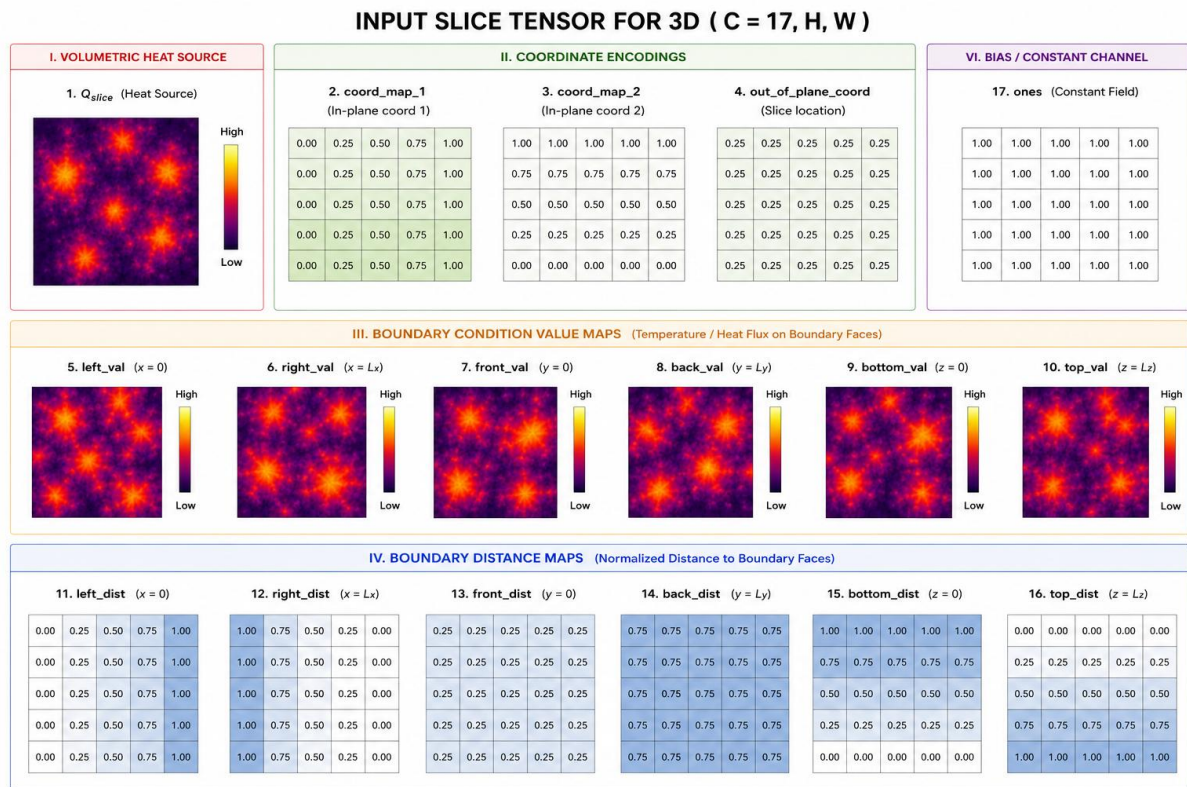


Fig. 8: Representation of the multi-channel input tensor for the slice-based 3D U-DeepONet framework. The input slice tensor contains volumetric heat-source information, coordinate encodings, boundary-condition value maps, boundary-distance channels and a constant bias channel, all together constituting the spatial information for transient temperature and liquid-fraction prediction.

The output tensor is:

$$Y_{slice} \in R^{2 \times H \times W}$$

where,

- I. channel 1 is the temperature field,
- II. channel 2 is the liquid fraction field.

Our framework estimates both temperature evolution and liquid fraction evolution.

Now that we have talked about the slice-based 3D U-DeepONet architecture we will discuss how to train it and the important details. We use the same U-DeepONet framework

that would work directly with multi-channel slice tensors. This makes it easier to batch things and it is more efficient when it comes to computing. This framework still keeps the three-dimensional thermal behaviour of the U-DeepONet.

Dataset Organization

Each PCM simulation is represented by one training example.

The entire dataset includes:

- I. 200 full 3D PCM simulations
- II. Grid resolution: $24 \times 24 \times 24$
- III. Time frame snapshots: $N_t = 24$
- IV. Slice samples are extracted along three orthogonal directions and as said earlier each slice sample is represented as:

$$X_{\text{static}} \in \mathbb{R}^{17 \times H \times W}$$

The dataset is split into:

- I. Training set
- II. Validation set
- III. Test set

with a common split of 80:10:10 cases.

Initially for each axis, 9 slices were taken together with a window radius of 4 around the central slice, with respect to which the loss was calculated and weights were updated by backpropagation. The entire architecture and data representation pipeline is given in Table 2.

Optimization and Training Parameters

The model is trained using the Adam optimizer with a learning rate of 10^{-3} . The principal training parameters are summarized below:

- I. Input channels per slice: 17
- II. Window size: 9 slices
- III. Base feature width: 64

- IV. Time embedding dimension: 64
- V. Fourier frequencies: 8
- VI. Activation function: LeakyReLU
- VII. Batch size: 128
- VIII. Epochs: 60

In the end we can say that the use of slice-based learning significantly reduced the computational cost and is simpler to implement compared to full volumetric Conv3D operator learning. This means we can keep the parts of the three-dimensional thermal characteristics while design a model to learn the phase change behaviour, which was our main goal to start with.

3.5.4 Three-Dimensional Melting and Solidification Solver

The two-dimensional solver described in Section 3.1 supplies the reference data for the planar studies. Generating training data for a genuinely three-dimensional problem requires the same enthalpy-based formulation to be carried into the volume, and that extension is described here. The governing equations, the phase-change rules and the treatment of the liquid fraction are unchanged; what changes is the spatial operator, the stability limit and the way boundary conditions are imposed.

The computational domain is a cube of side 50 mm discretised by 24 nodes in each direction, giving a uniform spacing of 2.17 mm. The energy equation retains the enthalpy form of Section 3.1, with the Laplacian now assembled over three directions by second-order central differences,

$$\nabla^2 T = \frac{T_{i+1,j,k} - 2T_{i,j,k} + T_{i-1,j,k}}{\Delta x^2} + \frac{T_{i,j+1,k} - 2T_{i,j,k} + T_{i,j-1,k}}{\Delta y^2} + \frac{T_{i,j,k+1} - 2T_{i,j,k} + T_{i,j,k-1}}{\Delta z^2} \quad (\text{Eqn. 31})$$

The enthalpy update, the recovery of the liquid fraction from the enthalpy field, and the classification of each cell into solid, mushy or fully liquid states follow the algorithm of Section 3.1.1 without modification, applied cell by cell over the volume. The explicit time-step restriction tightens because all three directions now contribute to the diffusive stability limit,

$$\Delta t = \frac{\text{CFL}}{2\alpha \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} + \frac{1}{\Delta z^2} \right)}, \quad \alpha = \frac{k}{\rho c_p} \quad (\text{Eqn. 32})$$

with the stability factor set to 0.30. Each case is advanced to 4000 s and twenty-four snapshots are stored, at 100 s intervals up to 2100 s and then at 2500 s, 3000 s and 3500 s, so that both the early conduction-dominated stage and the late, largely melted stage are represented in the training data.

Each of the six bounding faces carries an independent two-dimensional temperature field rather than a single value. A face may be held at a uniform reference temperature of 330 K, or may carry a Gaussian profile varying along one or both of its in-plane coordinates, with the peak position, width and amplitude drawn at random within prescribed limits. The resulting thermal environment is asymmetric in general, which obliges the surrogate to respond to the boundary data as well as to the internal generation field.

3.5.5 Prismatic Battery Heat-Source Configuration

The localised heat sources used in the two-dimensional study represent battery cells as regions of elevated generation within the PCM. The three-dimensional extension makes this explicit. One axis-aligned rectangular prism is embedded in each case, representing a single prismatic battery cell. Its edge lengths are drawn between 20 mm and 30 mm, so that the prism spans roughly nine to fourteen of the twenty-four cells along each axis, and its position is randomised subject to a clearance of at least 6 mm from every wall. Heat generation inside the prism is uniform, with a volumetric rate drawn between 1.4×10^5 and $2.5 \times 10^5 \text{ W m}^{-3}$, corresponding to a total dissipation of roughly 2.2 W to 3.9 W. The rate was scaled down from the values used for the smaller two-dimensional sources in proportion to the increased volume, so that the domain does not melt out well before the end of the simulated interval and the liquid-fraction targets do not saturate at unity.

The remainder of the domain carries a background generation field consisting of a floor value of 4.5 W m^{-3} with two to four smooth Gaussian blobs superposed, each of peak amplitude between 1×10^4 and $4 \times 10^4 \text{ W m}^{-3}$ and standard deviation between 6 mm and 14 mm, clipped at $5 \times 10^4 \text{ W m}^{-3}$. The background is thus well below the battery level, which keeps the two components separable, while ensuring that every extracted plane carries some structure even when it does not intersect the battery.

The battery is not a phase change material and must not be melted by the solver. Its cells are therefore excluded from the enthalpy–temperature relation and its phase fraction is

pinned at zero, while its temperature is obtained from a lumped energy balance. At each step the net conductive flow from the surrounding material into the prism is evaluated across its bounding faces from the same temperature field used in the Laplacian, so that the energy the battery gains is exactly the energy the PCM loses. The accumulated energy above the initial state is

$$E_{\text{bat}}^{n+1} = E_{\text{bat}}^n + \Delta t(P_{\text{gen}} + P_{\text{cond}}) \quad (\text{Eqn. 33})$$

and the battery temperature follows as

$$T_{\text{bat}} = T_{\text{bat},0} + \frac{E_{\text{bat}}}{C_{\text{bat}}}, \quad C_{\text{bat}} = \rho_b c_{p,b} V_{\text{bat}} \quad (\text{Eqn. 34})$$

where P_{gen} is the total generated power, P_{cond} the net conductive flow into the prism, V_{bat} its volume and C_{bat} its lumped heat capacity. The resulting temperature is imposed on the battery cells at the end of every step, and the prism is required to lie strictly inside the interior so that no prescribed face can overwrite it.

3.5.6 Slice-Window Fusion and Fourier Temporal Encoding

The slice-based framework of Section 3.5.3 decomposes the volumetric fields into planes along the three orthogonal directions. Two refinements are required before it can be applied to the battery-heated volume, and both follow from the fact that a plane cut from a three-dimensional problem is not self-contained.

The first concerns out-of-plane conduction. Heat also flows normal to the plane, so a single slice cannot in principle determine its own evolution. The formulation addresses this by presenting a window of neighbouring planes to a shared encoder and fusing their features. A window of thirteen planes centred on the target plane is used, with the index clamped at the ends of the axis so that near-boundary planes repeat their outermost neighbour. Each plane in the window is encoded independently by the same U-Net branch, the resulting feature maps are concatenated along the channel dimension and reduced by a 1×1 convolution,

$$Z_{\text{fused}} = \sigma(W_{\text{fuse}} * [Z_{-6}, Z_{-5}, \dots, Z_0, \dots, Z_{+6}]) \quad (\text{Eqn. 35})$$

where the fusion convolution maps 64×13 channels back to 64. Out-of-plane transport is thereby represented while every convolution in the network remains two-dimensional.

The second refinement concerns the temporal coordinate. Neither the temperature nor the liquid fraction evolves linearly during phase change: the temperature is arrested near the melting point while latent heat is absorbed, and the liquid fraction rises steeply once the front passes. A single normalised scalar is a weak input for a network asked to represent such behaviour, so the normalised time is expanded into a Fourier feature vector,

$$\gamma(t) = [t, \sin(2^0\pi t), \cos(2^0\pi t), \dots, \sin(2^7\pi t), \cos(2^7\pi t)] \quad (\text{Eqn. 36})$$

giving a seventeen-dimensional encoding with eight frequency octaves, which is mapped by a two-layer perceptron to a sixty-four-dimensional temporal embedding. The embedding is broadcast across the spatial dimensions and concatenated with the fused spatial features before the output head.

The slice tensor is correspondingly enlarged relative to the thirteen-channel two-dimensional input. Each stored plane is encoded as an eighteen-channel tensor comprising the volumetric heat-source values on the plane, three coordinate maps, six boundary-value images obtained by projecting the conditions on the six bounding faces, six orthogonal Euclidean distance maps giving the normalised distance from every pixel to each face, the battery mask, and a constant bias channel. The distance channels are what allow a single two-dimensional encoder to distinguish an interior plane from one near a wall: for a plane normal to x , the two x -distance channels are constant across the image and equal to the normalised distances to the two x -faces, while the four remaining channels vary linearly across the image.

The complete network lifts the eighteen input channels to sixty-four by a 1×1 convolution, applies three U-Net blocks with strided convolutional encoders and transposed-convolutional decoders, fuses the window, concatenates the temporal embedding and passes the result through two 3×3 convolutional layers and a final 1×1 projection to a single output channel. It has 1,930,689 trainable parameters. Separate models are trained for temperature and for liquid fraction, using the Adam optimiser with a learning rate of 1×10^{-3} , a batch size of 16 and 60 epochs.

3.5.7 Three-Dimensional Dataset Generation

Five hundred independent cases were generated, each with its own battery size, position and generation rate, its own background field and its own set of six boundary-face fields.

The cases were divided into 400 for training, 50 for validation and 50 for testing. Fifteen planes were stored along each of the three axes at uniformly spaced interior positions, avoiding the boundary-most planes, which gives forty-five stored planes per case and, with twenty-four snapshots each, 1080 slice images per case. The full volumetric temperature, liquid-fraction, heat-source and battery-mask fields were retained alongside the slices so that the volumetric solution remains available for reference. The parameters of the three-dimensional model and dataset are collected in Table 3.

Table 3: Parameters of the three-dimensional melting and solidification model and of the generated dataset.

Parameter	Value
Domain	$50 \times 50 \times 50$ mm
Grid	$24 \times 24 \times 24$ ($\Delta x = 2.17$ mm)
Time integration	Explicit FTCS, CFL = 0.30
Simulated interval	4000 s
Stored snapshots	24 (100–2100 s, then 2500, 3000, 3500 s)
Initial temperature	300 K
Reference boundary temperature	330 K
Boundary faces	Six independent 2D fields, uniform or Gaussian
Battery sources per case	One axis-aligned prism
Battery edge length	20–30 mm
Battery wall clearance	≥ 6 mm
Battery generation rate	$1.4 \times 10^5 - 2.5 \times 10^5$ W m ⁻³
Battery treatment	Lumped-capacitance balance, phase pinned at $f = 0$
Background generation	4.5 W m ⁻³ floor with 2–4 Gaussian blobs
Background clip	5×10^4 W m ⁻³
Number of cases	500 (400 train / 50 validation / 50 test)
Stored planes per axis	15
Slice images per case	45 planes $\times$ 24 snapshots = 1080
Input channels per slice	18
Slice-window size	13
Trainable parameters	1,930,689

3.6 Cross-Geometry Training Configurations

The dataset generalisation described in Section 4.1.5 mixes prismatic, circular and elliptical battery sources in equal proportion, so that the trained operator sees all three shapes. That design answers whether the model can represent a diverse population of sources. It does not answer a different and more demanding question: whether the operator has learned a mapping from the generation field to the thermal response, or has instead learned to recognise the particular shapes present in its training data.

To separate the two, a second training configuration was constructed in which the battery-embedded class contains prismatic sources only, with the sampling of position, size, orientation and generation rate otherwise unchanged. Two models were then trained under identical conditions: a prismatic-only model and a generalised model exposed to all three shapes. Both use the same architecture, the same optimiser settings, the same number of epochs and the same normalisation, so that the training set is the only variable.

Both models are subsequently evaluated, without retraining or fine-tuning, on identical test cases containing circular and elliptical sources. For the prismatic-only model these shapes are entirely absent from its training data, so the test measures transfer to an unseen source geometry. The governing formulation, the material properties, the boundary conditions and the numerical parameters are held fixed throughout, so that the test cases represent a change in the geometry of the input function rather than a change in the underlying physical problem. The results of this comparison are reported in Section 4.5.

CHAPTER 4:

RESULTS AND DISCUSSION

4.1 Data Characteristics and Heating Modes

4.1.1 Uniform Heat Source (Mushy/Phase Change Condition)

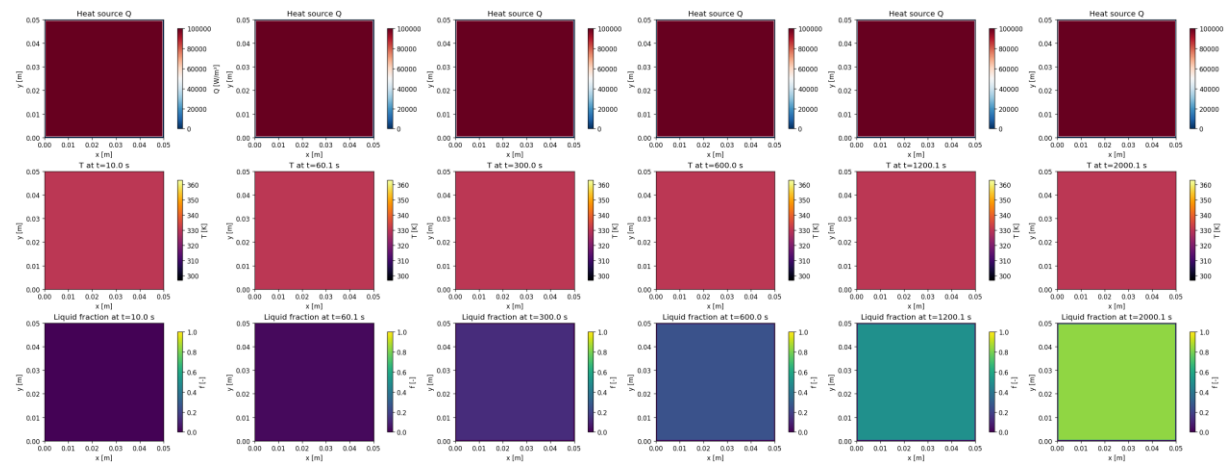


Fig. 9: Temperature maps when a constant heat source is applied everywhere inside the PCM block for mushy condition.

- *Constraints:*

Heat source: positive, scaled ($5 \times 10^4 \text{ W/m}^3$)

Total simulation time: 4000 s

Boundary temperature: 330 K (all sides)

Initial temperature: 330 K (domain)

Melting temperature: 330 K (material)

Effect: complete melting occurs

- *Observations:*

The heat enters everywhere evenly, so the PCM melts at nearly the same rate across the domain. The liquid fraction maps confirm that the phase change progresses smoothly and uniformly while temperature remains same. This demonstrates a baseline case with no sharp fronts.

4.1.2 Boundary Heat Source

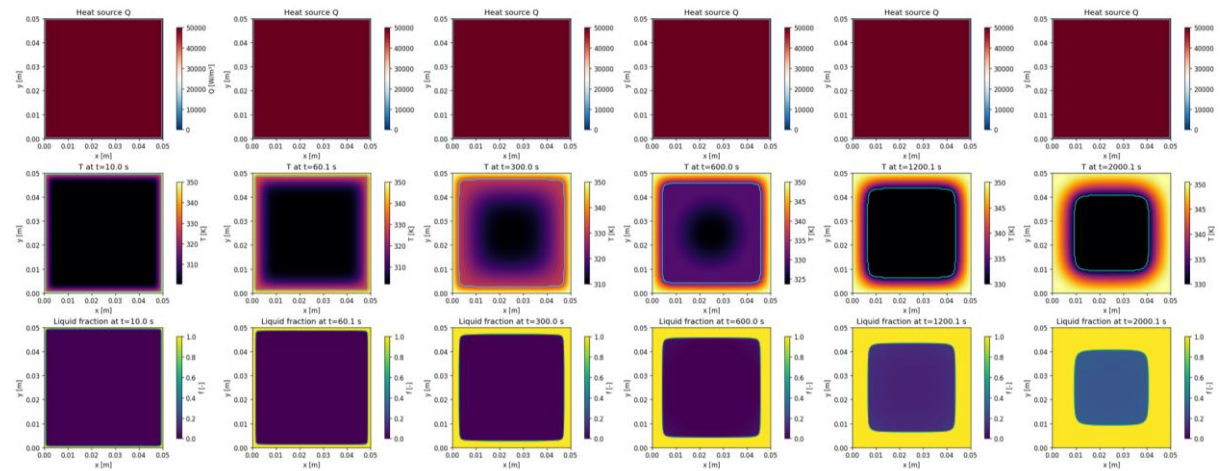


Fig. 10: Temperature maps with boundary heating where all sides are held hot at 350 K.

- *Constraints:*

Heat source: applied from boundaries

Total simulation time: 4000 s

Boundary temperature: 350 K (all sides)

Initial temperature: 300 K (domain)

Melting temperature: 330 K (material)

Effect: gradual melting from edges to center

- *Observations:*

Temperature maps show a strong gradient from the boundary's inward. The liquid fraction fields show a clear melting front progressing inward while the center stays solid for long. This highlights slow conduction limits in PCMs — and why surrogate models are needed for quicker response.

4.1.3 Observations from Random / Complex Heat Source

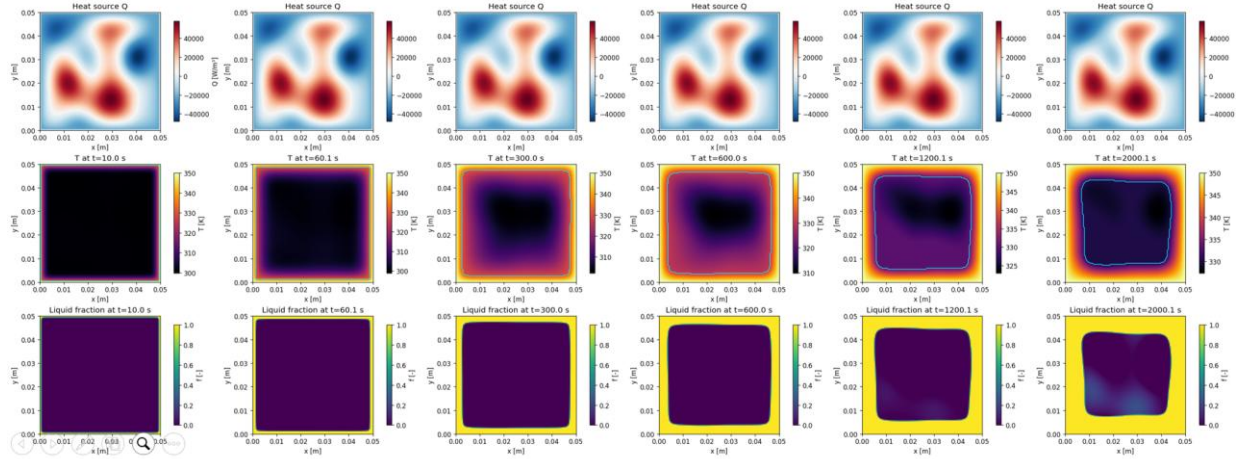


Fig. 11: Temperature maps with a randomly varying heat source generated via GPR.

- *Constraints:*

Heat source: random distribution, scaled ($5 \times 10^4 \text{ W/m}^3$)

Total simulation time: 4000 s

Boundary temperature: 350 K (all sides)

Initial temperature: 300 K (domain)

Melting temperature: 330 K (material)

Effect: irregular melting, local hot-spots drive melt pockets

- *Observations:*

Unlike the other cases, heat enters unevenly. Local hot-spots cause melting in patches that later spread and merge. This creates highly irregular melting fronts and non-uniform liquid fraction growth. Such conditions resemble real-world heating, where predicting exact melt fronts is challenging.

4.1.4 Training-Dataset Analysis

We have two heating sources which are temporally constant volumetric heat source Q and heating from boundary temperature profile. To understand how these two different heating mechanisms, influence melting behaviour in PCM domains individually, sample datasets were generated. To study each of these two heating sources, we have taken data containing systematic variations in:

- Volumetric heat-source intensity
 $Q_{\text{scale}} \in \{0, 5 \times 10^4, 8 \times 10^4, 1 \times 10^5, 3 \times 10^5\} \text{ (W/m}^3\text{)}$
- Boundary temperature profiles
 Left & right boundaries varied from 330–410 K using both Gaussian and constant profiles
- Fixed thermal conditions
 - Top & bottom boundaries held at 350 K (or 330 K in boundary-only heating experiments)
 - Initial temperature = 300 K
 - Melting temperature = 330 K
- Data was stored at nine instants:
 (100, 250, 400, 600, 1000, 1200, 1500, 1800, 2100 s)
- Spatial resolution
 All 2D fields were generated at 40×40 resolution, then flattened to 1D arrays for statistical comparison.

Both the cases produced both temperature and liquid-fraction fields, allowing detailed comparison of heating modes.

Heating from Volumetric Source Only

Here, initial temperature of the entire domain was set as 300 K and all boundary temperatures were fixed at the melting point 330 K, so melting was driven purely by internal heating.

Observations include:

- Higher Q_{scale} values create stronger early-time melting.
- In Fig. 12(a) and 12(b) at $Q_{\text{scale}} = 5 \times 10^4$ and 8×10^4 , melting is slow and spatially smooth.

- In Fig.12(c) and 12(d) at $Q_{\text{scale}} = 1 \times 10^5$, the melting becomes faster and as Q_{scale} is set to 3×10^5 , the domain nearly fully melts by 1200 s, showing strong sensitivity to internal heating.

This confirms that volumetric heat-input acts as a global heating mechanism, with melting initiation strongly dependent on the amplitude of Q .

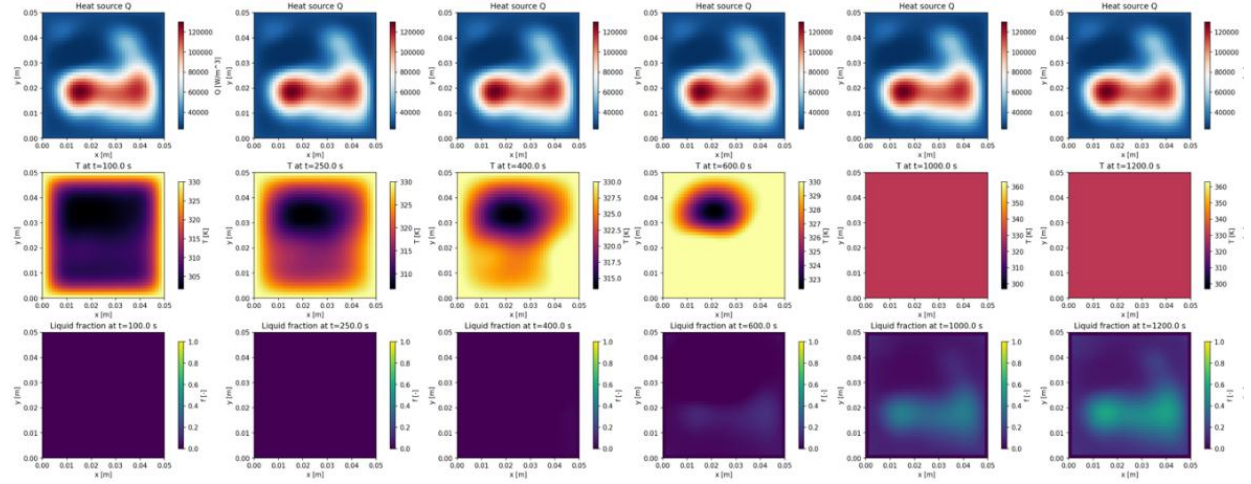


Fig. 12(a): Temperature maps for volumetric heat source scaled by 5×10^4 . Melting is slow

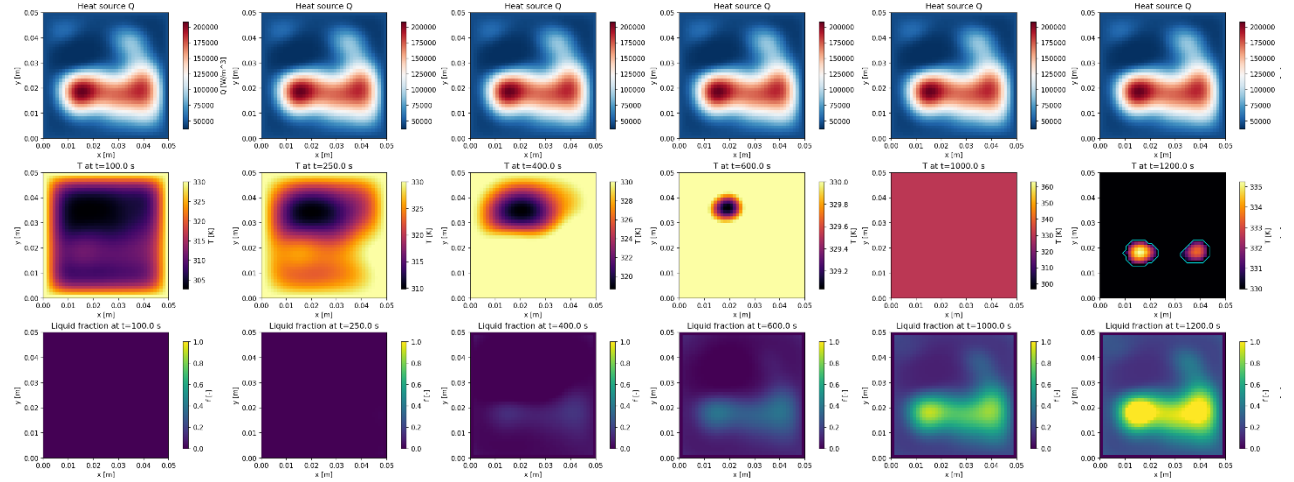


Fig. 12(b): Temperature maps for volumetric heat source scaled by 8×10^4 . Melting is faster than previous case

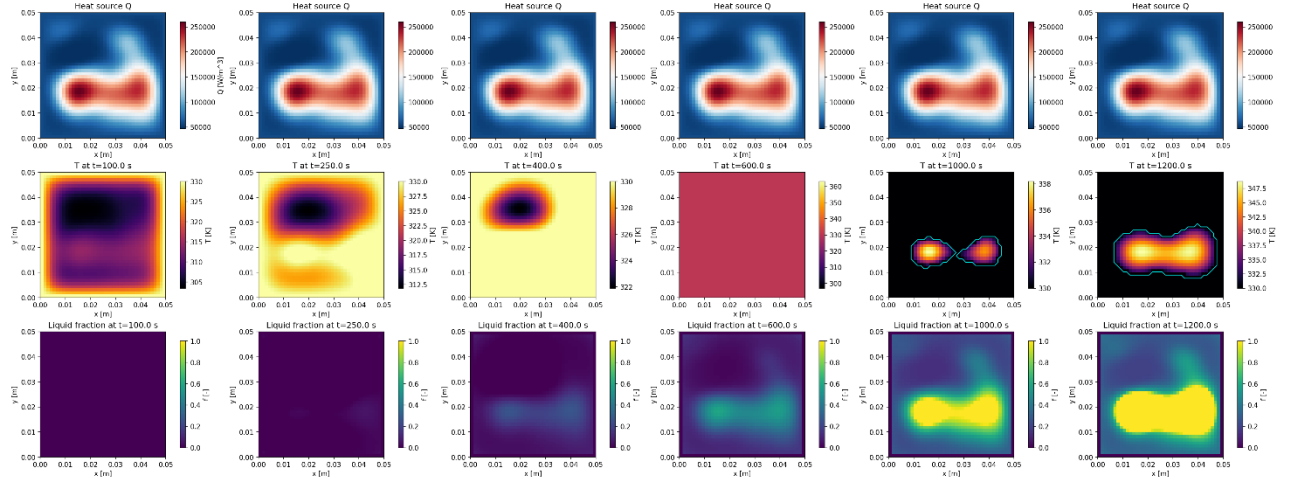


Fig. 12(c): Temperature maps for volumetric Heat source scaled by 1×10^5 . Melting is fast and spatially smooth.

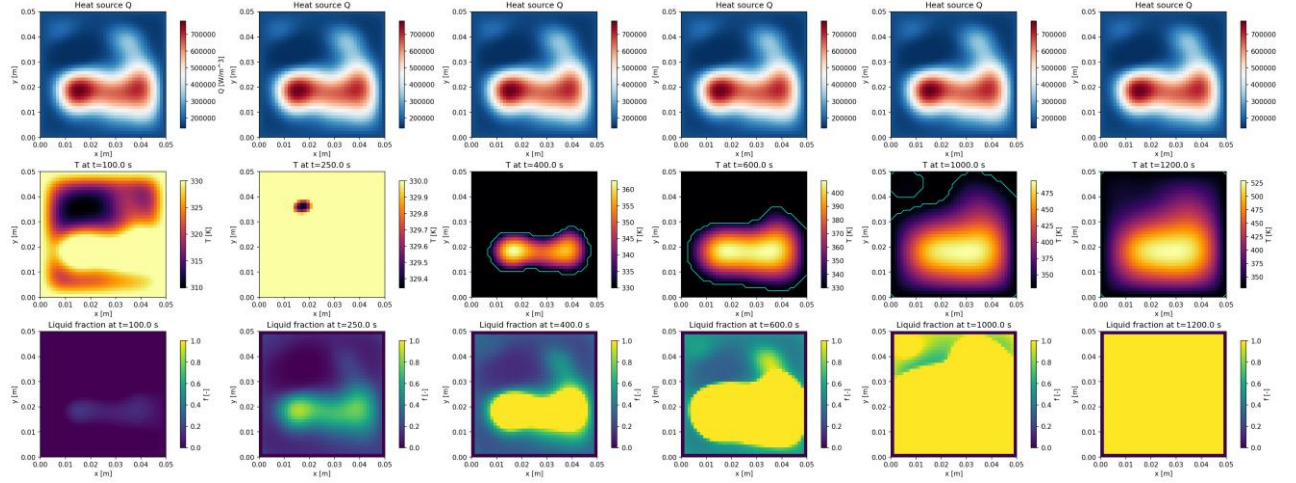


Fig. 12(d): Temperature maps for volumetric heat source scaled by 3×10^5 . Complete Melting occurs by 1200s showing stronger sensitivity to internal heating with given boundary conditions.

Heating from Boundary Temperature Source Only

In this regime, $Q_{\text{scale}} = 0$ and melting is driven solely by non-uniform boundary temperatures:

- Left and right boundaries vary between 330–410 K, while top/bottom remain fixed at 350 K.
- This creates a lateral melting front, strongest at locations with highest boundary temperature.

- Melting is delayed compared to volumetric heating, because heat diffuses from the boundaries inward.
- Gaussian boundary temperature profiles as in figure 13(b), result in localized melting zones, while constant 350 K as in figure 13(a), produces uniform front propagation.

Thus, boundary heating produces directional melting, unlike the domain-wide heating of volumetric sources.

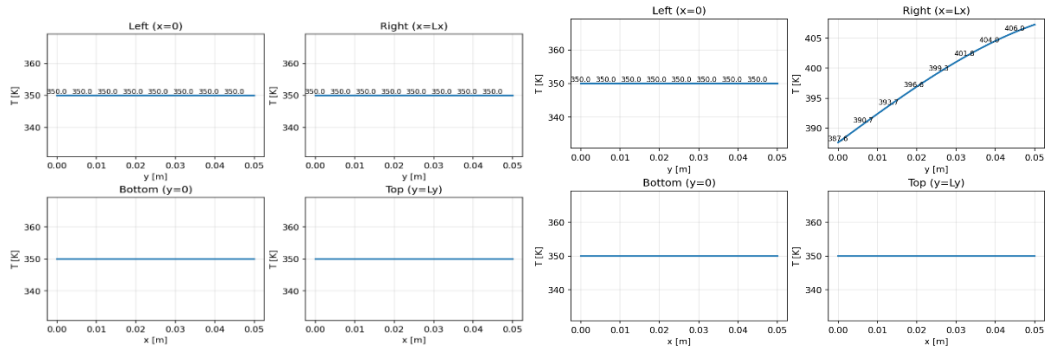


Fig. 13: Boundary Temperature cases for the samples shown; (a) constant 350 K on all sides, (b) constant 350 K on three sides and right-side boundary conditions given using gaussian profile, varying from 380 K to 410 K

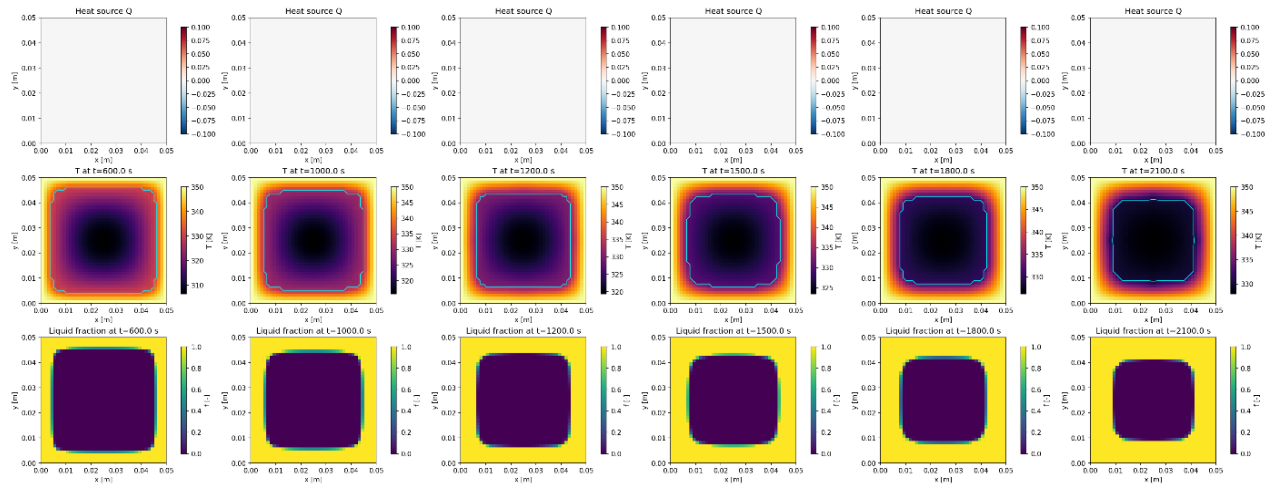


Fig. 13(c): Temperature maps for 350 K constant boundary temperature profile on all sides with zero volumetric heating from 600s to 2100s

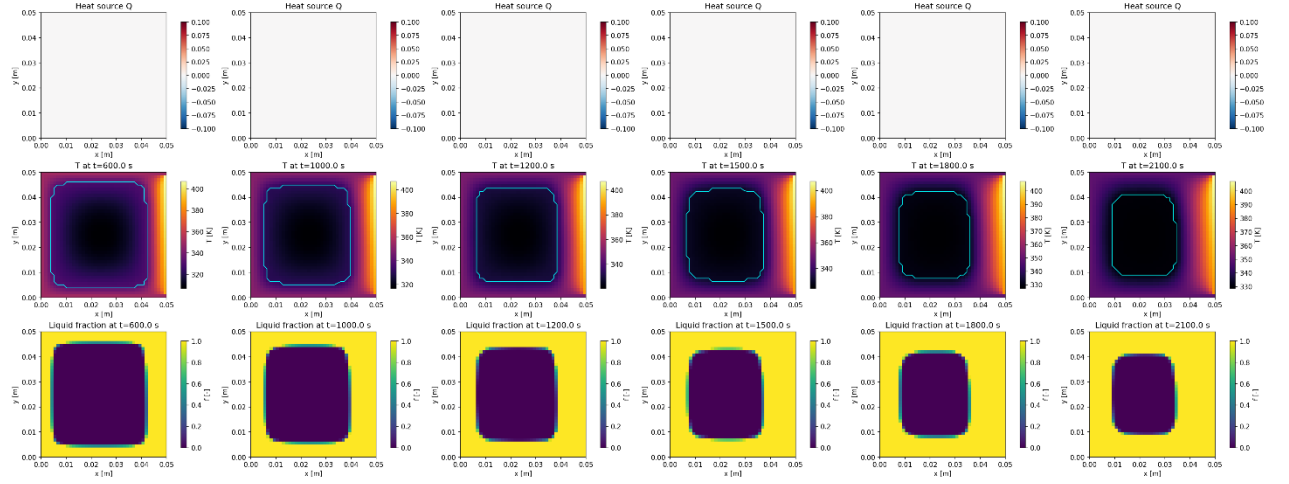


Fig. 13(d): Temperature maps for boundary temperature profile as shown in fig 9(b) with zero volumetric heating from 600s to 2100s

4.1.5 Dataset Generalisation

The surrogate model is initially trained and validated with a dataset of 200 full 2D PCM simulations. Each case represents one unique combination of scaled volumetric heat source field, computed temperature field, random boundary temperature profile and computed liquid fraction fields in same format as shown in the Fig. 12 and 13. In the initial stages of the work Gaussian generated volumetric heat source Q is primarily used which is scaled by a magnitude of 1×10^5 with a fixed melting temperature of 330 K for considering equal sensitivity to both heating modes as inferred from previous section. Boundary heating is applied to the left and right walls via Gaussian or constant temperature profiles between 330 and 410 K, while the top and bottom boundaries remain fixed at 330 K. Each simulation is run from an initial temperature of 300 K for 4000 s and snapshots are initially saved for temperature and liquid fraction at nine discrete times (100, 250, 400, 600, 1000, 1200, 1500, 1800 and 2100 s). These snapshots were tested across the initial pointwise pooled DeepONet model. Meanwhile All snapshots are flattened into one-dimensional arrays to compute statistical measures across the dataset. Later, for the U-DeepONet model architecture twenty-four-time (100, 200, 300, 400, 500, 600, 700, 800, 900...2000, 2100, 2500, 3000 and 3500 s) snapshots were used to generate one data case with the same unique combination of heat source, temperature fields and liquid fraction fields.

In the later stages of work, to generalise the dataset, we also tried to include and simulate prismatic battery sources, which is a practical heating method used for phase change materials energy storage systems. We also included rotated prismatic battery sources, circular and elliptic heat sources just as a challenge for the model to learn better as can be seen in Figure 17,18 and 19.

Dataset Statistics

Before moving on to the model training, the dataset's statistics was also done for cross-verification of whether appropriate variation is there in the data to enable the model to learn effectively across iterations and have a faster convergence. It would ensure that we can depend on the data current generator to be used for future where data would be generalised further.

Temperature Statistics Across 200 Cases

For each time snapshot, the values of temperature across entire domain for all 200 sets of data were taken to compute:

- mean temperature
- maximum temperature

To summarise the trend observed:

- At early times (100–400 s), the dataset shows narrow clustering of temperature values, primarily due to the PCM still being largely solid and the heat source not yet producing large gradients.
- Between 600–1500 s, the spread increases as melting starts in different regions depending on heat-source amplitude and boundary conditions.
- By 1800–2100 s, high-source cases produce significantly higher temperatures, leading to a broader spread.

Mean and Max Temperature Evolution are given as:

- Mean temperature across cases rises from around ~320 K at 100 s to approximately ~330–380 K range by 2100 s as shown in Fig. 14.
- Maximum temperatures display wider variation, ranging from ~340 K to >430 K as shown in Fig. 15, depending on localized heating magnitude and time of heating.

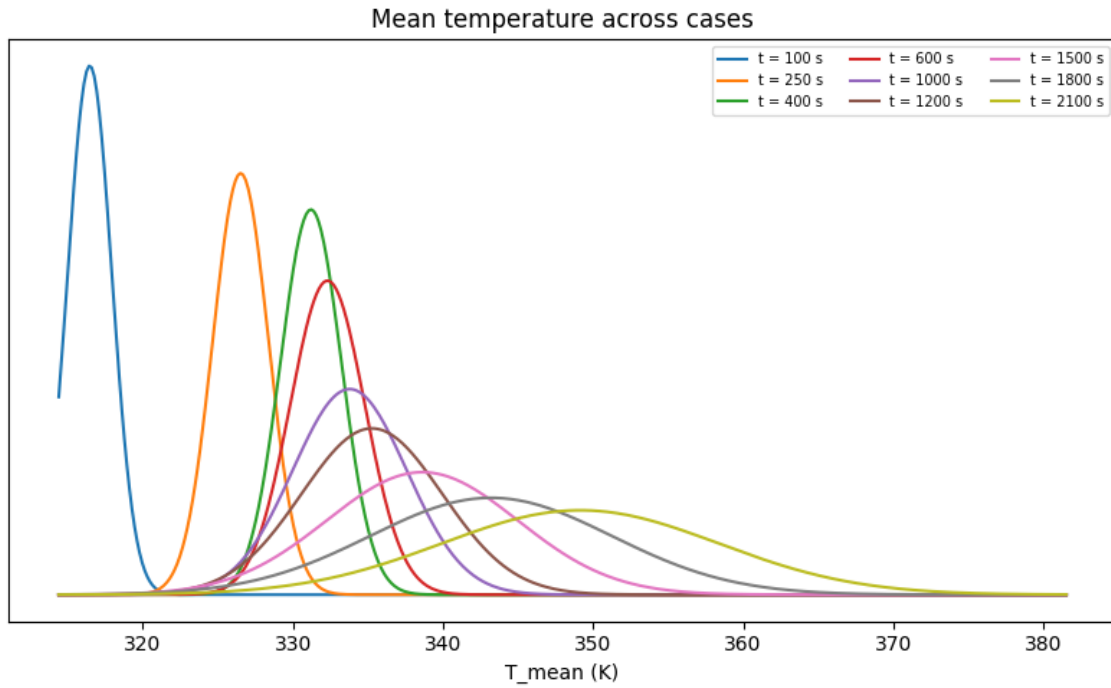


Fig. 14: Distribution of mean temperature across all 200 simulation cases for each time snapshot.
Each curve represents the spread of mean temperatures at a specific time

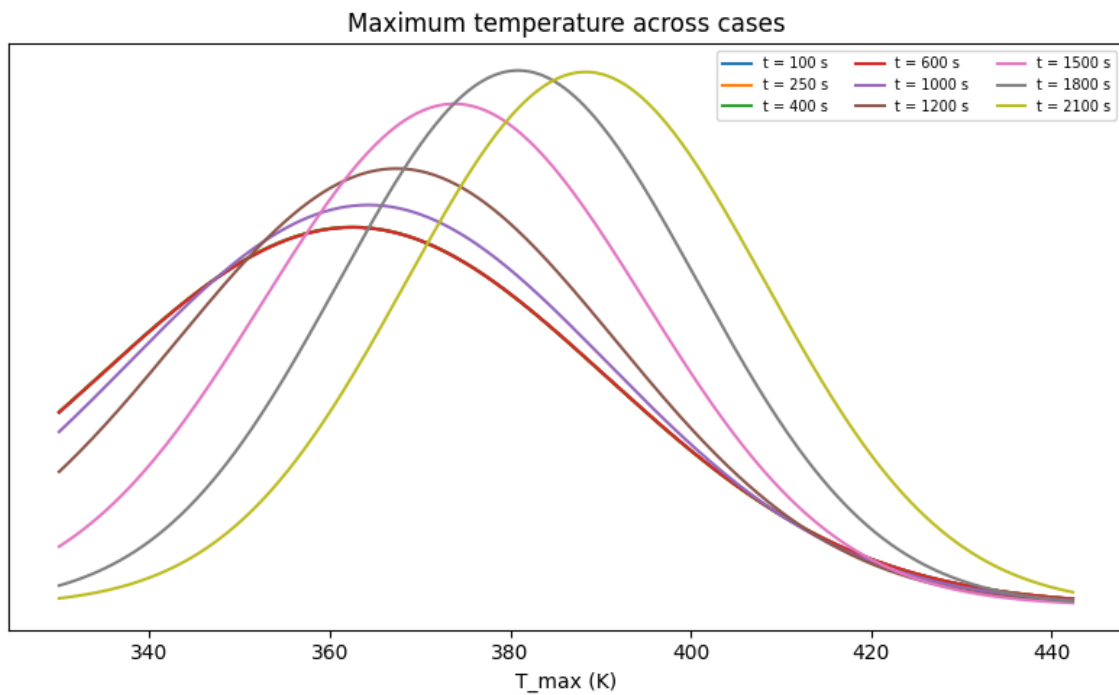


Fig. 15: Distribution of maximum temperature across cases at different time snapshots.

These variations show that the dataset has sufficient diversity for learning a generalized surrogate.

Liquid Fraction Statistics Across 200 Cases

Liquid fraction fields also show significant variation as PCM phase change occurs across the melting time. Temporal evolution analysis is given as in Fig. 16:

- Early snapshots (100–250 s) show near-zero liquid fraction for almost all cases.
- Mid-stage snapshots (400–1200 s) show some divergence as some cases begin melting earlier for high-Q cases, while others remain nearly solid.
- Late snapshots (1500–2100 s) show approximately same variation in melting progression, from partially melted to near-fully molten domains.

To summarize the trend:

- Mean liquid fraction increases gradually but remains evenly distributed for all time snapshots.
- Maximum liquid fraction values approach 1.0 much earlier in time in high-power cases and therefore its variance isn't of much significance in model training compared to other factors.
- Variance in liquid fraction grows a little in the middle melting time-range snapshots of the simulation, indicating strong enough case-to-case heterogeneity for model learning.

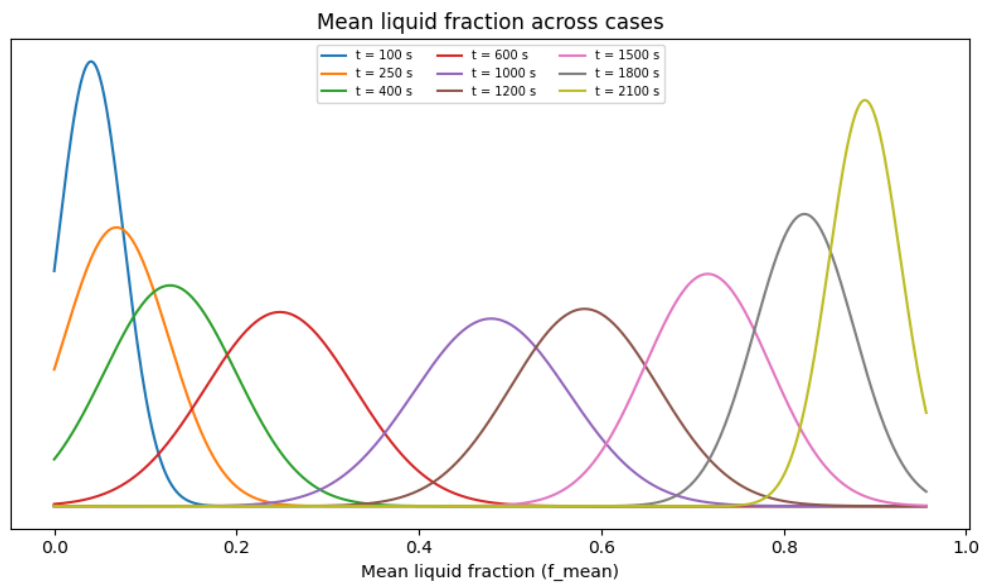


Fig. 16: Distribution of mean liquid fraction across all 200 cases at each time snapshot. The progression of curves from left to right reflects the onset, growth, and spread of melting

4.1.6. Generalised Data Samples

Here, in figure 17,18 and 19, there are a few data samples which were later used in the U-DeepONET model training process.

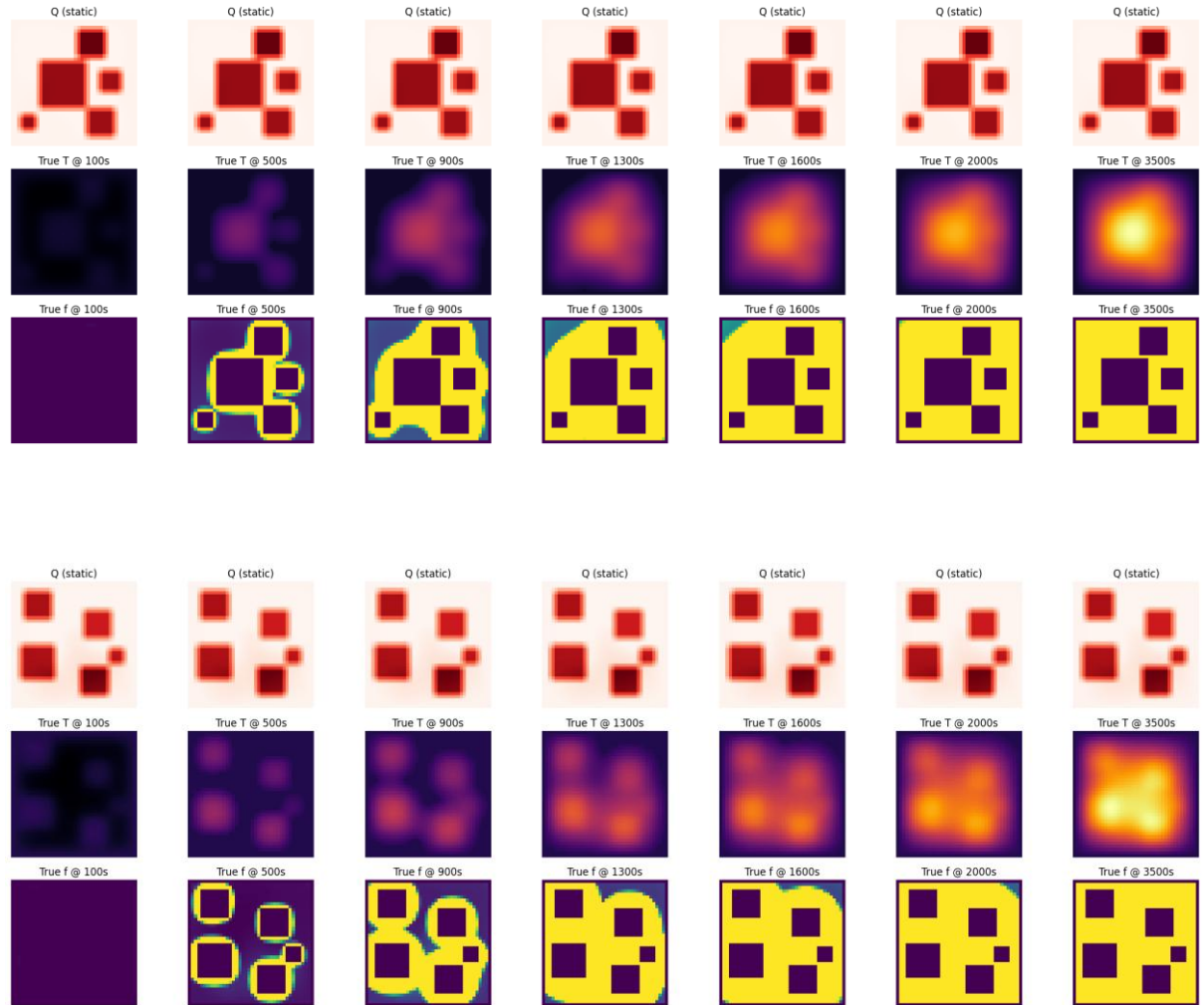
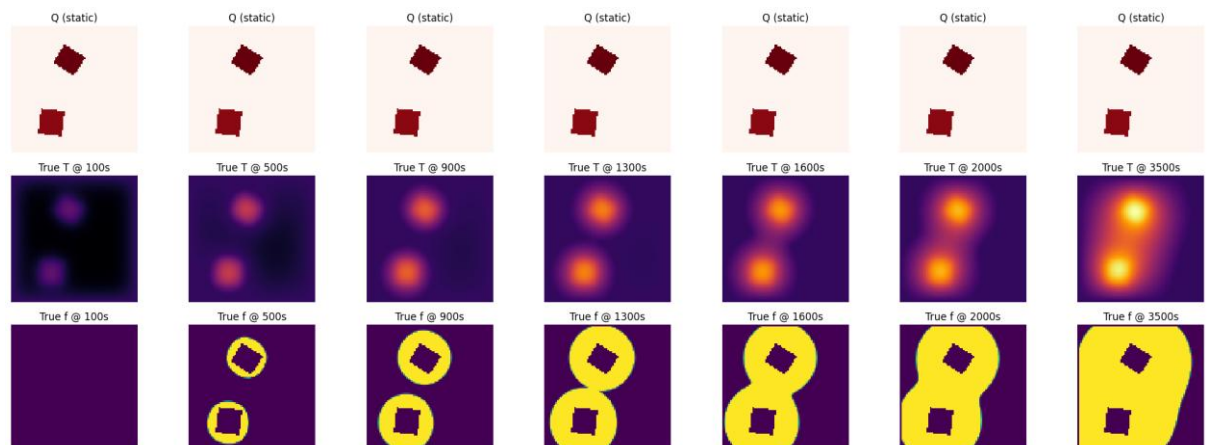


Fig. 17: Volumetric heat source, temperature field and liquid fraction maps for prismatic battery as heat source



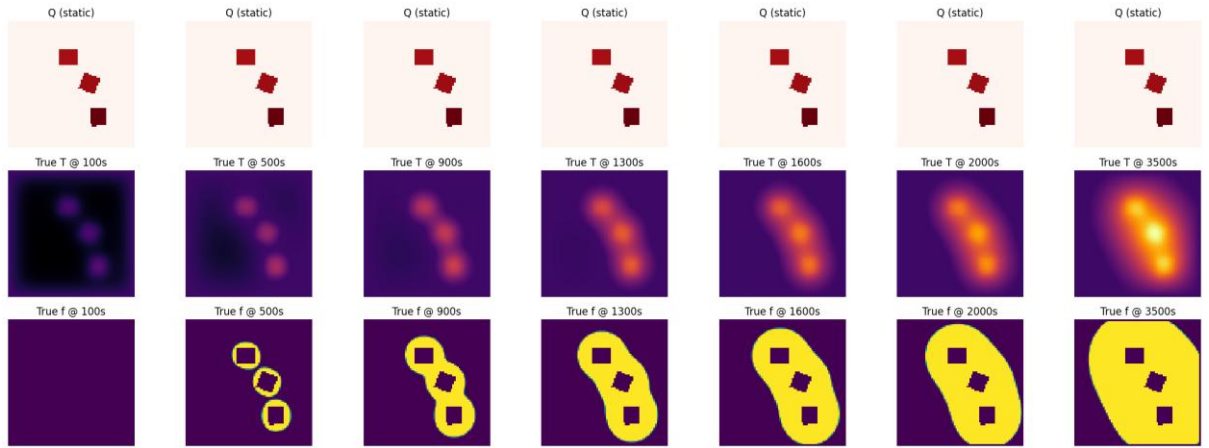


Fig. 18: Volumetric heat source, temperature field and liquid fraction maps for rotated prismatic battery as heat source. Rotation is within the range of 0 degrees to 90 degrees for symmetry

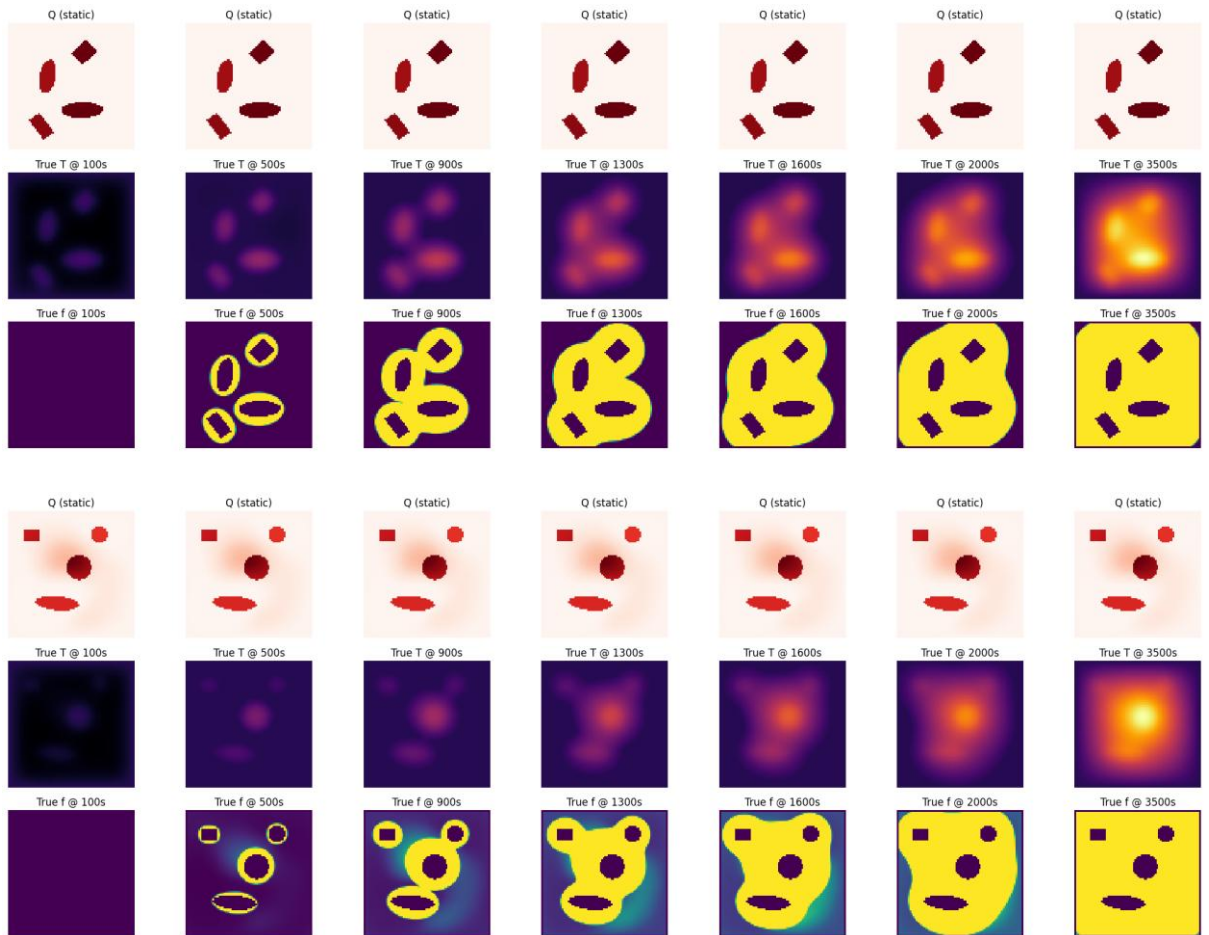


Fig. 19: Generalised Volumetric heat source, temperature field and liquid fraction maps for prismatic battery, circular and elliptic battery as heat sources along with rotation. Overall rotation is within the range of 0 degrees to 180 degrees for symmetry of ellipse

This is the current final dataset for 2D phase change systems which includes prismatic heat source, which is practically used in industries for energy storage, circular and elliptic heat sources though not generally used, yet it can help the model to generalise better.

4.2 Model Performance and Validation

4.2.1 Pointwise DeepONet Model

Both 1D and 2D pointwise DeepONet models are validated against simulation results from the PCM enthalpy formulation [5]. Agreement in temperature and liquid fraction fields demonstrates that DeepONet can serve as a reliable surrogate. The surrogate is then applied for parametric sweeps—testing porosity, pore size, and heat input variations—highlighting its value in rapid design optimization.



Fig. 20: Training and validation loss history of the 1D-DeepONet model, showing convergence with epochs

In the 2D setting, the pointwise DeepONet model is trained on complete spatially varying heat-source fields, temperature fields and boundary-condition profiles. The updated model's training curves demonstrate a sharp decline in both MSE and RMSE during the initial epochs, followed by a slow improvement as the network picks up more precise information about the temperature field.

As can be seen in Fig. 21(b), the final RMSE stabilises around 3–4 K, which demonstrates that the surrogate achieves physically meaningful accuracy despite the high dimensionality of the input space.

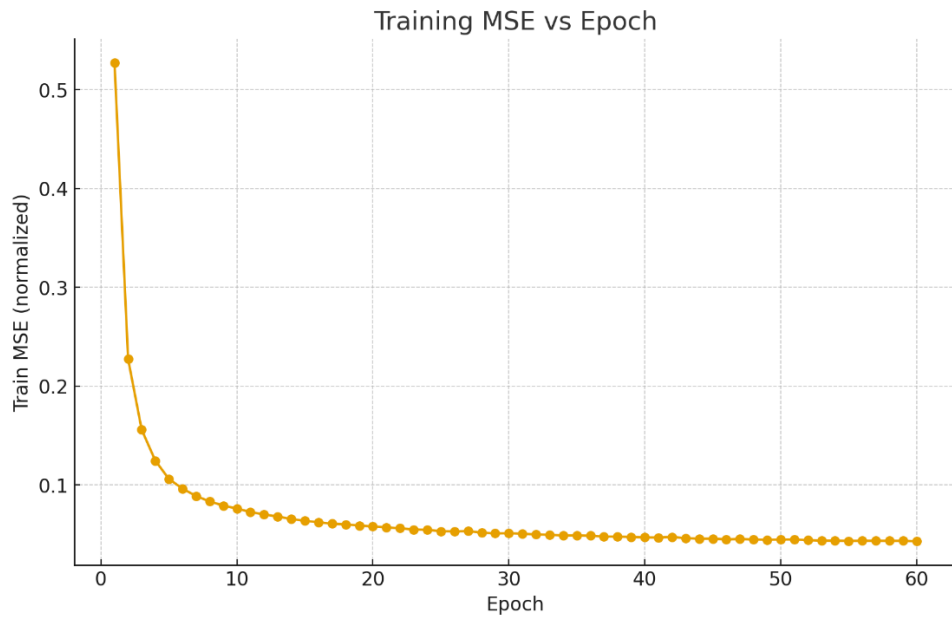


Fig. 21(a): Training MSE vs Epoch for the 2D Temperature DeepONet.

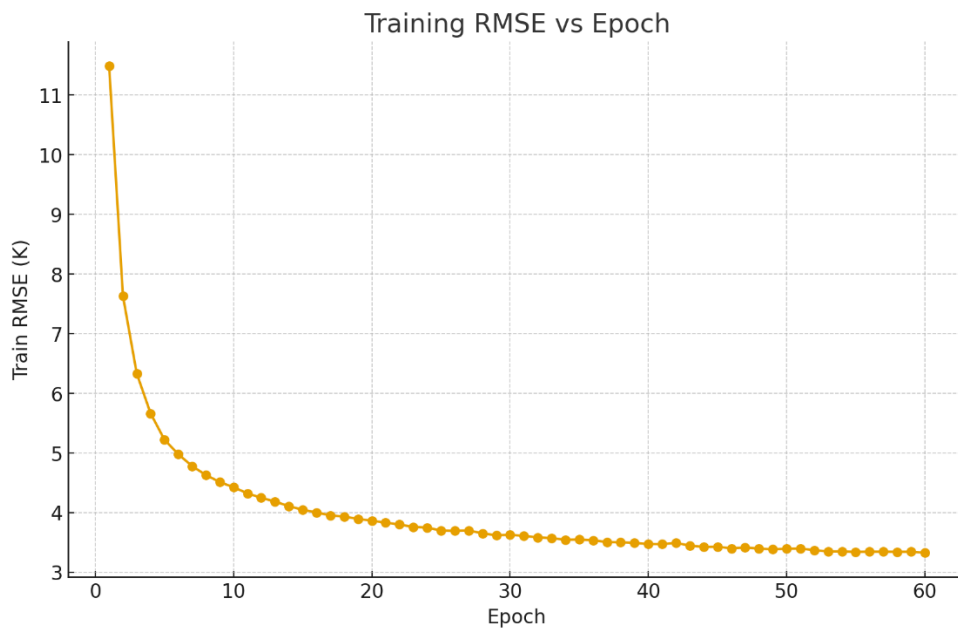


Fig. 21(b): Training RMSE vs Epoch for the 2D Temperature DeepONet.

Next, ground-truth numerical simulations from the 2D solver are compared with predicted temperature and liquid fraction fields to validate the model. Good agreement is observed in both the location and evolution of melting fronts.

This confirms that the pointwise DeepONet surrogate is capable of learning the operator mapping from (heat source, boundary conditions, spatial–temporal coordinates) to temperature and liquid fraction, but however the model is extremely slow compared to the much faster U-DeepONet architecture which was discussed earlier.

4.2.2 U-DeepONet Model and Data Sensitivity

The U-DeepONet model was trained for 60 epochs using the Adam optimizer. The learning rate was kept as 10^{-4} . Batch size was kept as 16. In Fig. 22(a) and 22(b), the training and validation loss curves are shown for different training set size. The accuracy of the model here is checked with a data sensitivity analysis, which was relatively difficult and time consuming in case of pointwise DeepONet model.

Rather than considering a fixed training dataset size of 200 data points, a series of training scenarios were explored with different numbers of simulations. In our case it was done for training set sizes of 80, 110, 140, 200, 300, 350, 400, 450 and 500 training set sizes.

Each configuration was trained independently under identical hyperparameters, which were mentioned earlier in the start of the section. In the initial stages of work, the battery geometry primarily used for data sensitivity was prismatic battery. At later stages of work, one more data sensitivity study was performed on a widely generalized dataset containing batteries having different geometries along with background gaussian heat sources with variable boundary conditions on all sides as well.

The purpose of this analysis was to examine:

- The rate at which prediction accuracy improves with an increasing number of data points.
- If the model converges after a certain number of training data points.
- The number of simulations necessary for reliable operator learning.

The loss curves plotted in Fig. 21 are Root-Mean Square Error (RMSE) vs Number of epochs for different test sizes, as can be seen.

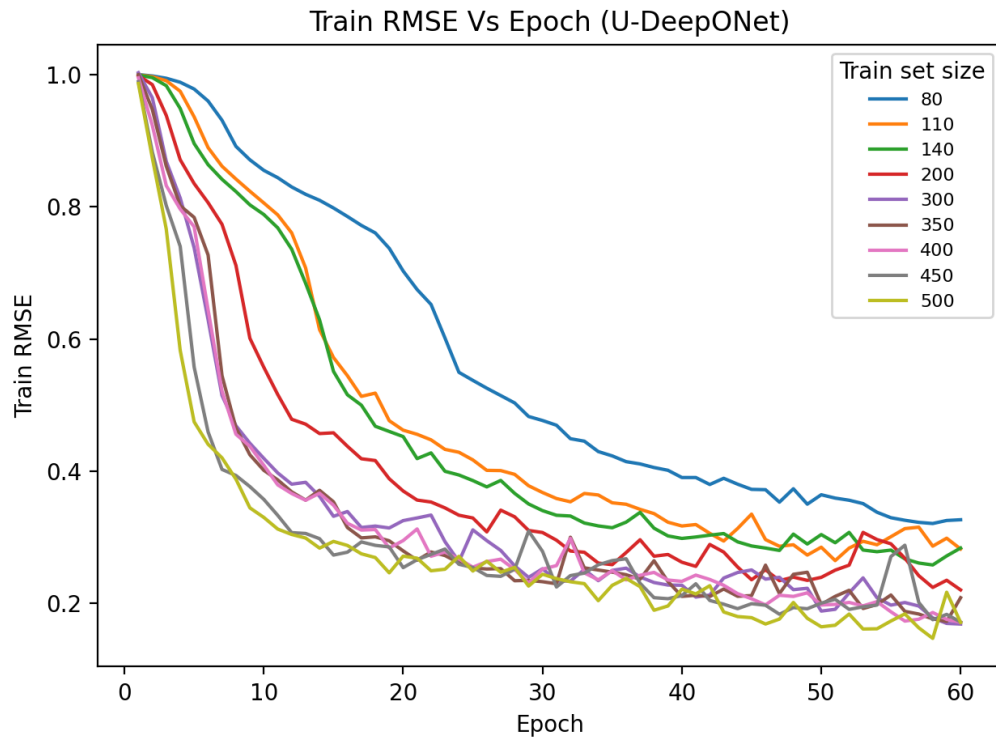


Fig. 22(a): Training RMSE vs Epoch for the 2D Liquid Fraction U-DeepONet for different training set sizes.

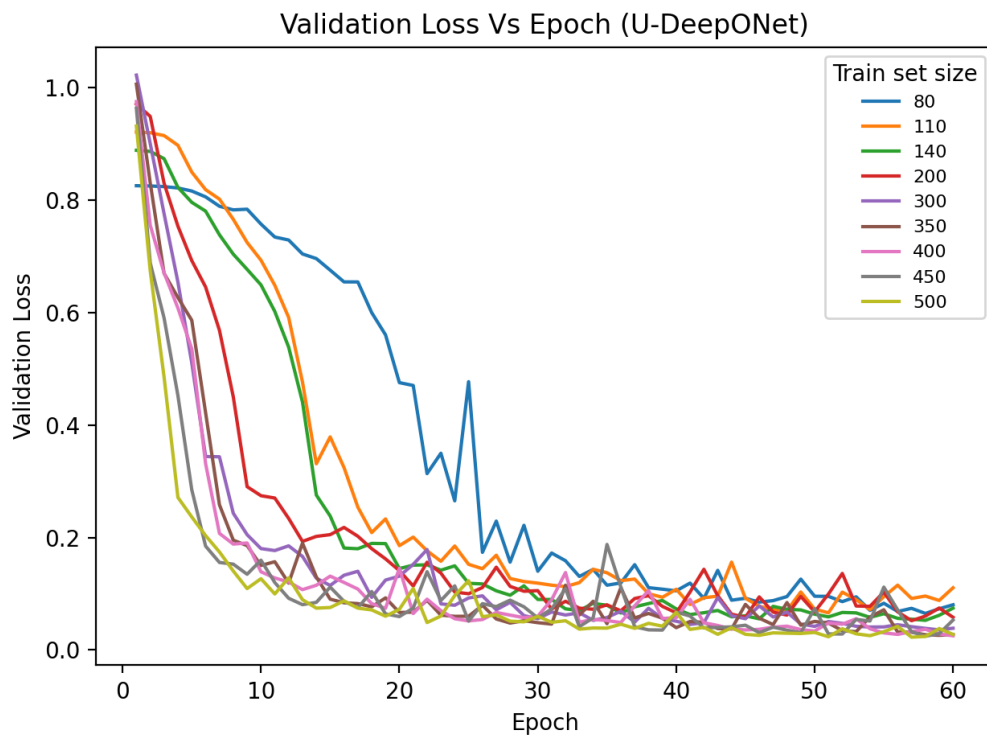


Fig. 22(b): Validation RMSE vs Epoch for the 2D Liquid Fraction U-DeepONet for different training set sizes.

Here we noticed, there is:

- Fast loss reduction in the early epochs.
- Slow convergence after the midpoint of training.
- Lack of divergence between the training and validation loss curves.

The small oscillations in the training and validation curves in fig. 22(a) and 22(b) are expected due to stochastic mini-batch optimization with a batch size of 16 and small dataset sizes, and they do not imply instability because the overall pattern of convergence is steadily decreasing.

The best validation loss for each training test size is given in Table 4, which can show the improving trend and later in fig.23 we can clearly visualize it.

Table 4: Best Validation RMSE on normalized data for different training set sizes which were trained on U-DeepONet model architecture for liquid fraction ^[15]

Training Cases	Best Validation RMSE
80	0.25497
110	0.25706
140	0.22968
200	0.22806
300	0.18900
350	0.16098
400	0.15838
450	0.15643
500	0.14997

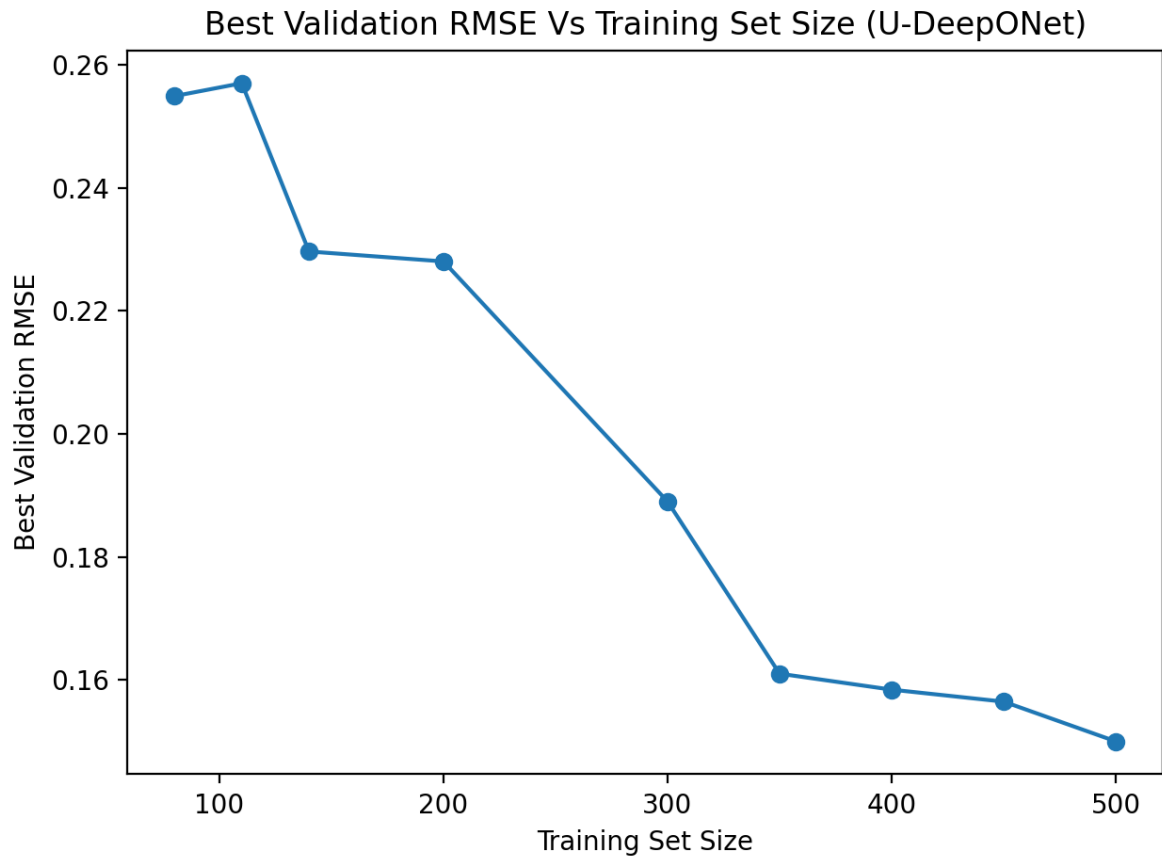


Fig. 23: Best Validation RMSE vs Training set size for the 2D Liquid Fraction U-DeepONet as per Table 4

Observations

The results reveal three distinct regimes:

(1) Low-Data Regime (80–140 cases)

- There is a slight oscillation in performance.
- The model starts learning global patterns.
- The RMSE stays above 0.22-0.25.
- There is some underfitting.

At this point, the network is not yet fully exposed to the spatial variations of heat source distributions and melting patterns.

(2) Moderate-Data Regime (200–300 cases)

- There is a definite improvement in the validation RMSE.
- The error goes below 0.20 normalized temperature.
- Convergence becomes more stable.
- The spatial patterns are learned more accurately.

This shows that a simulation number of 200-300 is sufficient to introduce diversity in the operator learning process.

(3) High-Data Regime (350–500 cases)

- There is a gradual but diminishing improvement.
- The rate of RMSE reduction becomes smaller after 400 cases.
- The performance starts to saturate.

The improvement from 400 to 500 cases is very small, which shows that the model is reaching the capacity of the current architecture.

So, we have understood that the training set size for the current U-DeepONet architecture^[15] is a crucial factor which determines the accuracy of the model over generalized dataset. In fig. 24(a) and 24(b) we can see the stark difference between the predicted liquid fraction plots when we increase the training set size from 80 to 500 for a given set of conditions.

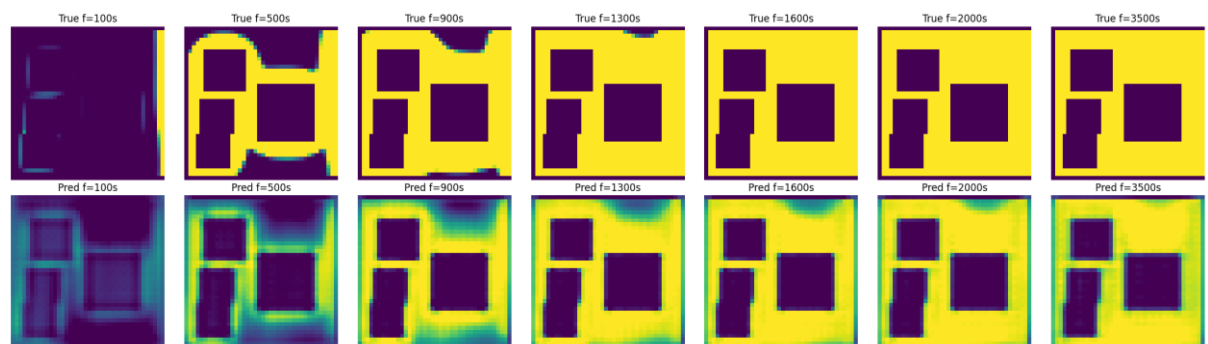


Fig. 24(a): True and Predicted liquid fraction fields for a given set of conditions. The inference plot is drawn by using the weights of a model trained on dataset size of 80.

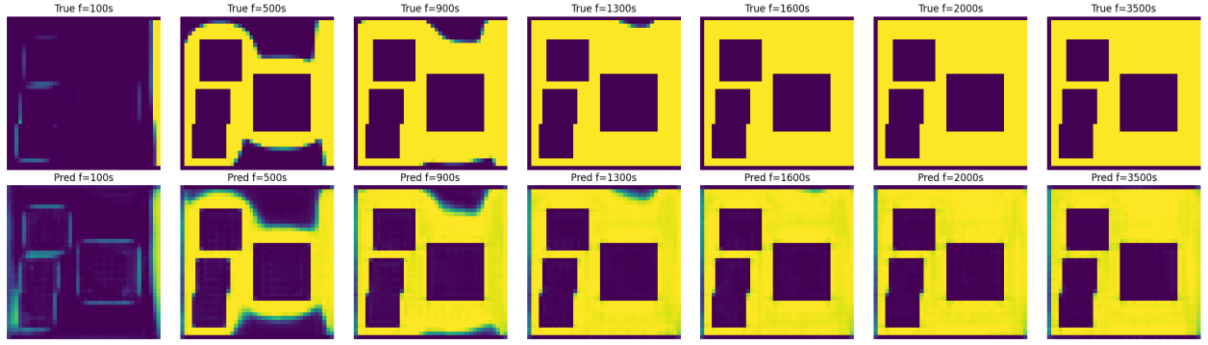


Fig. 24(b): True and Predicted liquid fraction fields for a given set of conditions. The inference plot is drawn by using the weights of a model trained on dataset size of 500

The data-sensitivity study reported above was subsequently repeated over a wider and more finely spaced range of training-set sizes, and on the generalised dataset that contains all three battery geometries together with the smooth and hybrid heating classes. Nested datasets of 80, 140, 200, 300, 350, 400, 450 and 500 simulations were constructed, each larger set retaining all cases of the smaller ones and adding new ones while preserving the proportion of the three source classes. The physical and numerical parameters, the heat-source generation procedure, the temporal sampling, the architecture, the optimiser settings and the normalisation were held fixed across all subsets, and the validation and test cases were kept separate from the training data in every instance, so that the effect of data availability was isolated from every other factor. The smallest set represents 16 % of the largest.

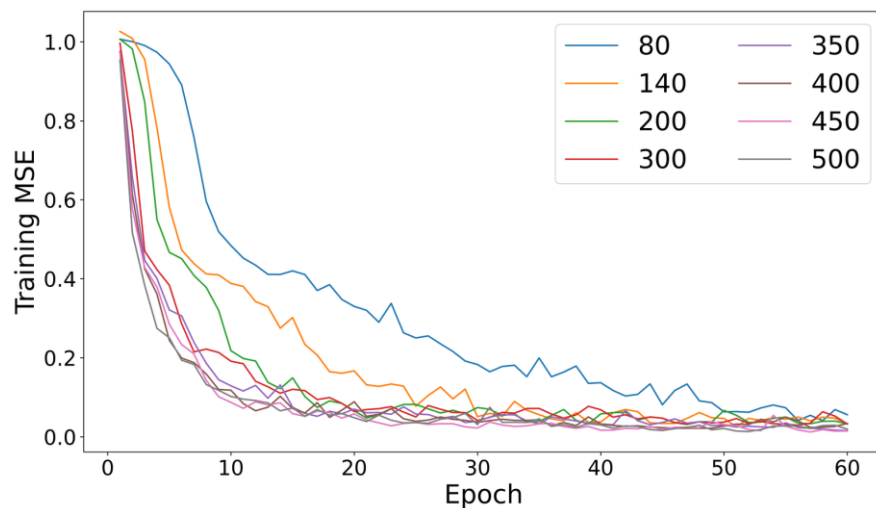


Fig. 25: Training mean squared error of the U-DeepONet for temperature prediction over 60 epochs, for different numbers of training simulations. Legend entries give the number of simulations.

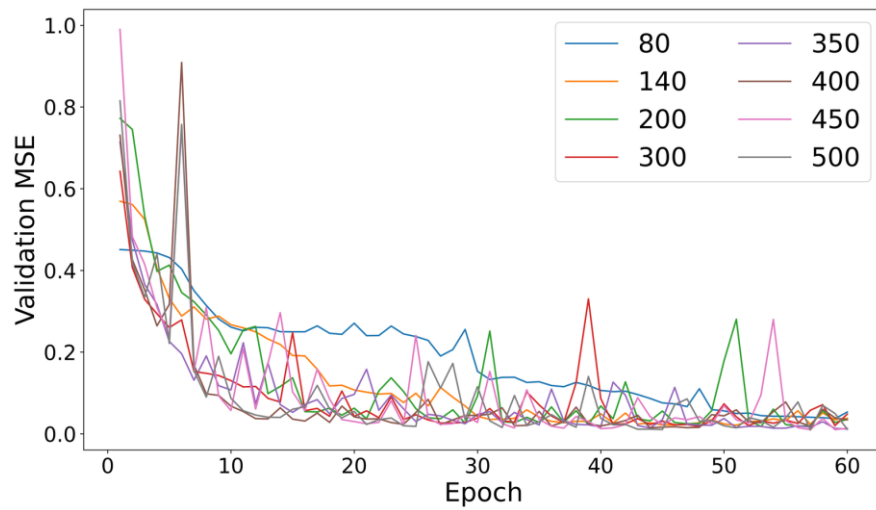


Fig. 26: Validation mean squared error for temperature prediction, for the same training-set sizes as Fig. 25.

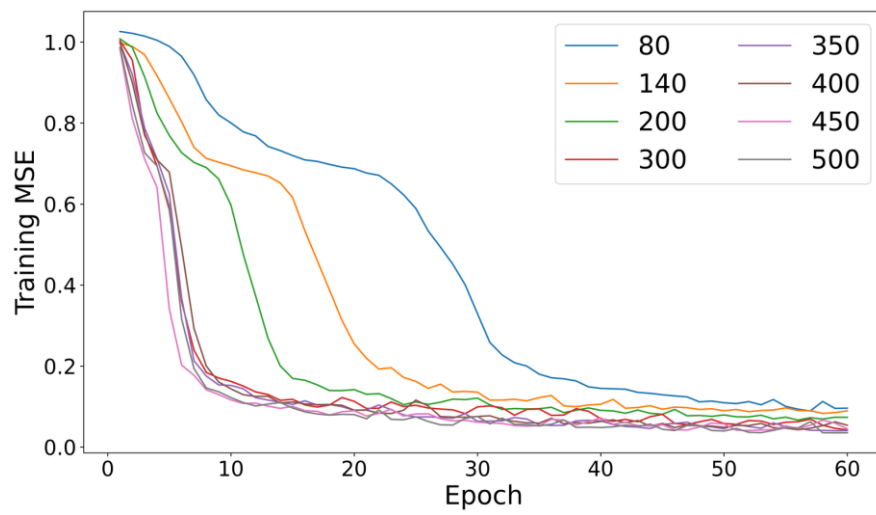


Fig. 27: Training mean squared error of the U-DeepONet for liquid-fraction prediction over 60 epochs, for different numbers of training simulations.

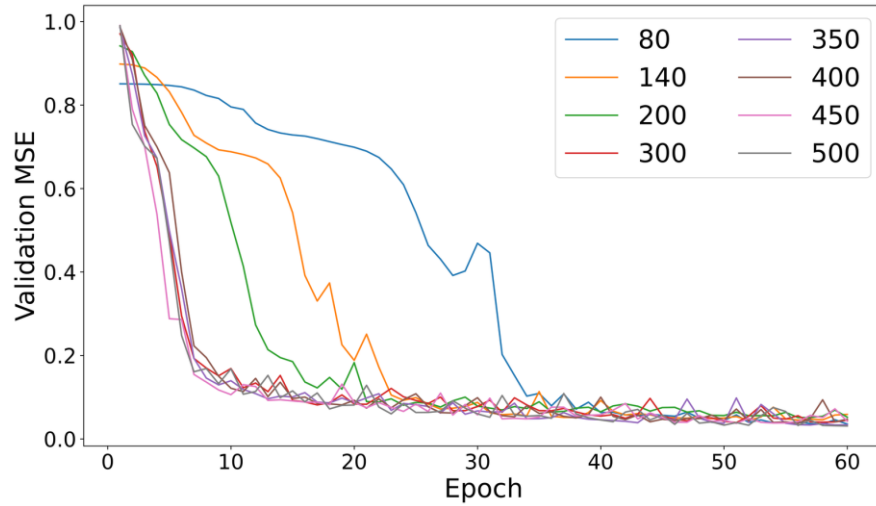


Fig. 28: Validation mean squared error for liquid-fraction prediction, for the same training-set sizes as Fig. 27.

The convergence behaviour in Figs. 25 to 28 is consistent across both quantities as earlier. All models show a rapid reduction in error over the first epochs, indicating that the dominant spatial and temporal characteristics of the response are learned quickly, after which the rate of improvement falls and the error approaches a stable level while the finer gradients and phase-transition patterns are refined. Training and validation curves decrease together without a sustained rise in the validation error, so there is no evidence of significant overfitting at any dataset size.

The smallest sets behave visibly differently. With 80 and 140 simulations the training error falls more slowly and remains at a higher level for a larger part of the run, and the validation error fluctuates more, which is what would be expected when the training set does not cover the variability of the problem. The effect is stronger for temperature than for liquid fraction, and is most pronounced for the localised and hybrid heating cases, where changes in source geometry, position, strength and interaction with the surrounding field produce substantially different transient responses.

Table 5: Best training and validation mean squared error obtained by the U-DeepONet for temperature and liquid-fraction prediction over 60 epochs, for different numbers of training simulations. Errors are evaluated on normalised targets.

Training simulations	Temperature training MSE	Temperature validation MSE	Liquid fraction training MSE	Liquid fraction validation MSE
80	0.040844	0.037153	0.088959	0.035172
140	0.025032	0.021300	0.082957	0.044069
200	0.031283	0.017836	0.066546	0.051055
300	0.029402	0.017736	0.042189	0.038900
350	0.015544	0.011696	0.040174	0.033183
400	0.021512	0.014910	0.042663	0.037392
450	0.012238	0.010291	0.040796	0.037112
500	0.013364	0.009541	0.035045	0.030831

Table 5 quantifies the trend. The validation error falls from 0.037153 to 0.009541 for temperature and from 0.035172 to 0.030831 for liquid fraction as the dataset grows from 80 to 500 simulations. The improvement is not strictly monotonic at every intermediate size, which is consistent with the variability introduced by mini-batch optimisation and by the particular cases added at each step, but the direction is clear and the larger sets converge more consistently to the low-error regime. The curves flatten appreciably beyond roughly 400 simulations, so that further high-fidelity runs buy progressively less accuracy for the same cost. On this basis the 500-case dataset was adopted for the detailed prediction studies and for the cross-geometry analysis of Section 4.5.

The liquid fraction behaves less regularly than the temperature. Its validation error at 200 simulations is higher than at 80, and it recovers only as the dataset grows further. This is consistent with the error being dominated by the placement of a thin melt front rather than by the smoothness of a field: a small displacement of the front saturates the local error across the interface, so the metric is sensitive to which particular cases happen to fall in the validation split at each size.

4.3 1D and 2D Prediction Plots and Inference

The initial DeepONet surrogate model was tested in a simpler 1D setting by comparing a few randomly chosen test samples to their ground-truth solutions. In Fig. 29(a), the top panel shows the input heat-source field $Q(x)$ for each test case. The bottom panel shows the difference between the true temperature field and the temperature field predicted by DeepONet.

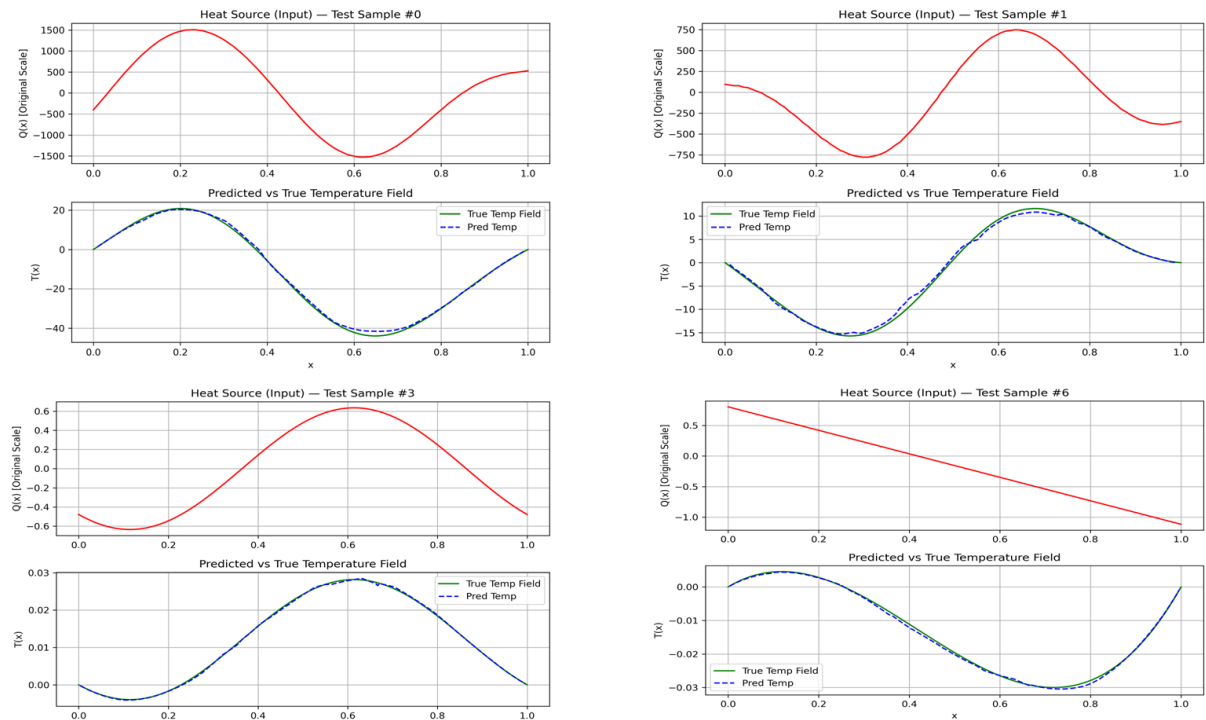


Fig. 29(a): Comparison of predicted and ground-truth 1D temperature fields for four representative test samples. Each plot shows the input heat-source function $Q(x)$ (top) and the corresponding true vs. DeepONet-predicted temperature field $T(x)$ (bottom).

The predicted curves closely follow the real temperature distribution in all samples, accurately showing both the amplitude and phase of the thermal response. Small deviations appear mainly in regions of steep gradients, but overall, the DeepONet model demonstrates excellent interpolation accuracy in the 1D regime.

The Pointwise DeepONet surrogate model was further tested in 2D setting by comparing a few randomly chosen test samples to their ground-truth solutions. For example, in Fig 29(b), row 1 shows the input heat-source field $Q(x, y)$ for each test case. Rows 2 and 3 show the difference between the true temperature field and the temperature field predicted by 2D DeepONet model. Rows 4 and 5 show the difference between the true liquid fraction

field and the liquid fraction field predicted by 2D DeepONet model. Predictions are made over total seven-time snapshots including 100, 250, 400, 600, 1000, 1200 and 1500s.

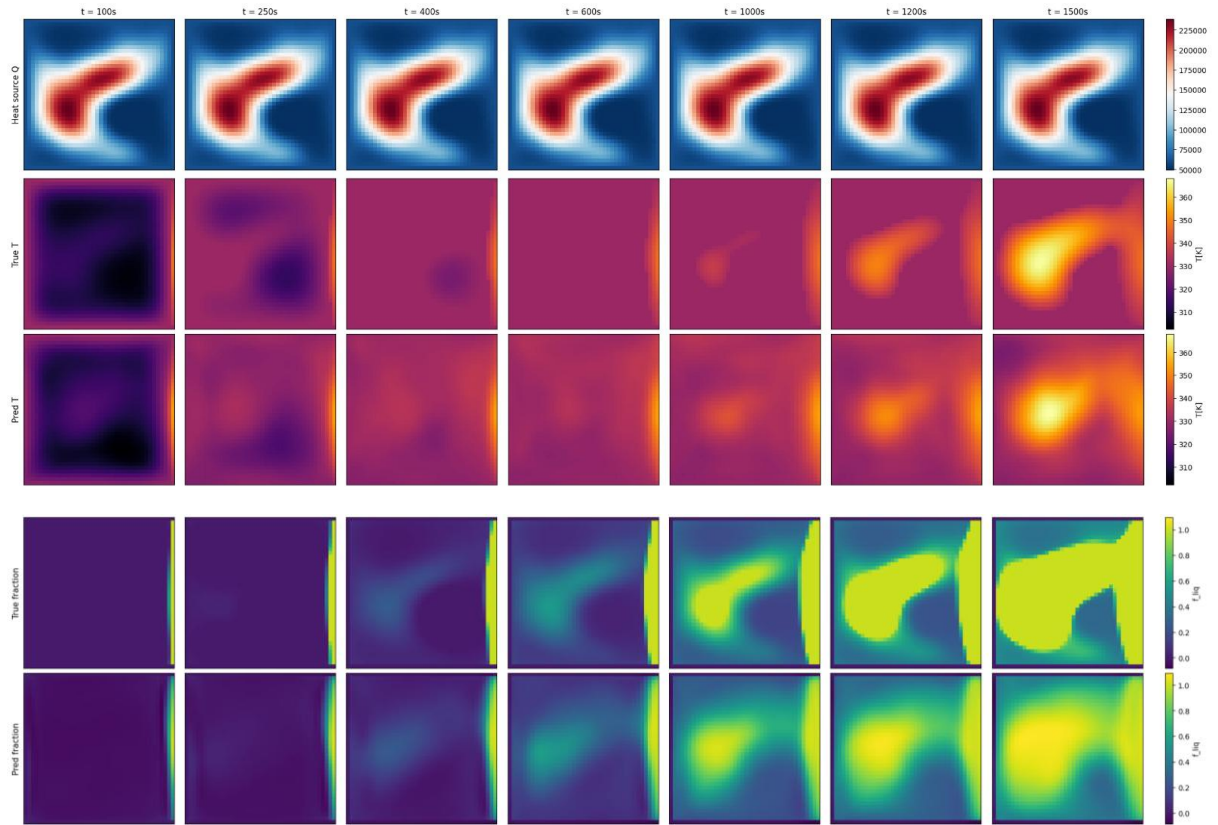
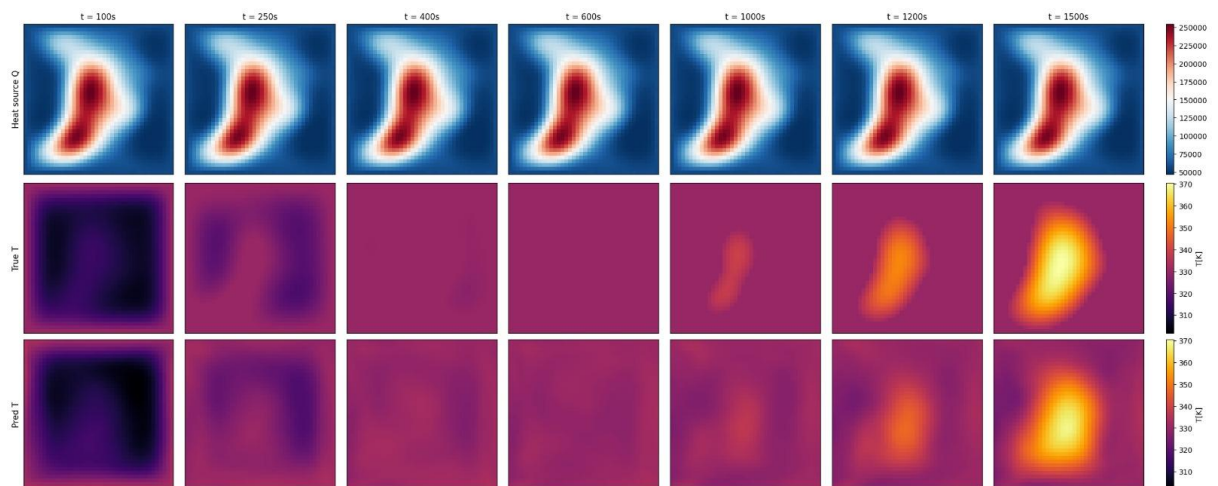


Fig. 29(b): Case-1 of comparison of predicted and ground-truth 2D temperature fields and corresponding liquid fraction fields for (100, 250, 400, 600, 1000, 1200 and 1500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true vs. DeepONet-predicted temperature field $T(x, y)$ (row 2 & 3) and the corresponding true vs. DeepONet-predicted liquid fraction field $F(x, y)$ (row 4 & 5)



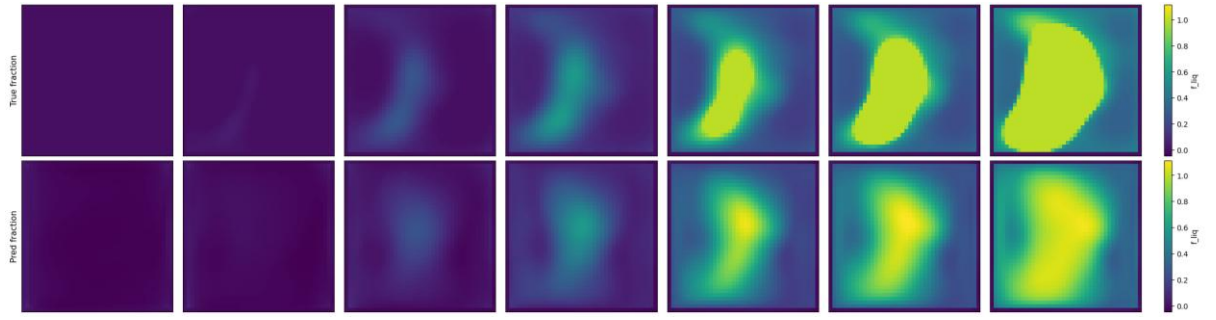


Fig. 29(c): Case-2 of comparison of predicted and true temperature fields and liquid fraction fields

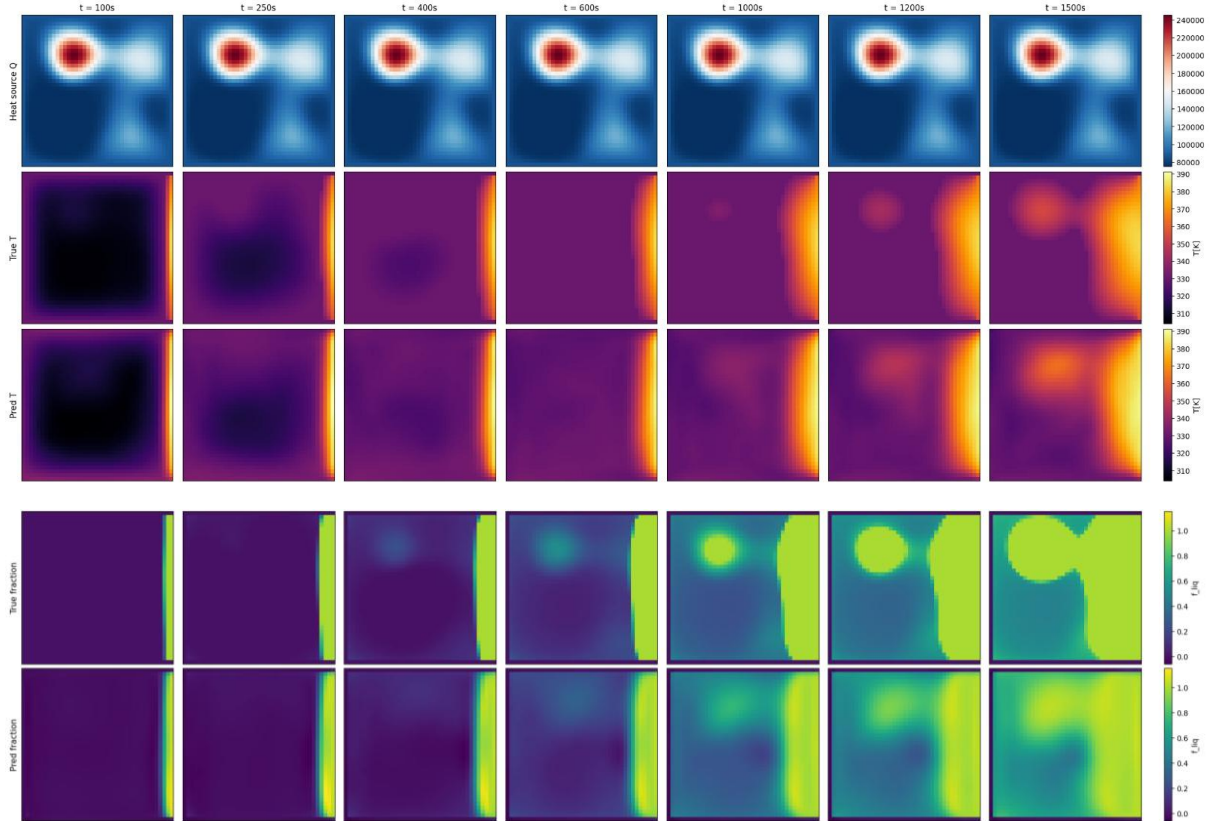


Fig. 29(d): Case-3 of comparison of predicted and true temperature fields and corresponding liquid fraction fields

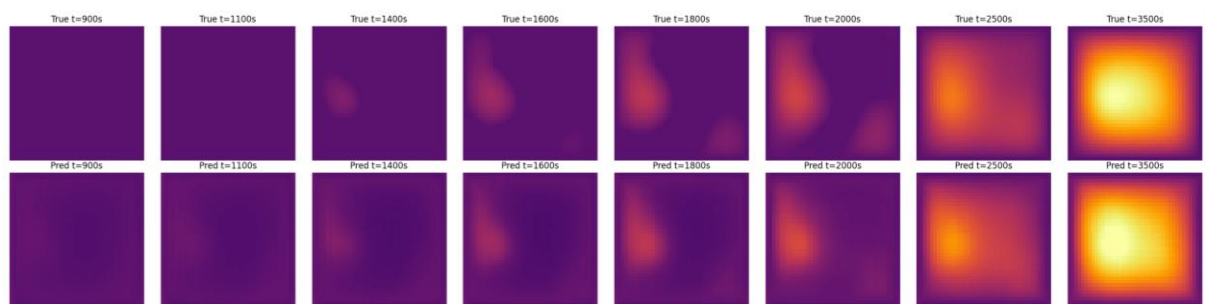
However, the Pointwise DeepONet model was training time intensive and required significant memory usage for the large batch size which had to be used to bring the number of iterations per epoch under the practical training time constraints. Compared to that the U-DeepONet provided results with atleast equal or better accuracy for significantly less amount of time. The comparison between the models is done in table 4, as per the observations done during training.

Table 6: Comparison of observed stats of Pointwise and U-DeepONet model architecture ^[15]

Feature	Pointwise	U-DeepONet
Training samples	2.8M	200
Batch Size	16384	16
Iterations per epoch	~180	~12
Training time	~7 Hours	~30 min
Spatial structure	Lost	Preserved
Melting front sharpness	Smoothened	Better captured

Prediction plots from U-DeepONet

In fig. 26, we have two cases with the true temperature in row 1 and predicted temperature maps in row 2 obtained from the new model. The plots look promising and accurate as well. Here the spatial heat source, boundary conditions were taken in the branch network as a multi-channel input image. 200 sets of data were taken and trained for 60 epochs with a batch size of 16.



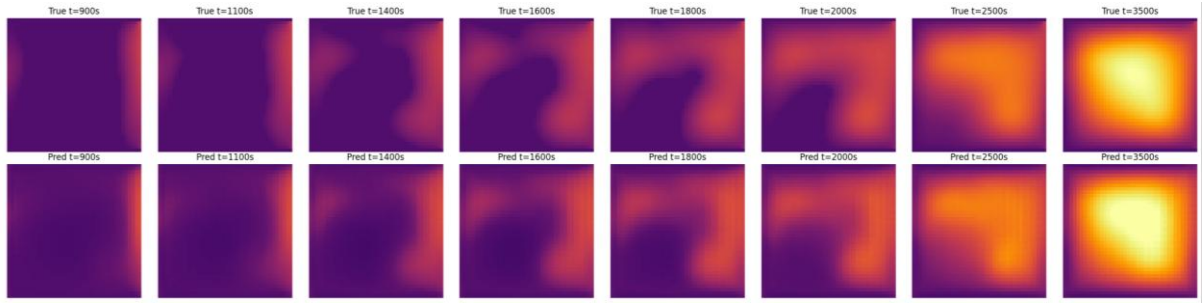


Fig. 30: Comparison of true and predicted temperature fields in U-DeepONet model

Prediction plots for generalised dataset

Next, we trained the model on the generalised dataset containing the battery sources as well. Rotated prismatic heat sources, circular and elliptic heat sources were considered as can be seen in plots present in fig. 27(a), 27(b), 27(c), 27(d) and 27(e). For example, in Fig 31(a), row 1 shows the input heat-source field $Q(x, y)$ for each test case. Rows 2 and 3 show the true temperature field and the liquid fraction field computed by forward euler explicit scheme in CFD. Row 4 shows the liquid fraction field predicted by 2D U-DeepONet model. Predictions are made over total seven-time snapshots including 100, 500, 900, 1300, 1600, 2000 and 3500s.

Fig. 31(a) and 27(b) are the plots inferred from the model which was trained on a dataset size of 500 cases for only generalised prismatic heat sources. These are plots based on the unseen cases which were stored beforehand during data generation process.

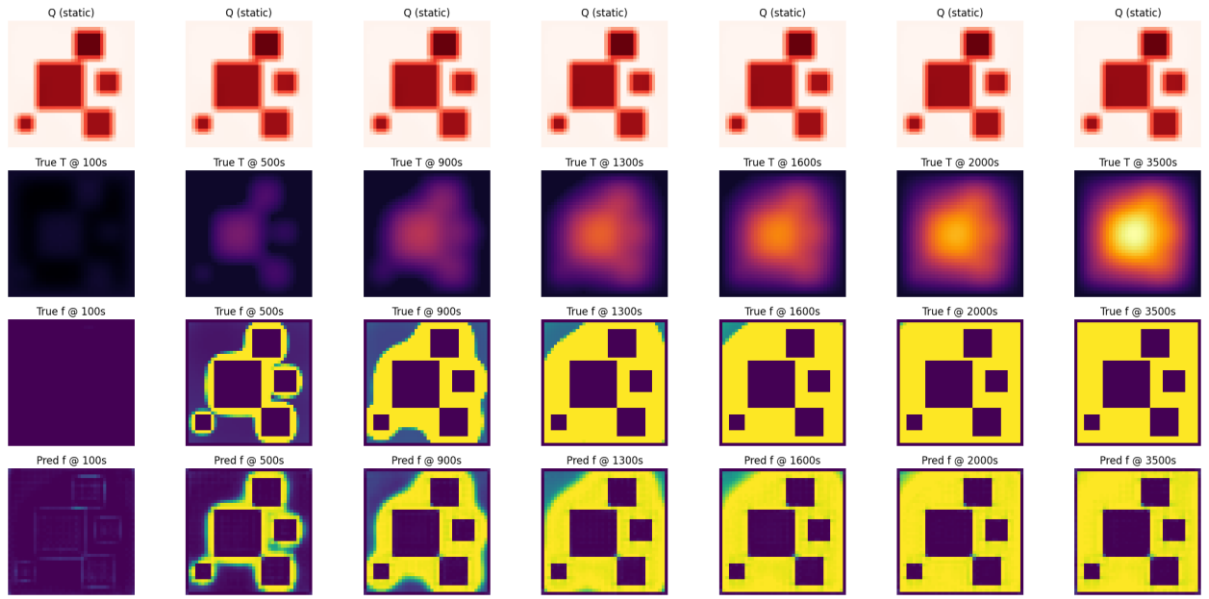


Fig. 31(a): Case-1 of comparison of predicted and ground-truth 2D liquid fraction fields for (100, 500, 900, 1300, 1600, 2000 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true temperature field $T(x, y)$ and liquid fraction field $F(x, y)$ (row 2 & 3) and the corresponding DeepONet-predicted liquid fraction field $F(x, y)$ (row 4)

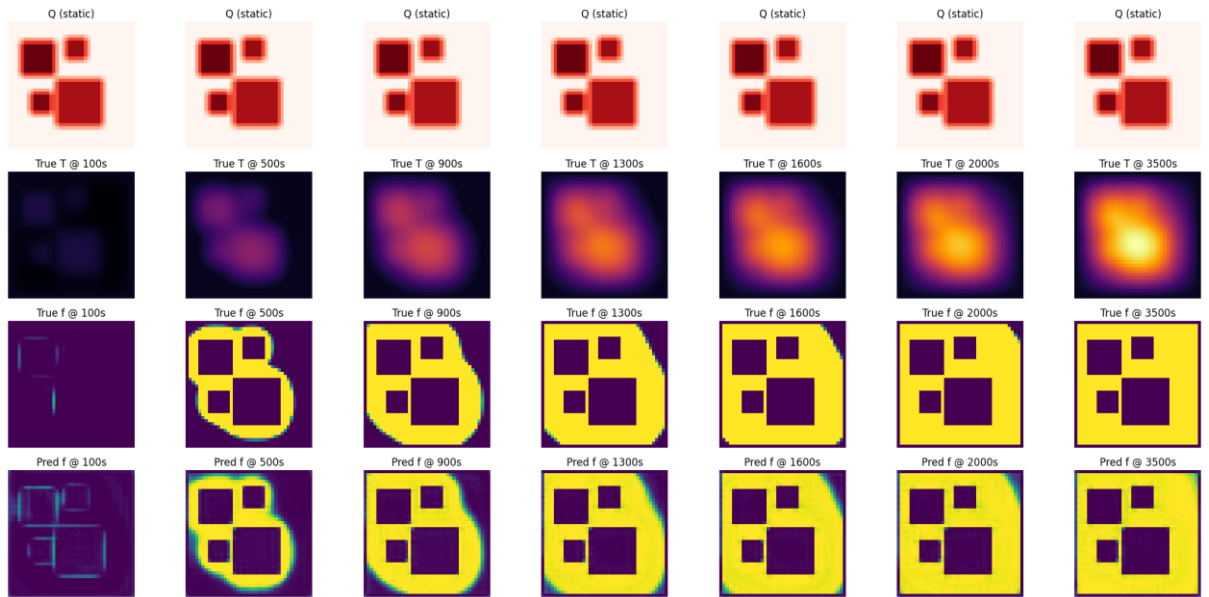


Fig. 31(b): Case-2 of comparison of predicted and true liquid fraction fields

Fig 31(c), 31(d) and 31(e) are the plots inferred from a model which was completely trained on the generalised dataset of 130 cases containing prismatic, circular and elliptic heat sources along with rotation. These are also plots based on the unseen cases which were stored beforehand during data generation process.

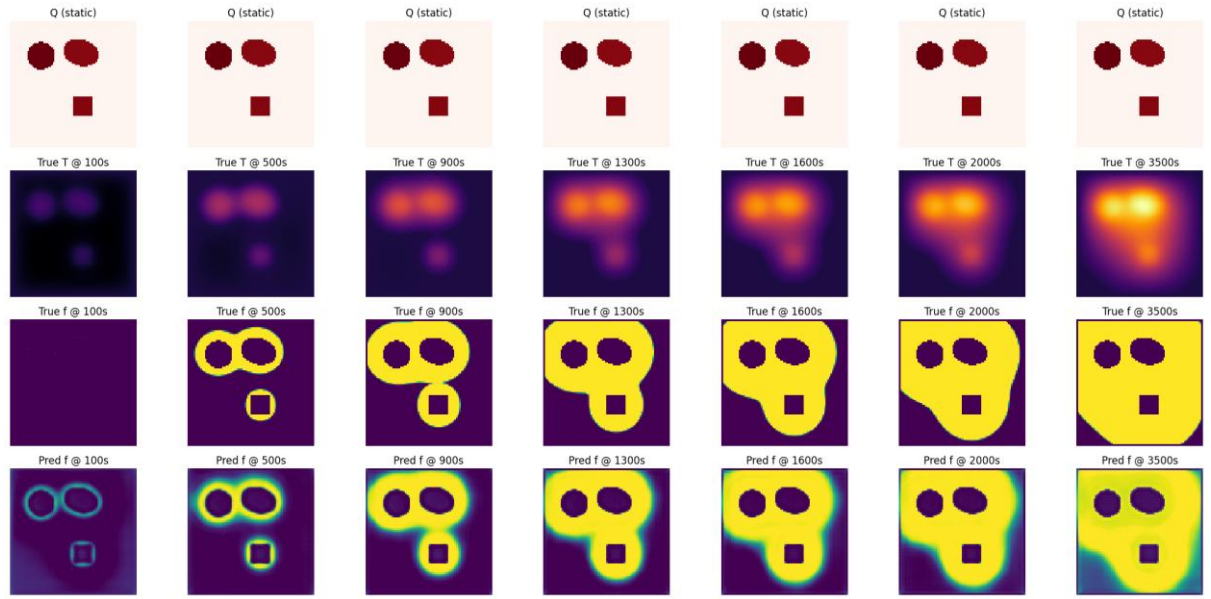


Fig. 31(c): Case-3 of comparison of predicted and ground-truth 2D liquid fraction fields for (100, 500, 900, 1300, 1600, 2000 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true temperature field $T(x, y)$ and liquid fraction field $F(x, y)$ (row 2 & 3) and the corresponding DeepONet-predicted liquid fraction field $F(x, y)$ (row 4)

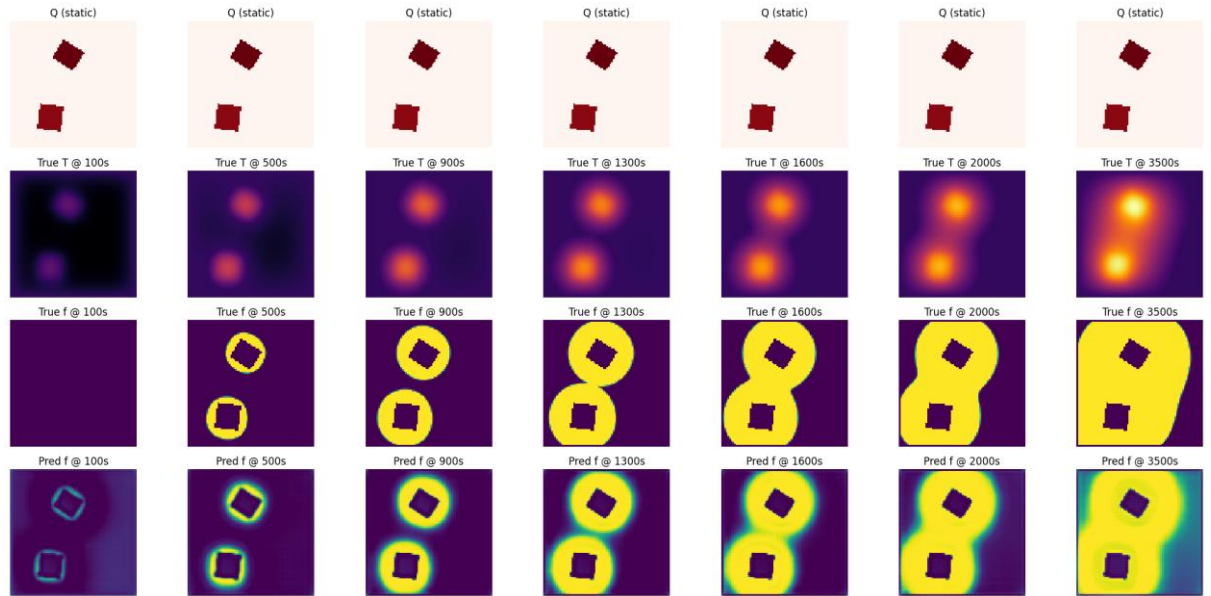


Fig. 31(d): Case-4 of comparison of predicted and true liquid fraction fields

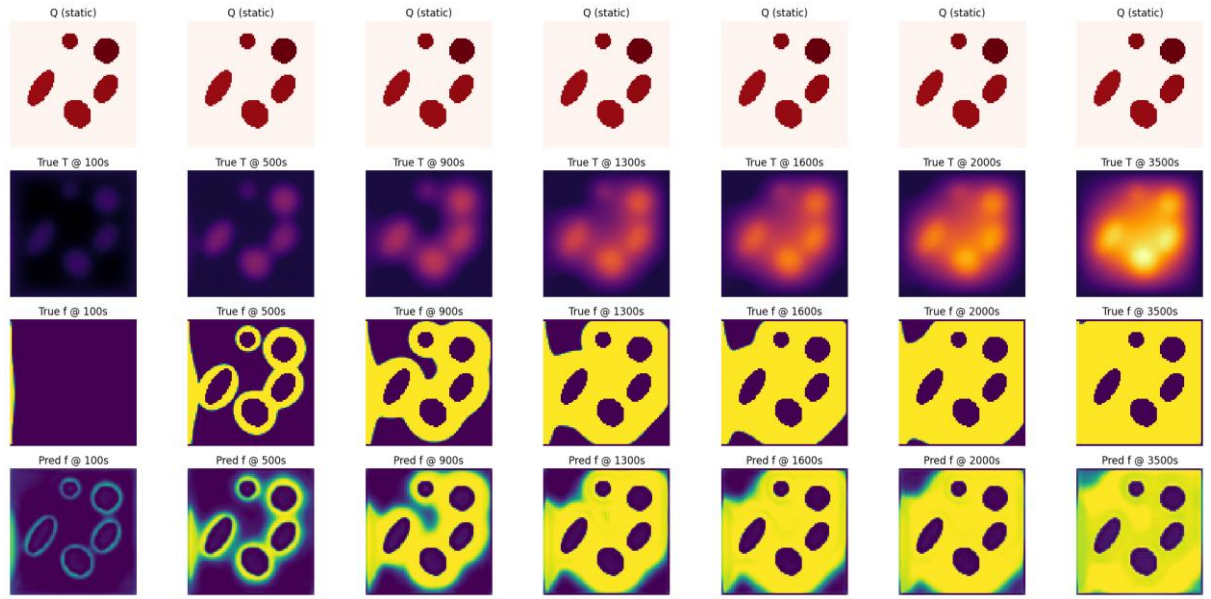


Fig. 31(e): Case-5 of comparison of predicted and true liquid fraction fields

4.4 Slice Based 3D U-DeepONet Results

Finally, we trained the U-DeepONet model modified to handle the 2D slices for 3D PCM domains. For each of the slices along 3 orthogonal axes x, y, z , the dataset stores the temperature fields and corresponding liquid fraction fields at multiple times in the form of snapshots. Here we have shown the prediction plots with respect to ground truth labels for both temperature maps and liquid fraction maps.

The training behaviour of surrogate model trained on temperature maps is shown in figure 28 and 29 for normalized error and physical error respectively. Both the training and validation root mean square error (RMSE) decrease steadily with increasing epochs, indicating stable convergence of the model. The model also gradually converges in the later stages of training. Some oscillations in the validation curves can be expected because of the non-linear temporal phase change behaviour and indication of diversity of heating conditions present in the dataset for different cases. Overall, the convergence behaviour does show that the current model framework is capable of learning 3D temporal phase change phenomenon accurately while also ensuring good generalization performance.

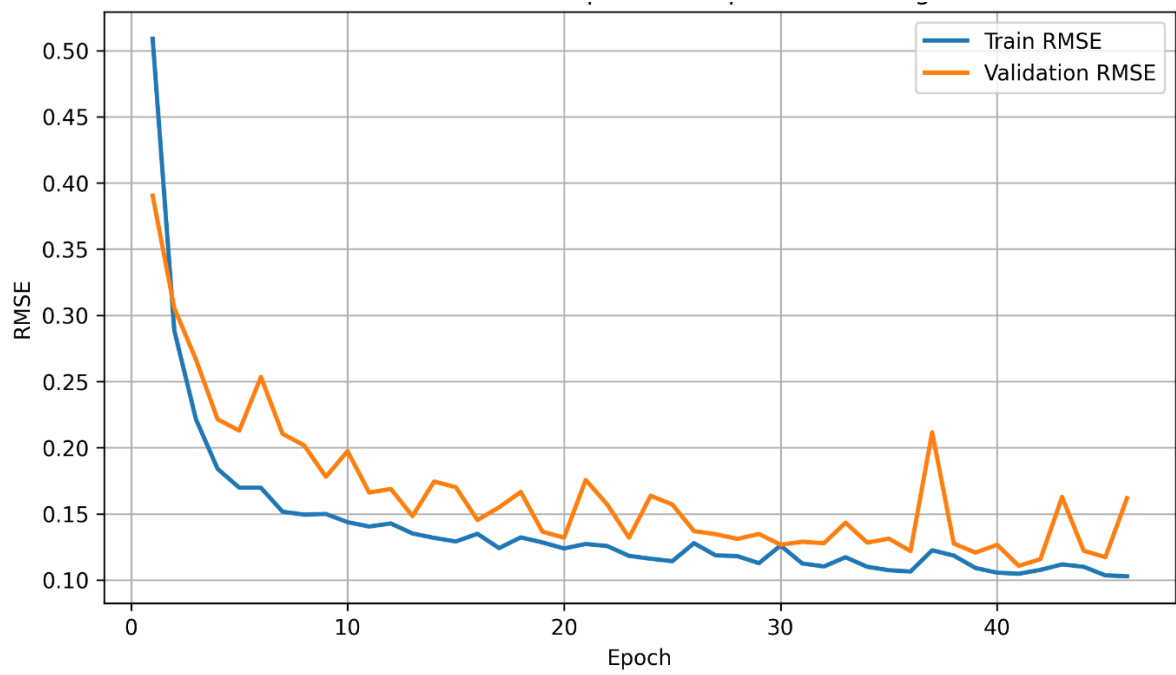


Fig. 32: Training and Validation (normalized) RMSE history for 3D U-DeepONet Temperature Surrogate model

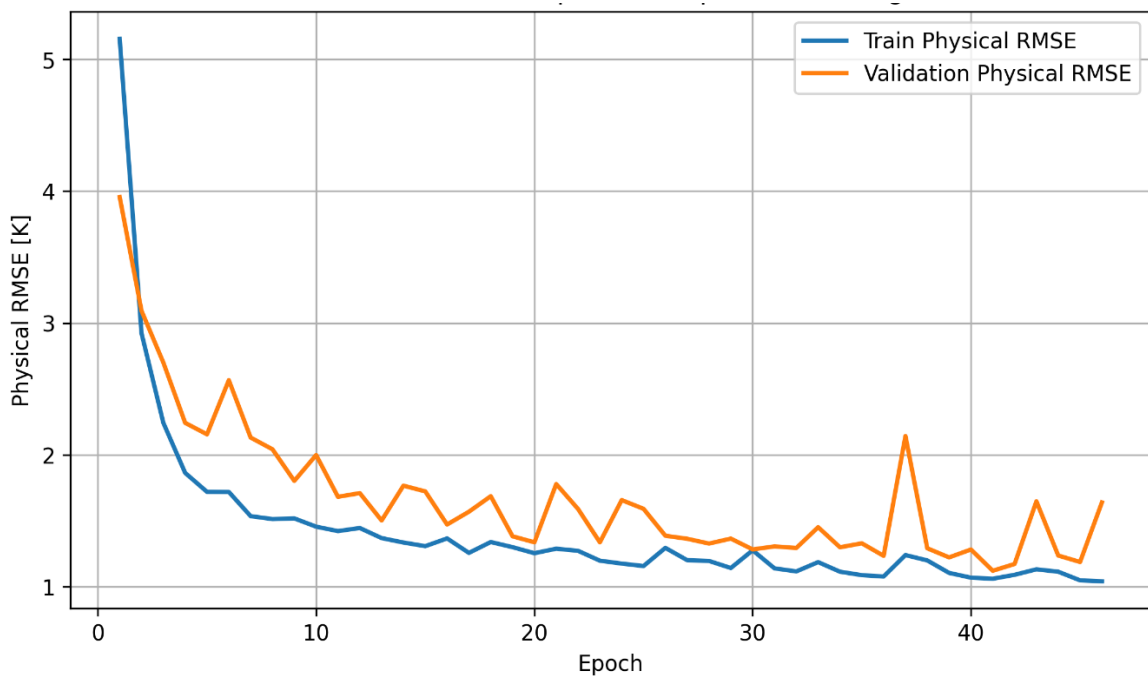


Fig. 33: Training and Validation (physical) RMSE history for 3D U-DeepONet Temperature Surrogate model

Now a single 3D case stored in the unseen part of the dataset (which was neither used for training or validation) is taken to check the prediction results from the trained model. In

figure 34, we have the prediction plots for temperature fields of different x-slices of the same 3D case. Figure 34(a) shows the 2D gaussian boundary conditions used in this case. In figure 35 and 36, we have the corresponding temperature maps of different y-slices and z-slices respectively.

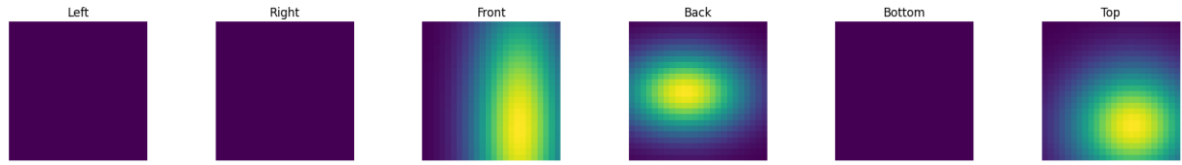


Fig. 34(a): 2D gaussian boundary conditions across left, right, front, back, bottom and top faces as per the used orientation for prediction of temperature plots across different slices

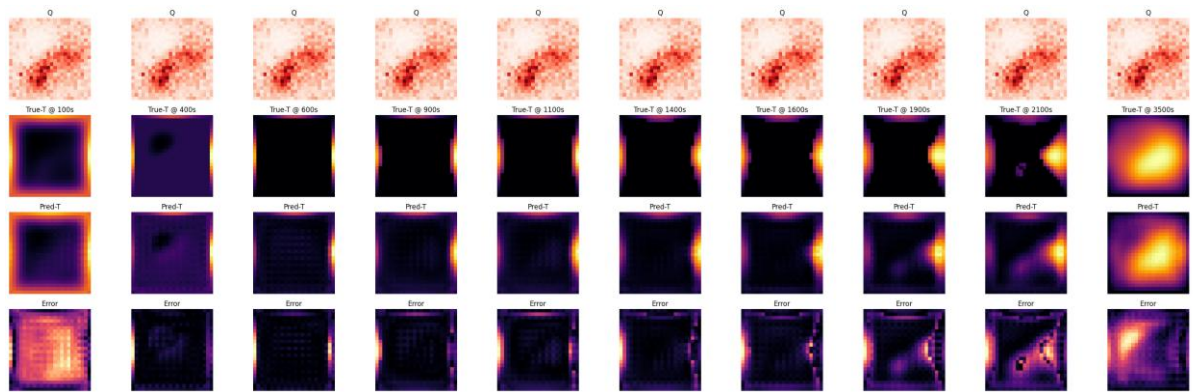


Fig. 34(b): Comparison of predicted and ground-truth 3D slice-based temperature fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true temperature field $T(x, y)$ and predicted temperature field $Pred_T(x, y)$ (row 2 & 3) and the corresponding absolute error maps(row 4) for X-Slice-1

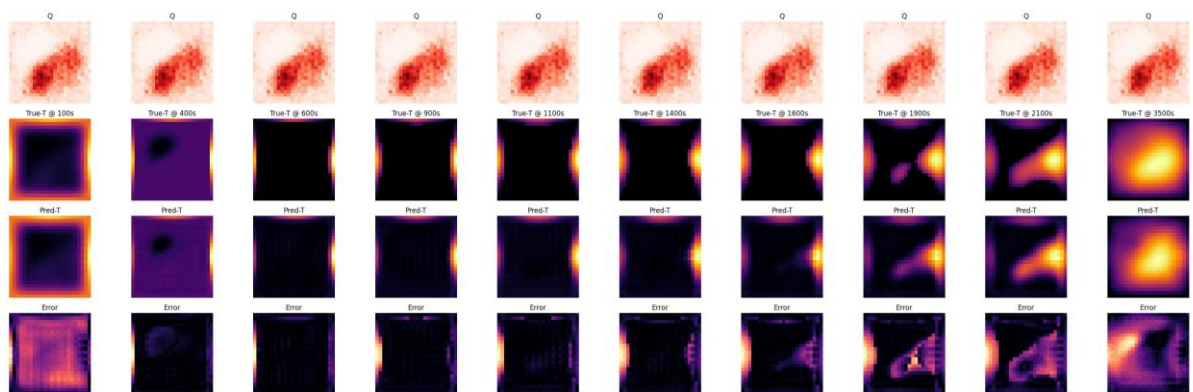


Fig. 34(c): Comparison of predicted and ground-truth 3D slice-based temperature fields for X-Slice-2

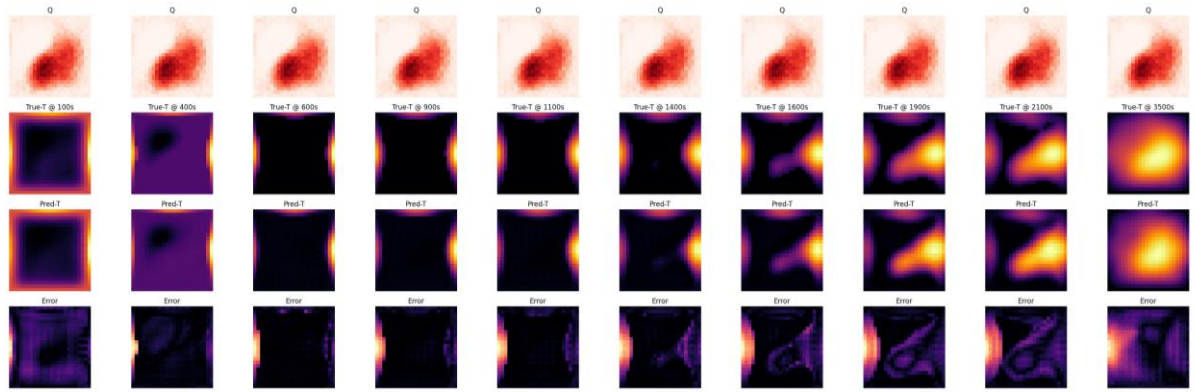


Fig. 34(d): Comparison of predicted and ground-truth 3D slice-based temperature fields for X-Slice-3

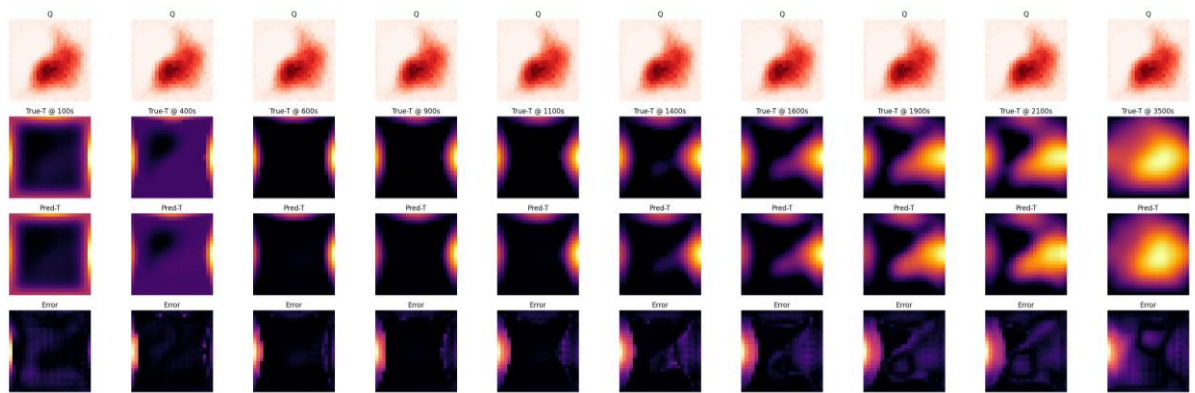


Fig. 34(e): Comparison of predicted and ground-truth 3D slice-based temperature fields for X-Slice-4

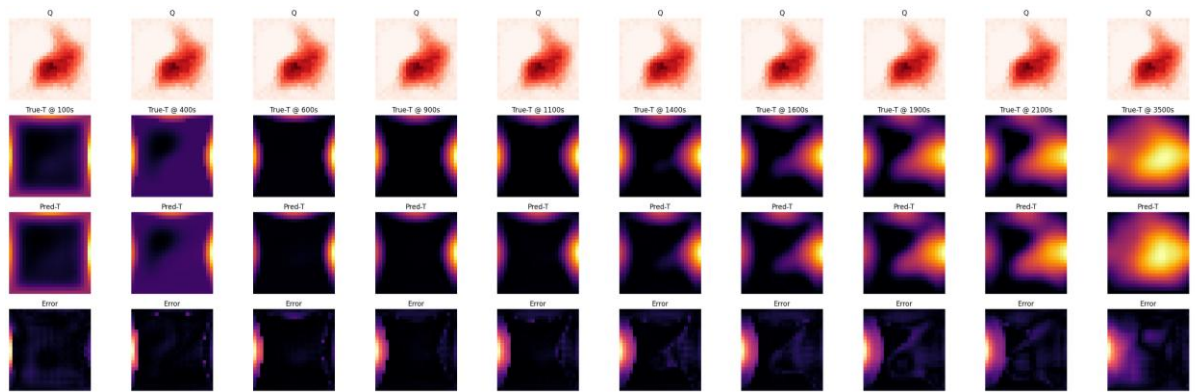


Fig. 34(f): Comparison of predicted and ground-truth 3D slice-based temperature fields for X-Slice-5

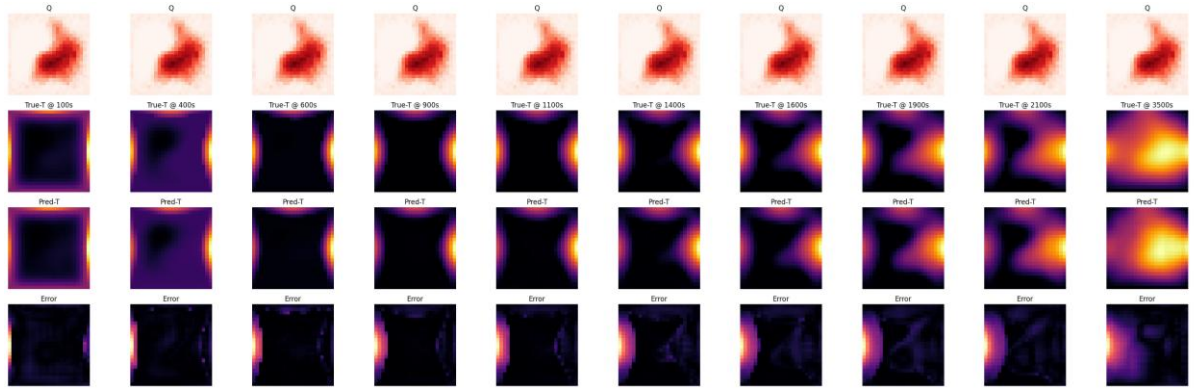


Fig. 34(g): Comparison of predicted and ground-truth 3D slice-based temperature fields for X-Slice-6

Next comes the temperature prediction plots for different y-slices as shown in figure 31 for the same boundary conditions mentioned in figure 30(a).

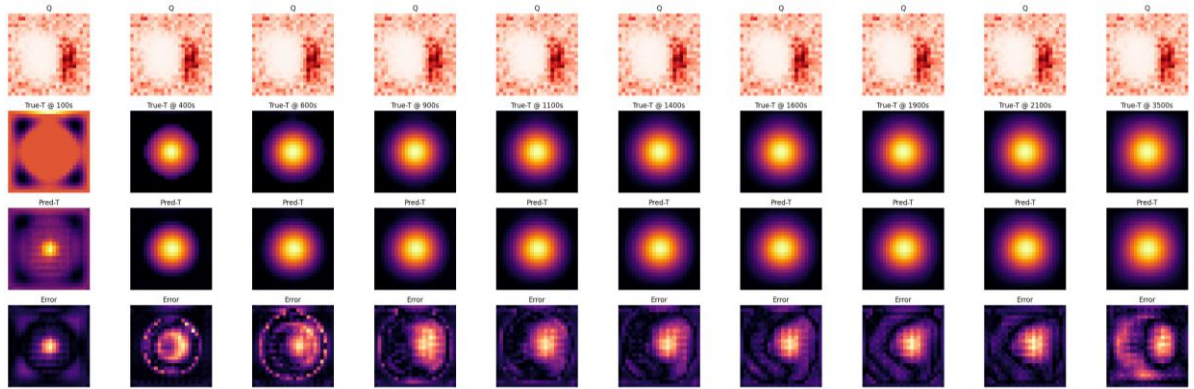


Fig. 35(a): Comparison of predicted and ground-truth 3D slice-based temperature fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true temperature field $T(x, y)$ and predicted temperature field $Pred_T(x, y)$ (row 2 & 3) and the corresponding absolute error maps (row 4) for Y-Slice-1

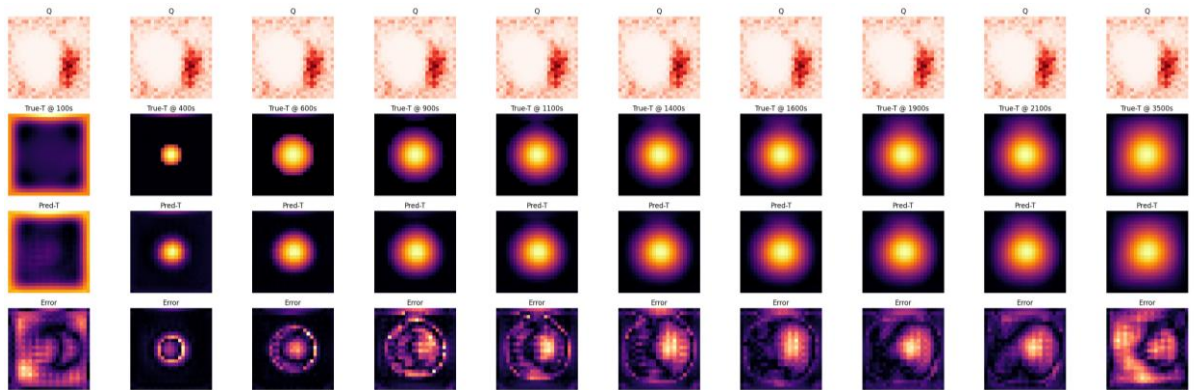


Fig. 35(b): Comparison of predicted and ground-truth 3D slice-based temperature fields for Y-Slice-2

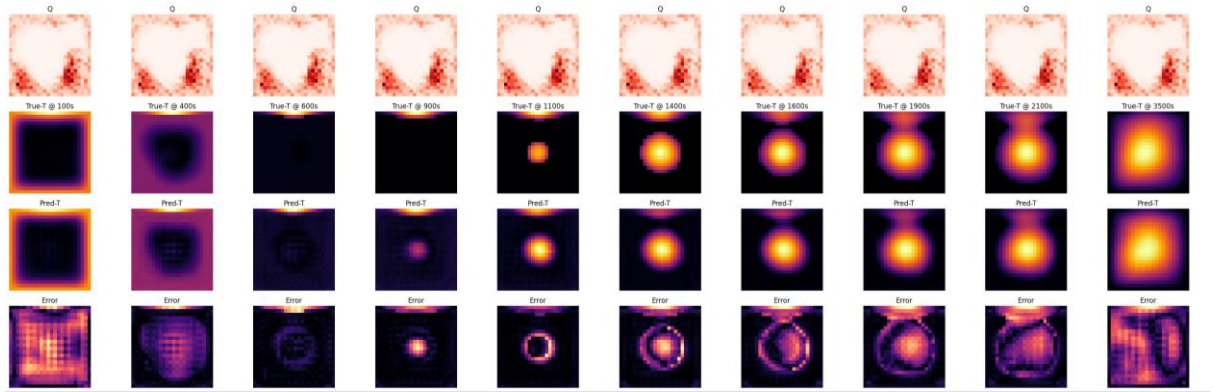


Fig. 35(c): Comparison of predicted and ground-truth 3D slice-based temperature fields for Y-Slice-3

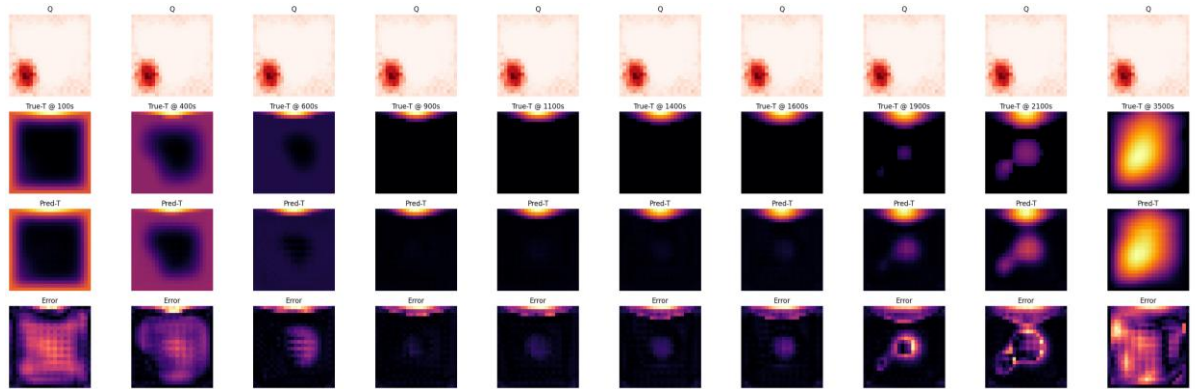


Fig. 35(d): Comparison of predicted and ground-truth 3D slice-based temperature fields for Y-Slice-4

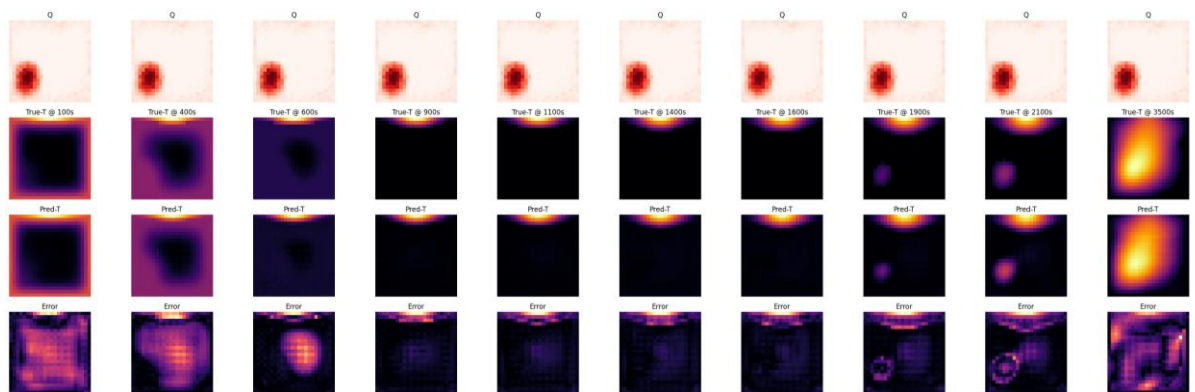


Fig. 35(e): Comparison of predicted and ground-truth 3D slice-based temperature fields for Y-Slice-5

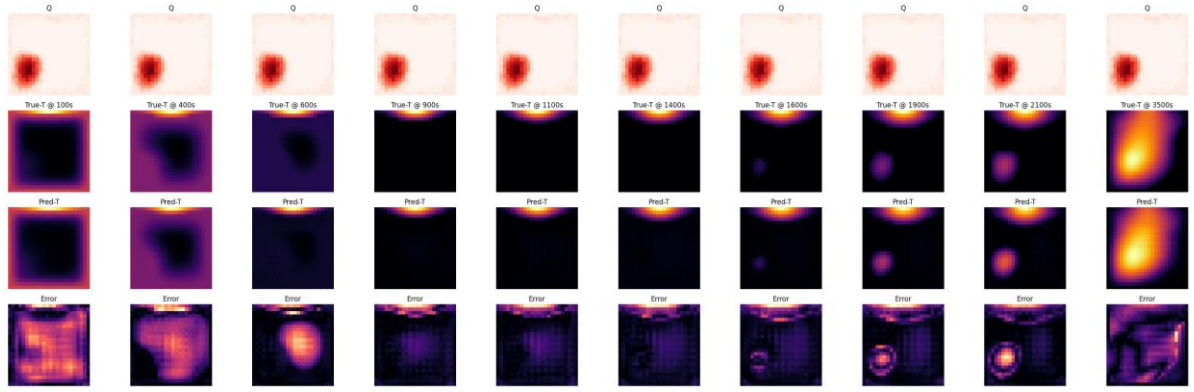


Fig. 35(f): Comparison of predicted and ground-truth 3D slice-based temperature fields for Y-Slice-6

Next comes the temperature prediction plots for different z-slices as shown in figure 36 for the same boundary conditions mentioned in figure 34(a).

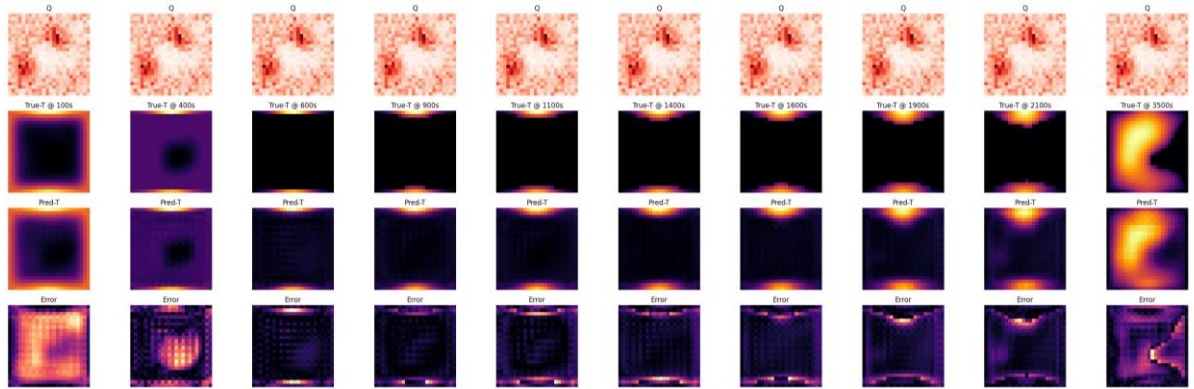


Fig. 36(a): Comparison of predicted and ground-truth 3D slice-based temperature fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true temperature field $T(x, y)$ and predicted temperature field $Pred_T(x, y)$ (row 2 & 3) and the corresponding absolute error maps(row 4) for Z-Slice-1

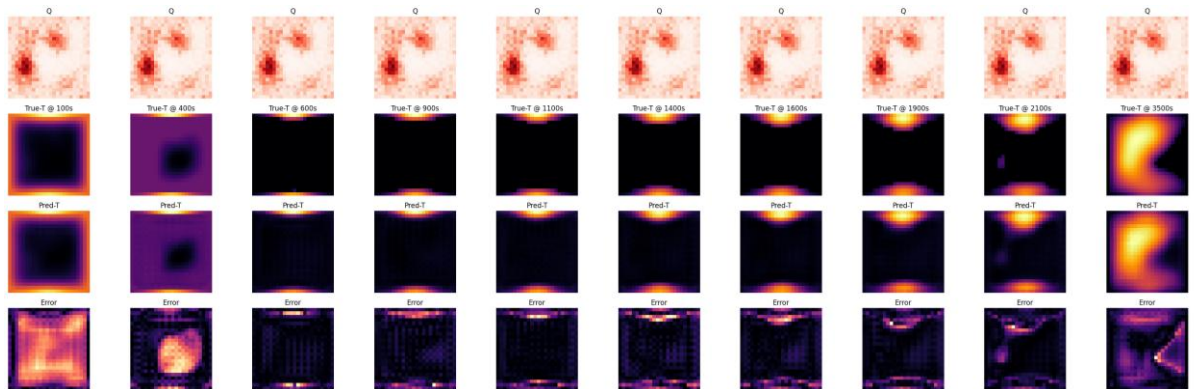


Fig. 36(b): Comparison of predicted and ground-truth 3D slice-based temperature fields for Z-Slice-2

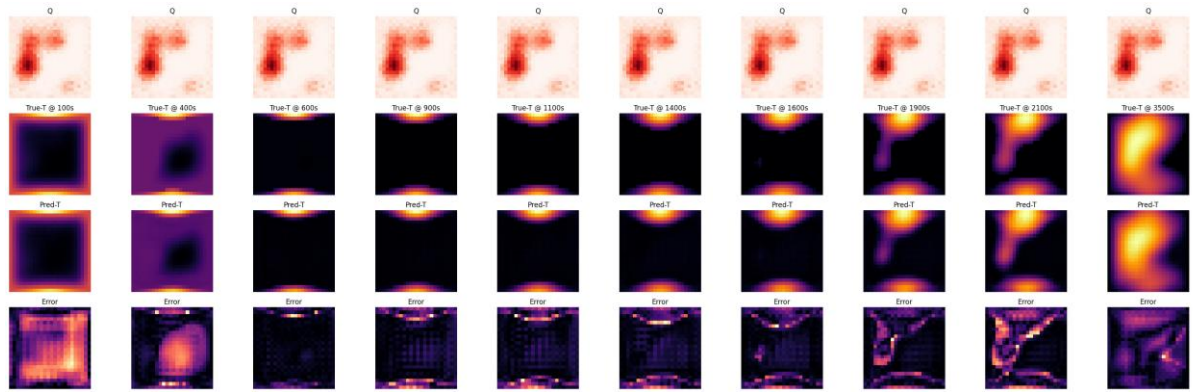


Fig. 36(c): Comparison of predicted and ground-truth 3D slice-based temperature fields for Z-Slice-3

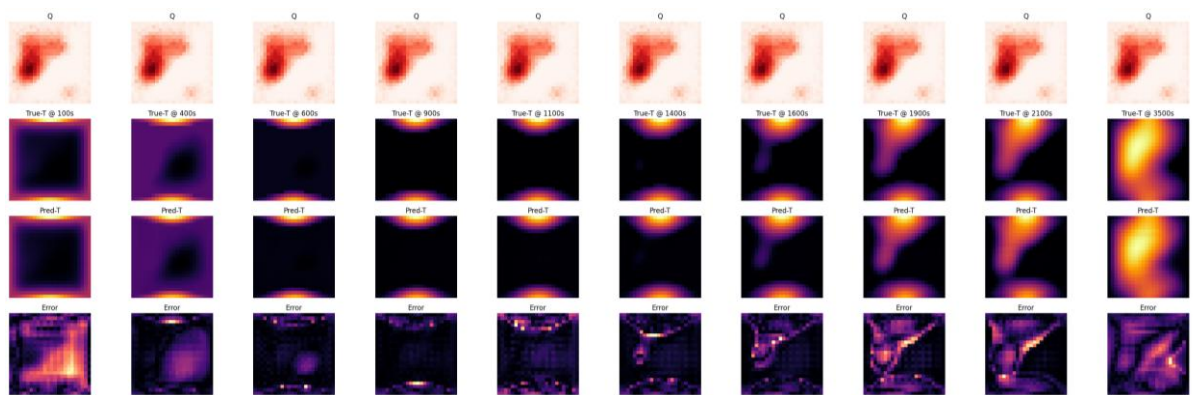


Fig. 36(d): Comparison of predicted and ground-truth 3D slice-based temperature fields for Z-Slice-4

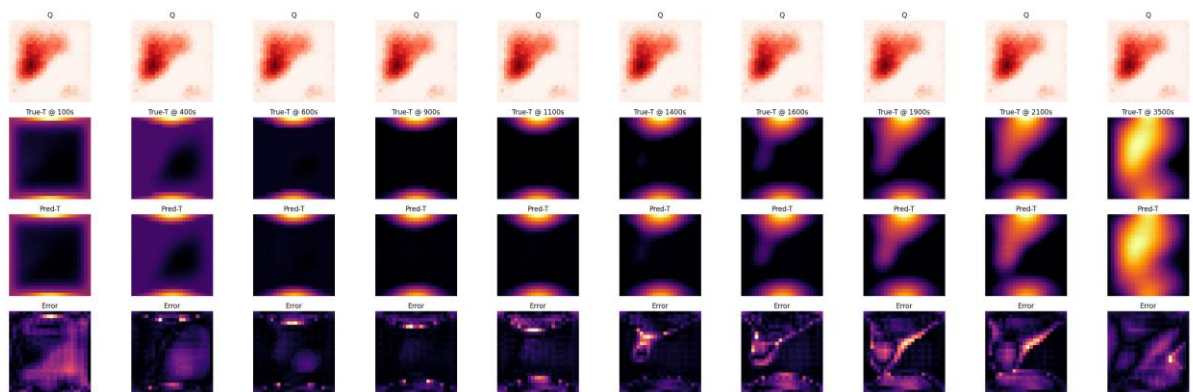


Fig. 36(e): Comparison of predicted and ground-truth 3D slice-based temperature fields for Z-Slice-5

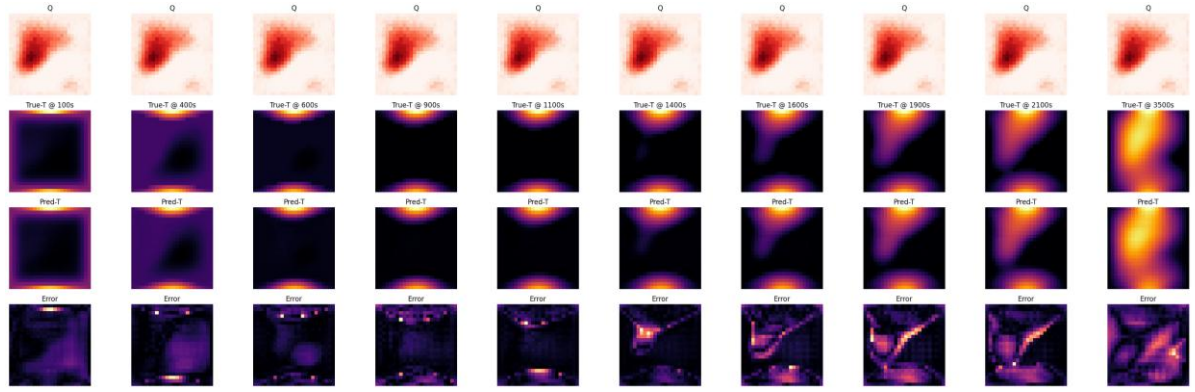


Fig. 36(f): Comparison of predicted and ground-truth 3D slice-based temperature fields for Z-Slice-6

As we saw, the predicted temperature fields were quite accurate and showed strong agreement with the numerical solution plots across all 10-time snapshots. It was for a single case where we had 2D gaussian boundary conditions along the 6 faces of the cube domain.

Now, we go for the liquid fraction prediction plots in figure 37, 38 and 39 across x, y, z slices for a single 3D case whose 2D gaussian boundary conditions are shown in figure 37(a).

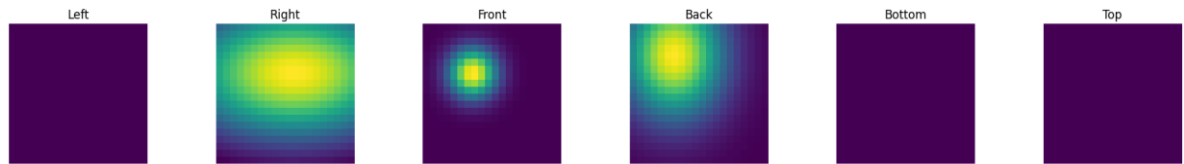


Fig. 37(a): 2D gaussian boundary conditions across left, right, front, back, bottom and top faces as per the used orientation for prediction of liquid fraction plots across different slices

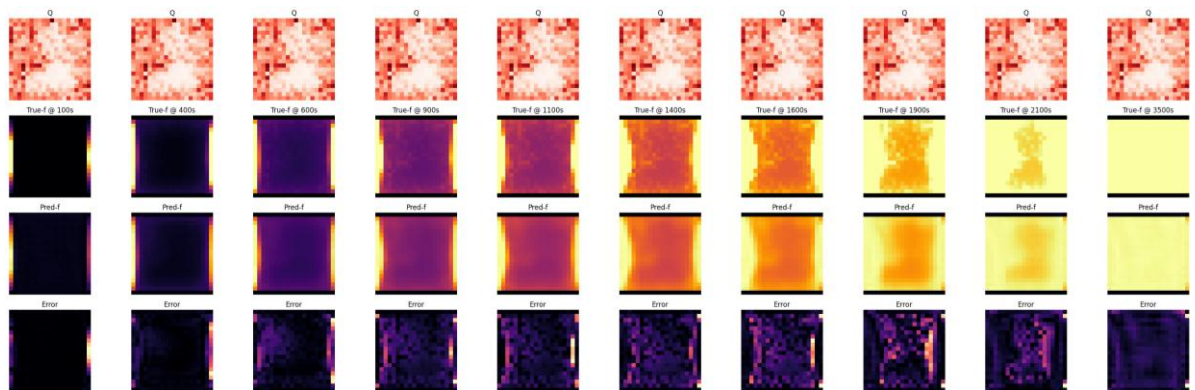


Fig. 37(b): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true liquid fraction field $F(x, y)$ and predicted liquid fraction field $Pred_F(x, y)$ (row 2 & 3) and the corresponding absolute error maps (row 4) for X-Slice-1

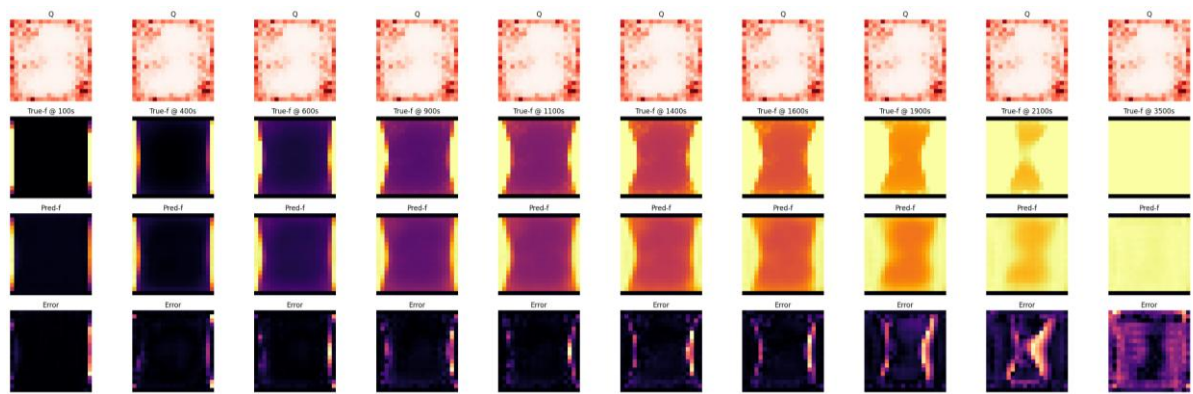


Fig. 37(c): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for X-Slice-2

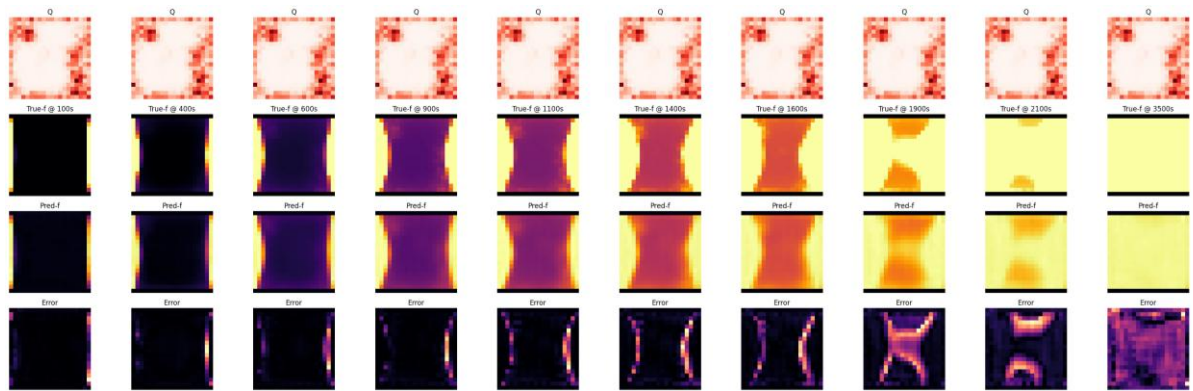


Fig. 37(d): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for X-Slice-3

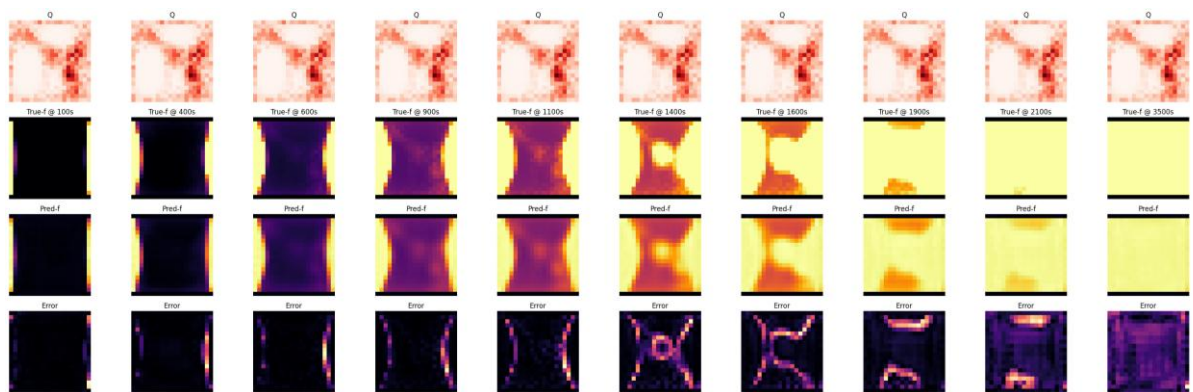


Fig. 37(e): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for X-Slice-4

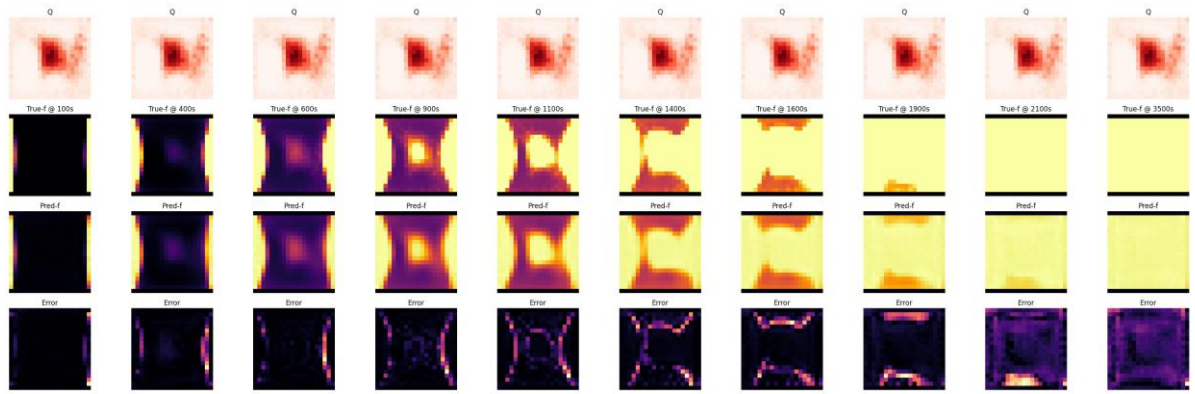


Fig. 37(f): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for X-Slice-5

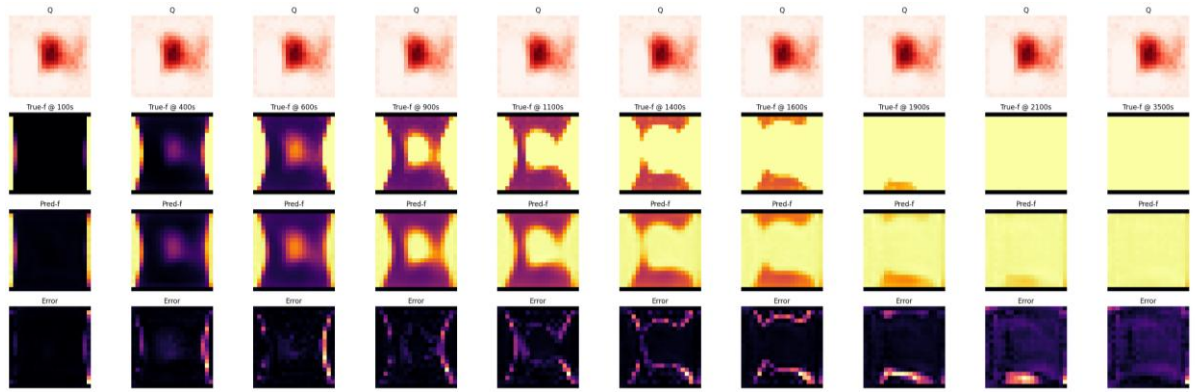


Fig. 37(g): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for X-Slice-6

Next comes the liquid fraction prediction plots for different y-slices as shown in figure 38 for the same boundary conditions mentioned in figure 37(a).

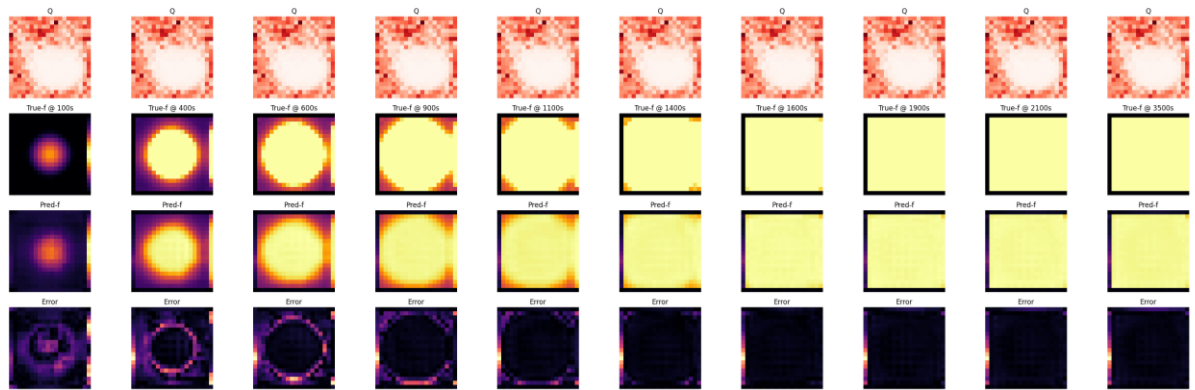


Fig. 38(a): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true liquid fraction field $F(x, y)$ and predicted liquid fraction field $Pred_F(x, y)$ (row 2 & 3) and the corresponding absolute error maps (row 4) for Y-Slice-1

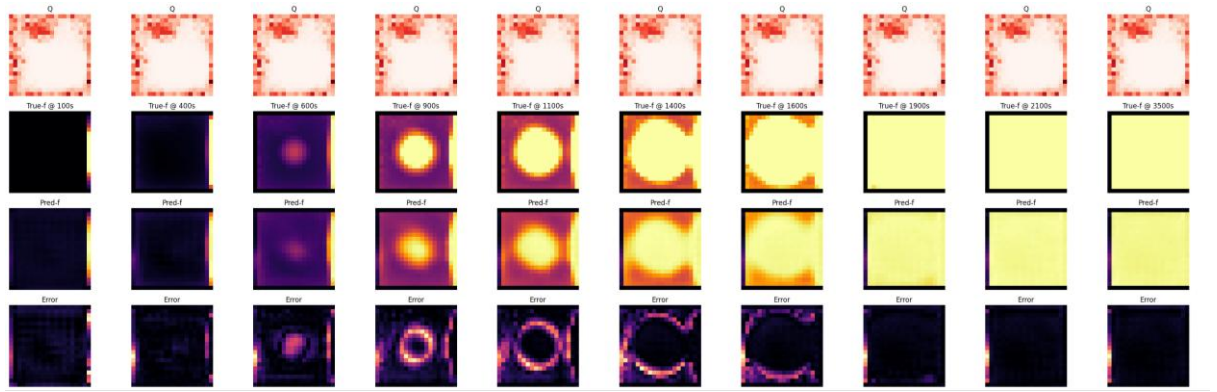


Fig. 38(b): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Y-Slice-2

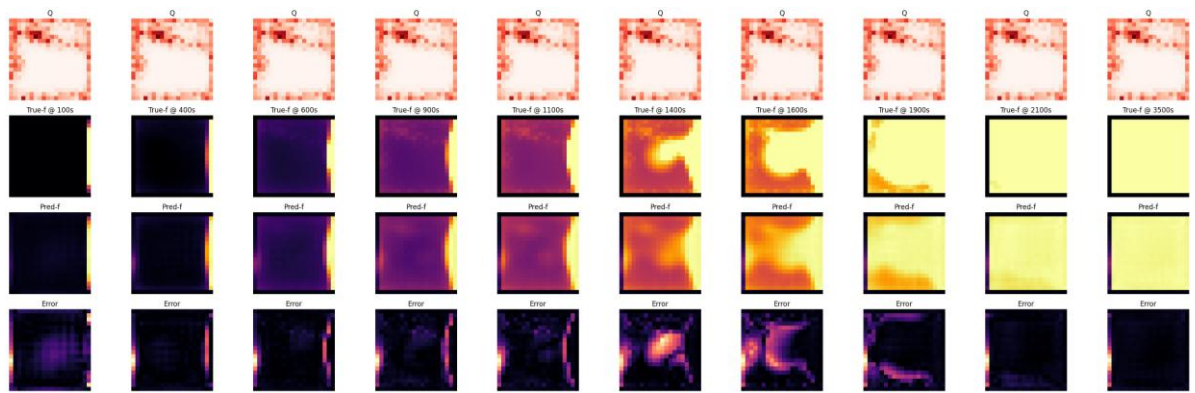


Fig. 38(c): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Y-Slice-3

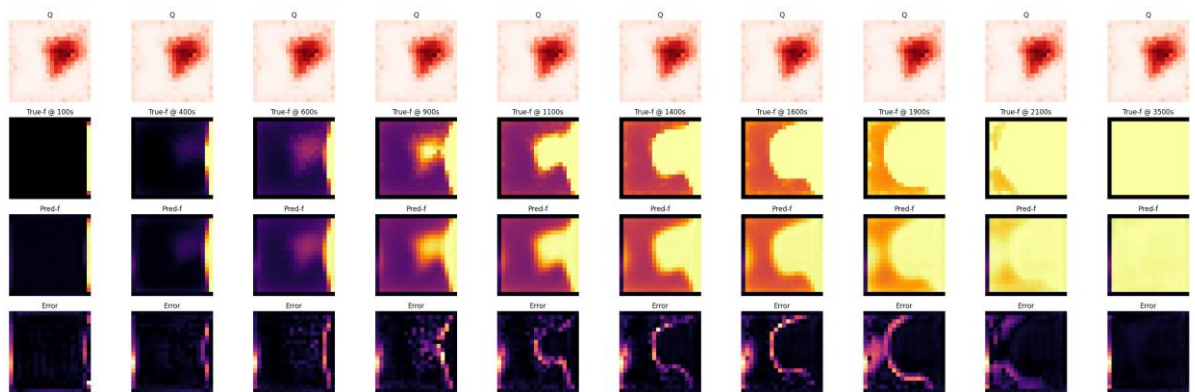


Fig. 38(d): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Y-Slice-4

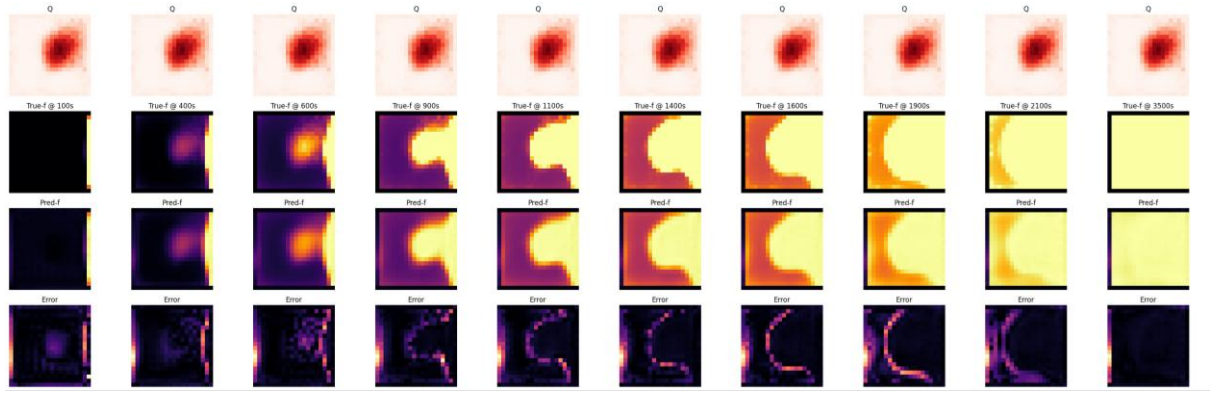


Fig. 38(e): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Y-Slice-5

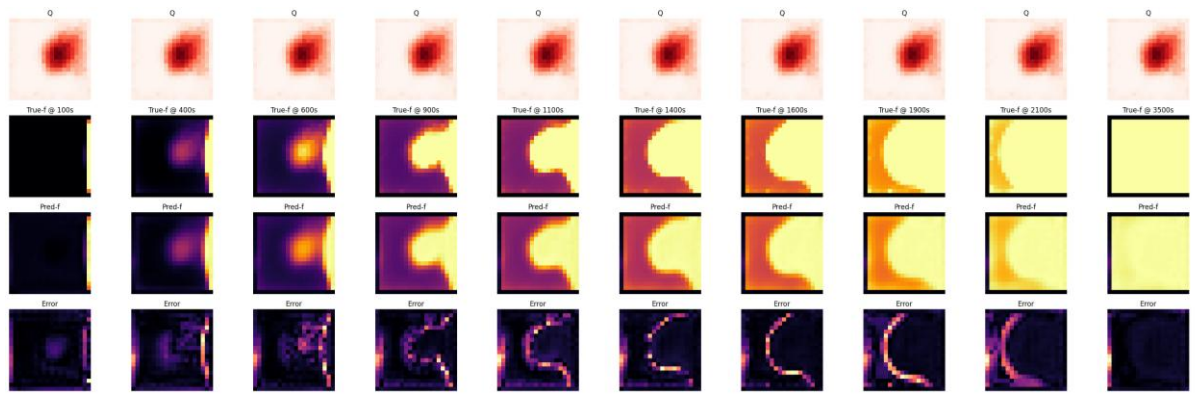


Fig. 38(f): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Y-Slice-6

Next comes the liquid fraction prediction plots for different z-slices as shown in figure 39 for the same boundary conditions mentioned in figure 37(a).

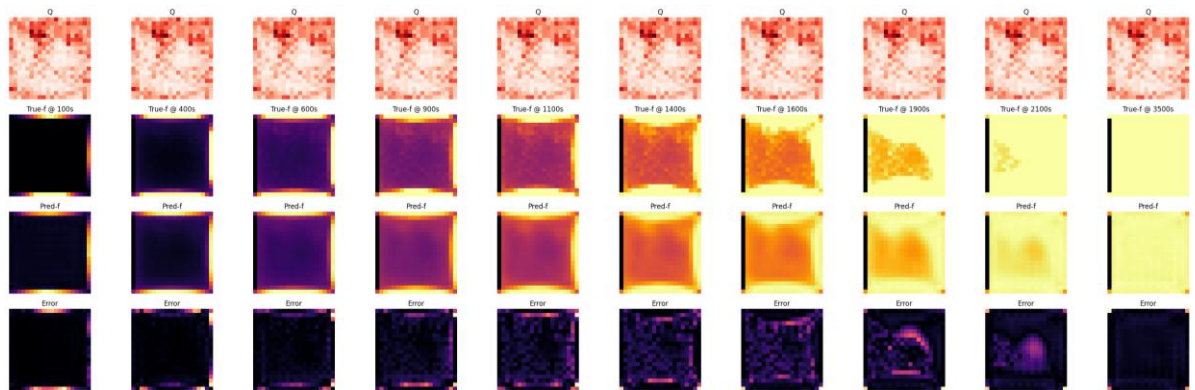


Fig. 39(a): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for (100, 400, 600, 900, 1100, 1400, 1600, 1900, 2100 and 3500s). Each plot shows the input heat-source function $Q(x, y)$ (row 1) and the corresponding true liquid fraction field $F(x, y)$ and predicted liquid fraction field $Pred_F(x, y)$ (row 2 & 3) and the corresponding absolute error maps (row 4) for Z-Slice-1

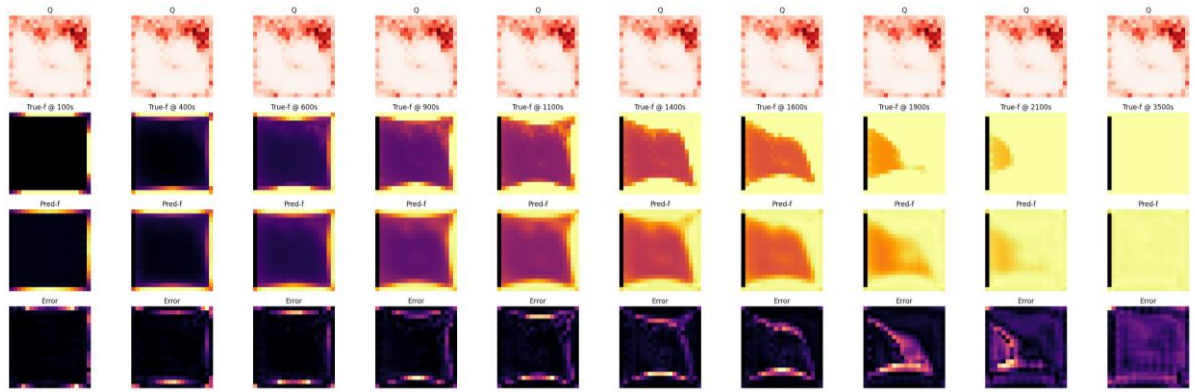


Fig. 39(b): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Z-Slice-2

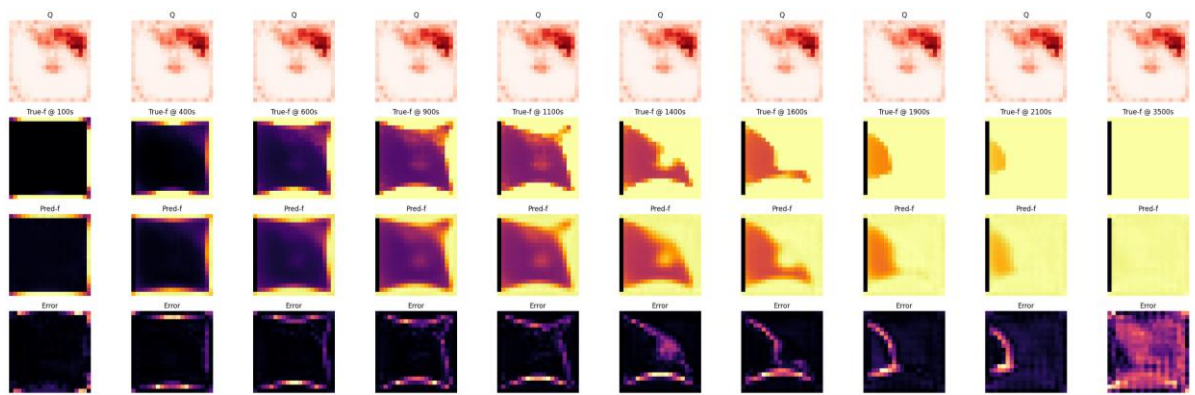


Fig. 39(c): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Z-Slice-3

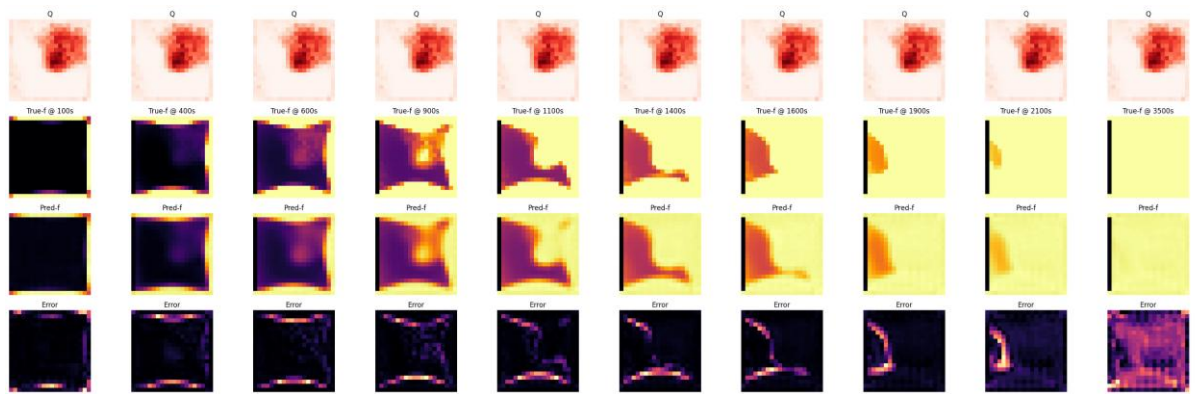


Fig. 39(d): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Z-Slice-4

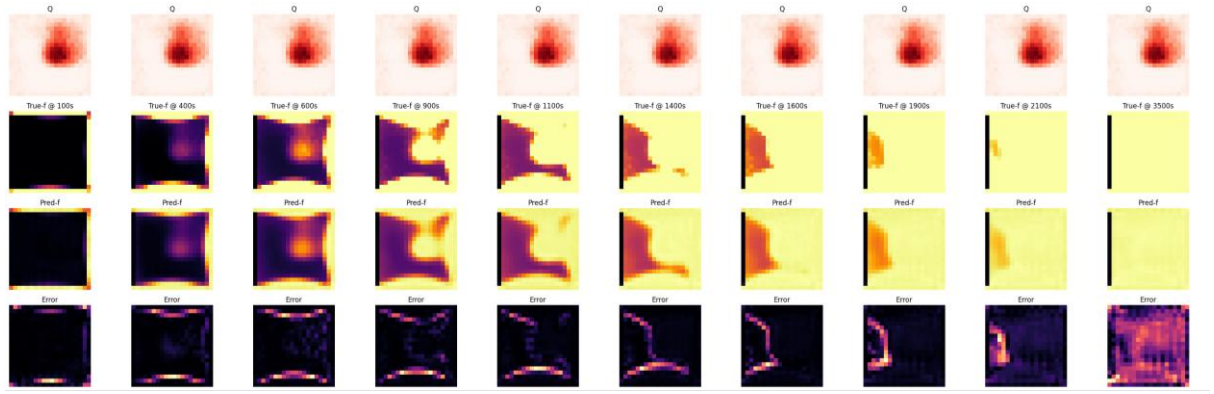


Fig. 39(e): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Z-Slice-5

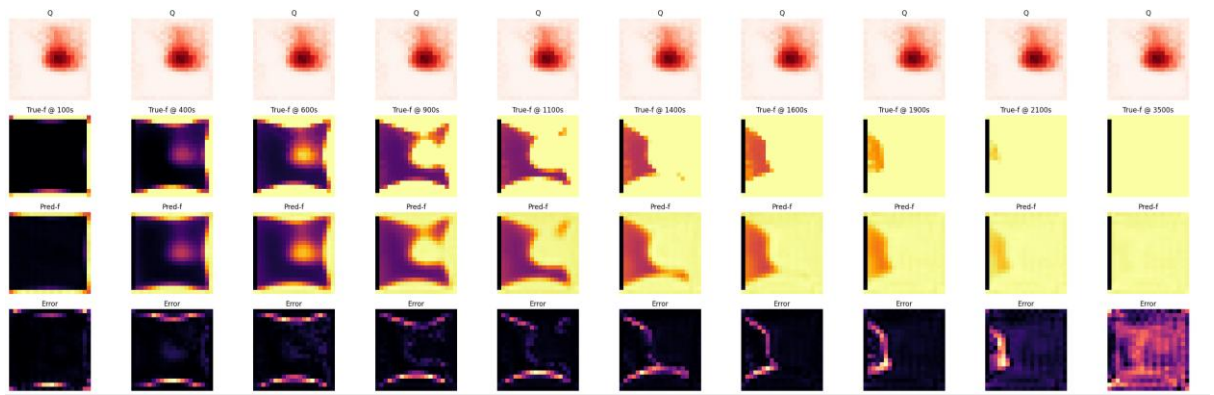


Fig. 39(f): Comparison of predicted and ground-truth 3D slice-based liquid fraction fields for Z-Slice-6

Overall, the slice-based 3D U-DeepONet gives quite accurate results for temperature and liquid-fraction prediction plots. The model predicts temperature slices that match the numerical solution at different times. It captures the spatial heat changes, hotspot regions and how heat spreads inside the material. The error is mostly localized near regions with sharper gradients but the overall temperature change is still accurate.

For liquid fraction prediction, the model also learns the main melting patterns and growth of liquid regions effectively. The predicted melting fronts follow the trend as the ground truth data. This shows that the model is well capable to learn how the phase change occurs over time from the 2D slice data. Some smoothing is seen near the solid-liquid interface, which is expected since the liquid fraction changes sharply during phase change.

The resolution is however a bit low as the grid elements are only 24 along each axis, which was taken due to memory constraints of the computer system.

The prediction plots show that the slicing approach can learn 3D material melting behaviour using 2D slice inputs. The model gets both the temperature field and phase-change evolution accurate. It avoids the cost of full 3D learning implementation as of now. This means slice-based 3D U-DeepONet is not a perfect yet efficient tool, for predicting phase change behaviour.

4.4.1 Three-Dimensional Prismatic Battery Heat-Source Configuration

The slice-based results presented above use the volumetric Gaussian heating of the earlier dataset. The extension developed in Sections 3.5.4 to 3.5.7 replaces that heating with an embedded prismatic battery cell, which is the configuration of direct interest for battery thermal management. The framework was trained on the 400 training cases of the three-dimensional dataset and evaluated on the held-out test split. The results below are for one representative test case, referred to throughout as a single case (Case-23/500 training cases), which contains a single prismatic battery source and two-dimensional Gaussian temperature profiles on several of the bounding faces. Forty-five stored planes are predicted for this case, fifteen normals to each axis, at twenty-four-time instants each.

The boundary conditions imposed on the six faces are shown in Fig. 40. The left, right and bottom faces are held at a uniform temperature, while the front, back and top faces carry two-dimensional Gaussian profiles with different peak positions and amplitudes. The resulting thermal environment is asymmetric, so the surrogate must respond to the boundary data as well as to the internal generation field.

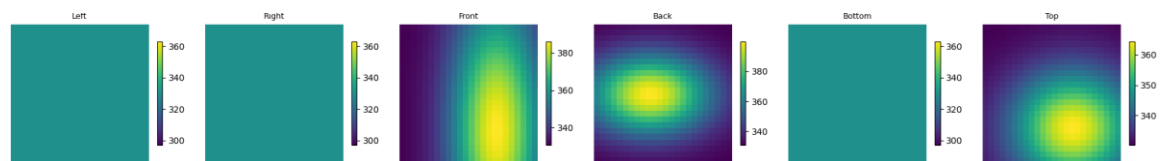


Fig. 40: Two-dimensional temperature fields imposed on the six bounding faces of the domain for the representative three-dimensional prismatic battery test case. The left, right and bottom faces are uniform; the front, back and top faces carry Gaussian profiles.

Of the fifteen stored planes on each axis, six intersect the battery on the x-axis (positions 4 to 9), nine on the y-axis (positions 4 to 12) and seven on the z-axis (positions 4 to 10). The six representative planes presented below for each axis were selected as contiguous sequences covering the battery: planes 4 to 9 for the x- and z-axes and planes 5 to 10 for

the y-axis. A contiguous sequence was preferred over a scattered selection so that the progression of the fields through the battery can be followed from one figure to the next.

4.4.2 Slice-Based Temperature Prediction for the Prismatic Battery

Figure 41 compares the reference and predicted temperature fields on the six selected planes normal to the x-axis. In each panel the first row gives the volumetric heat-source values on the plane with the battery outline superposed, the second and third rows the reference and predicted temperature fields, and the fourth the absolute error, at eight instants between 100 s and 3500 s.

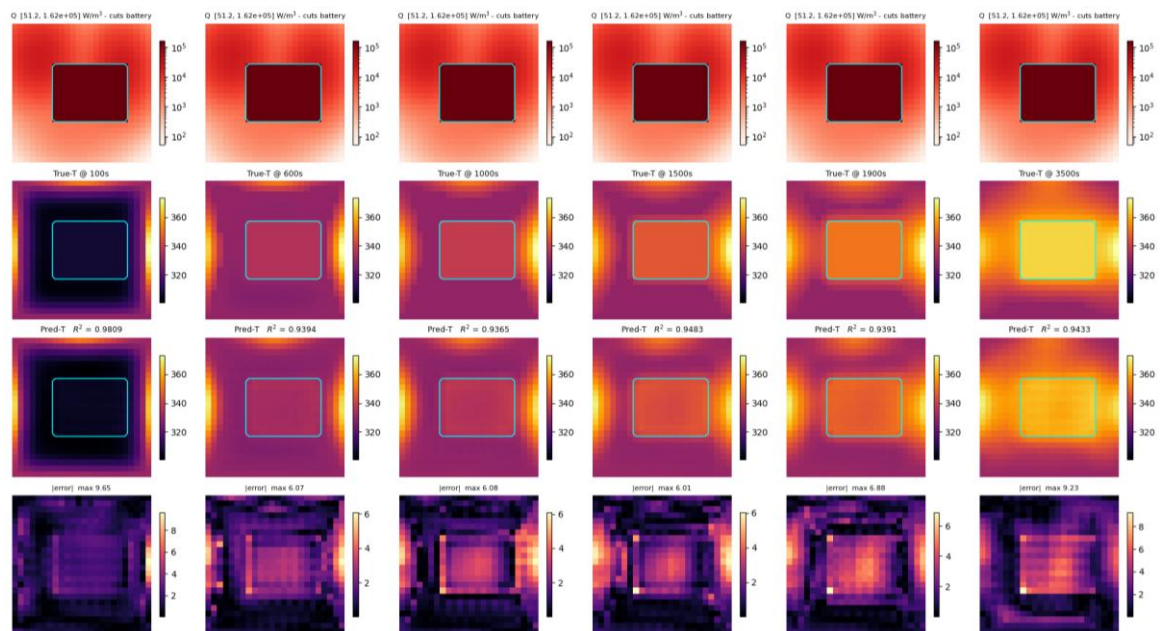


Fig. 41(a): Reference and predicted three-dimensional slice-based temperature fields for x-normal plane 4 (yz-plane), with $R^2 = 0.9672$ over the full space–time field of the plane.

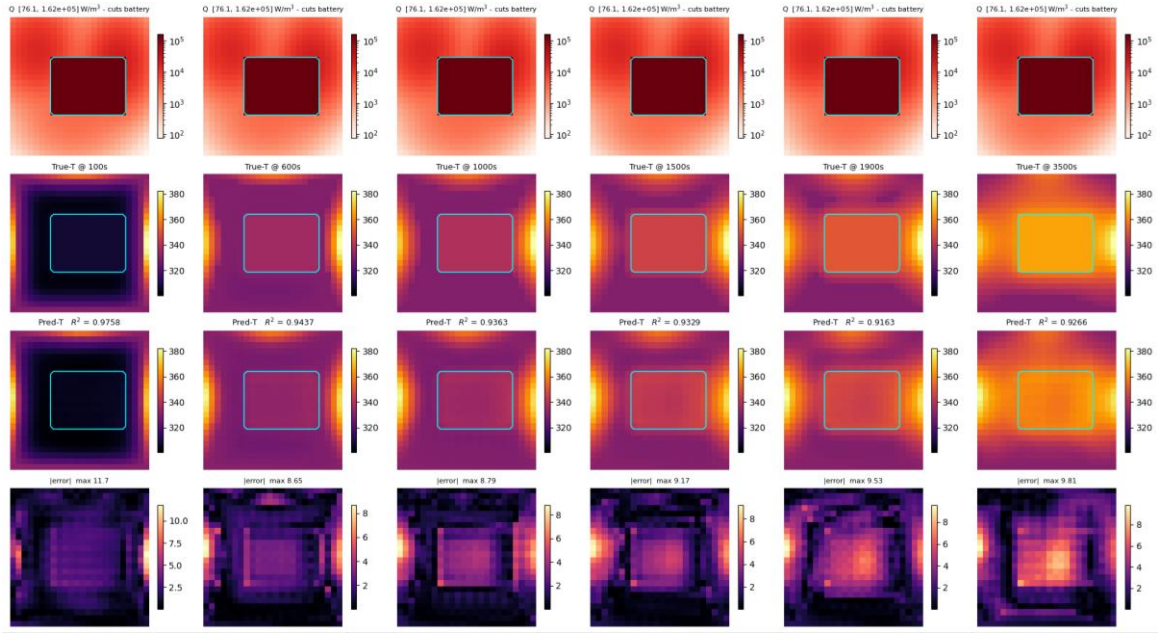


Fig. 41(b): Reference and predicted three-dimensional slice-based temperature fields for x-normal plane 5 (yz-plane), with $R^2 = 0.9586$ over the full space–time field of the plane.

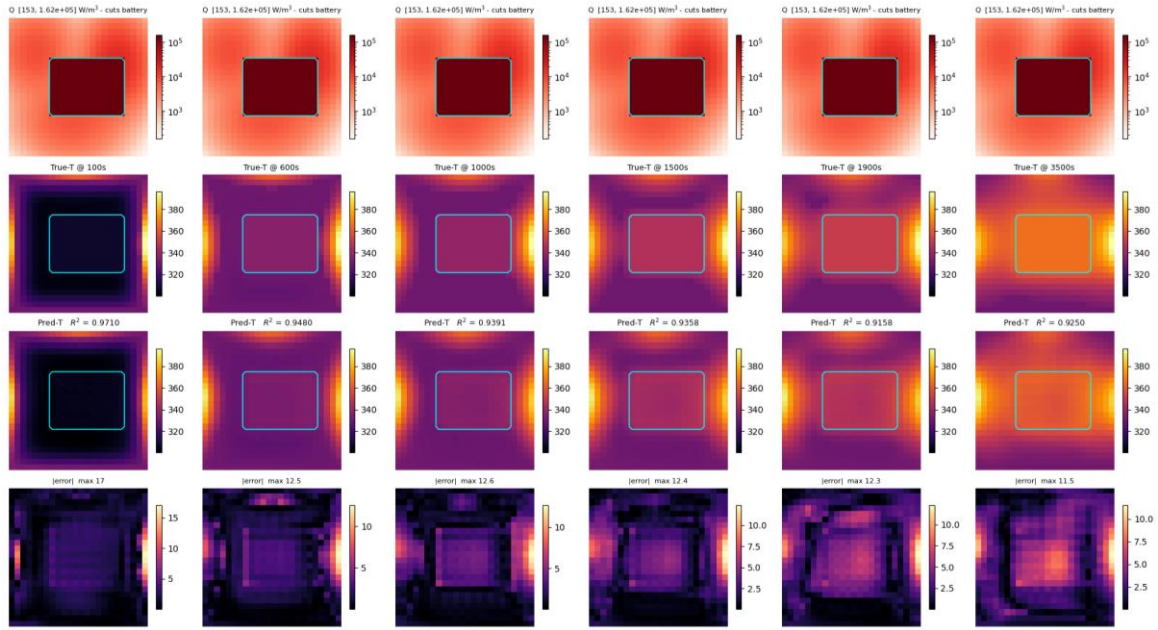


Fig. 41(c): Reference and predicted three-dimensional slice-based temperature fields for x-normal plane 6 (yz-plane), with $R^2 = 0.9562$ over the full space–time field of the plane.

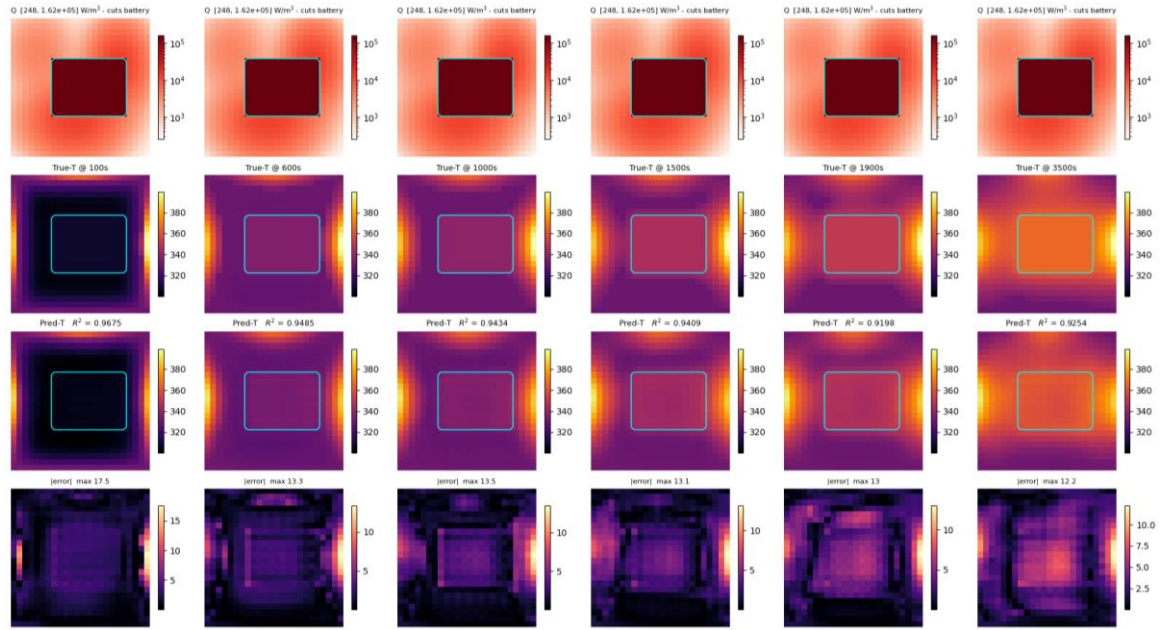


Fig. 41(d): Reference and predicted three-dimensional slice-based temperature fields for x-normal plane 7 (yz-plane), with $R^2 = 0.9573$ over the full space–time field of the plane.

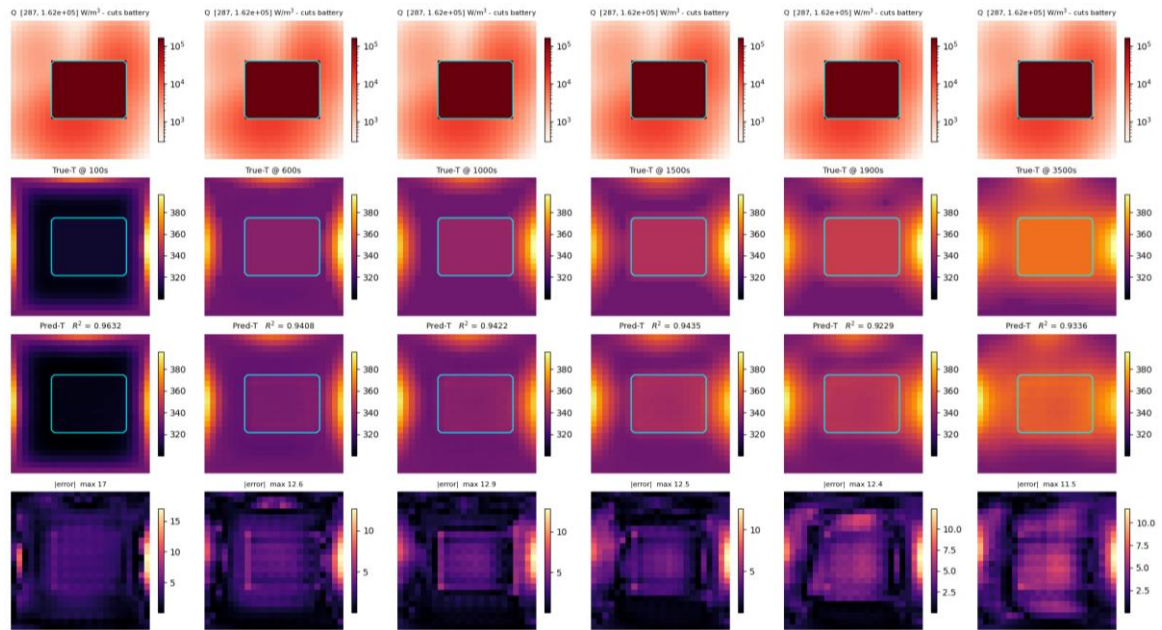


Fig. 41(e): Reference and predicted three-dimensional slice-based temperature fields for x-normal plane 8 (yz-plane), with $R^2 = 0.9574$ over the full space–time field of the plane.

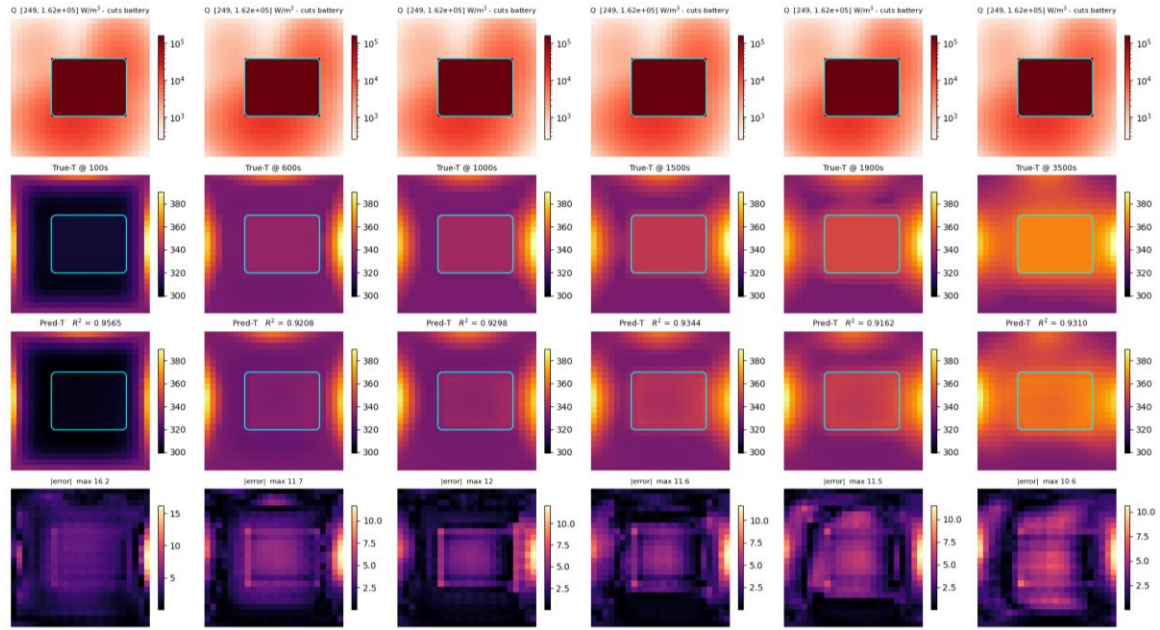


Fig. 41(f): Reference and predicted three-dimensional slice-based temperature fields for x-normal plane 9 (yz-plane), with $R^2 = 0.9517$ over the full space–time field of the plane.

Figure 42 compares the reference and predicted temperature fields on the six selected planes normal to the y-axis. In each panel the first row gives the volumetric heat-source values on the plane with the battery outline superposed, the second and third rows the reference and predicted temperature fields, and the fourth the absolute error, at eight instants between 100 s and 3500 s.

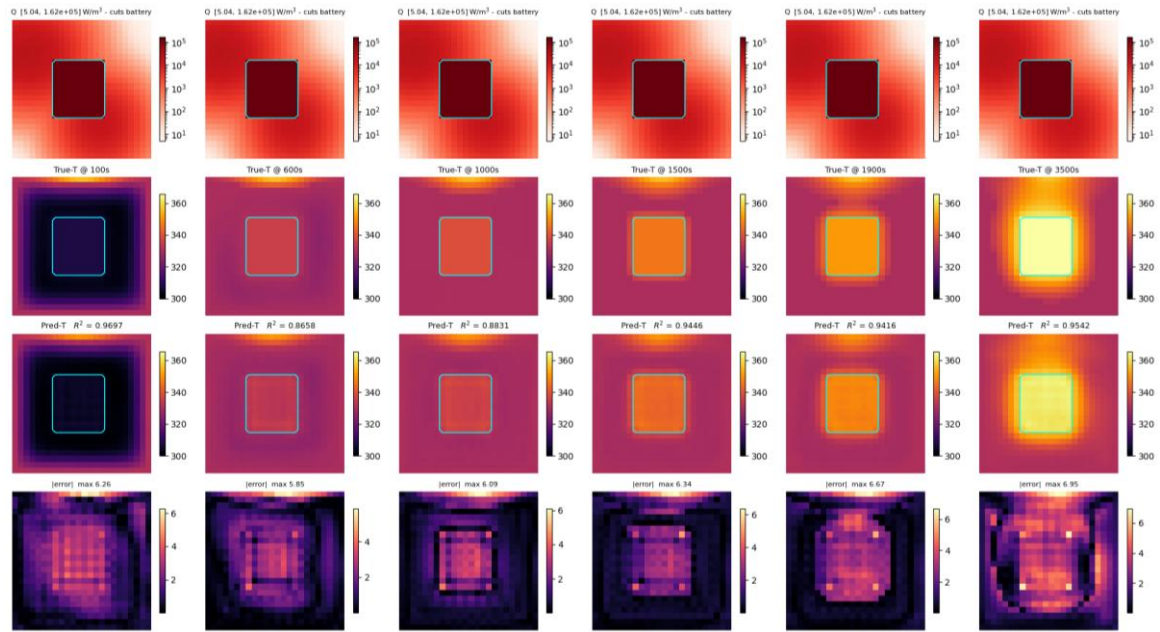


Fig. 42(a): Reference and predicted three-dimensional slice-based temperature fields for y-normal plane 5 (xz-plane), with $R^2 = 0.9651$ over the full space–time field of the plane.

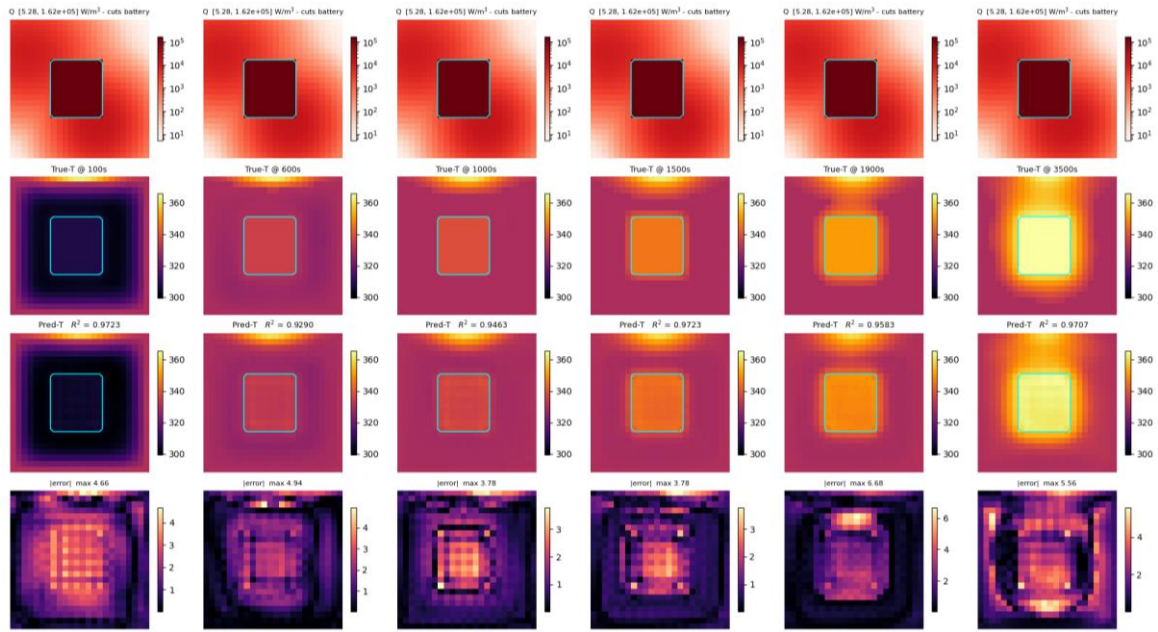


Fig. 42(b): Reference and predicted three-dimensional slice-based temperature fields for y-normal plane 6 (xz-plane), with $R^2 = 0.9772$ over the full space–time field of the plane.

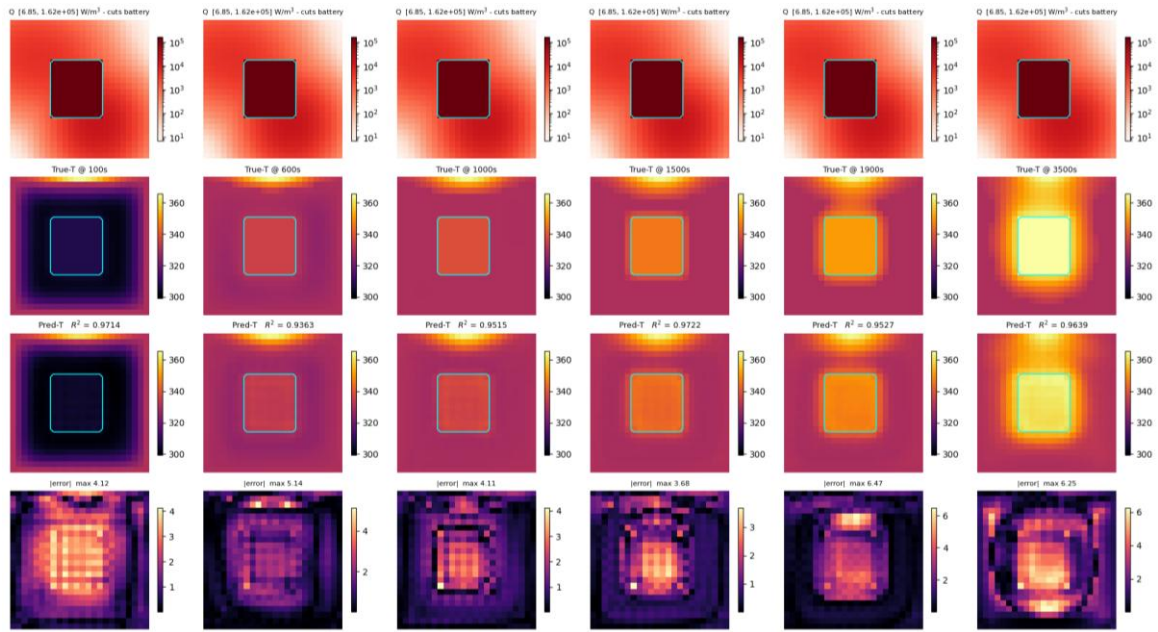


Fig. 42(c): Reference and predicted three-dimensional slice-based temperature fields for y-normal plane 7 (xz-plane), with $R^2 = 0.9758$ over the full space–time field of the plane.

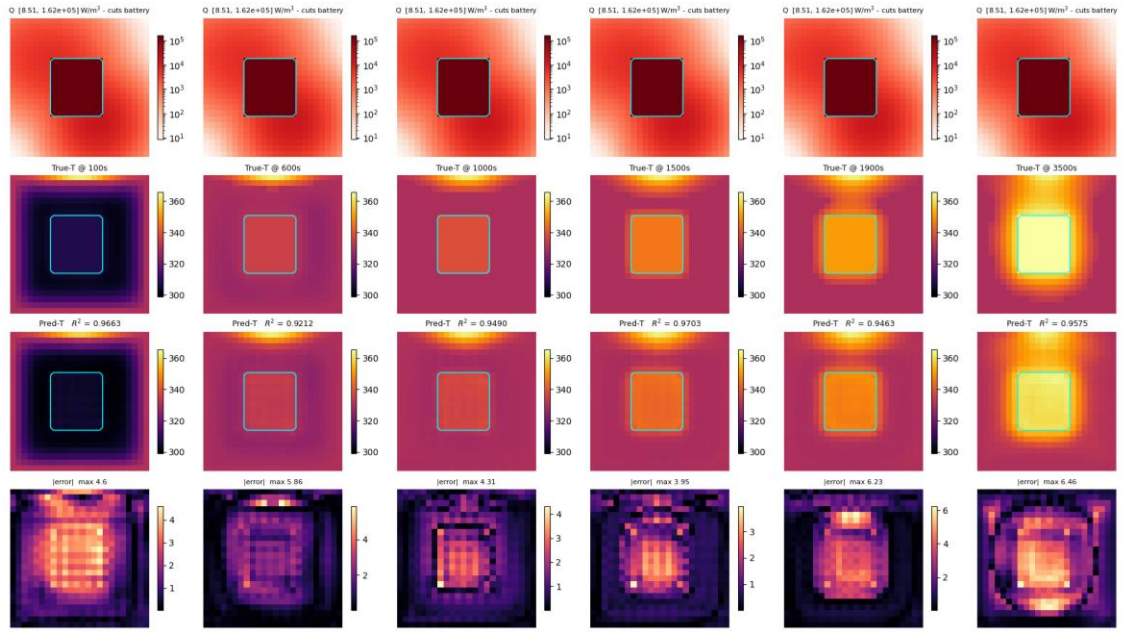


Fig. 42(d): Reference and predicted three-dimensional slice-based temperature fields for y-normal plane 8 (xz-plane), with $R^2 = 0.9731$ over the full space–time field of the plane.

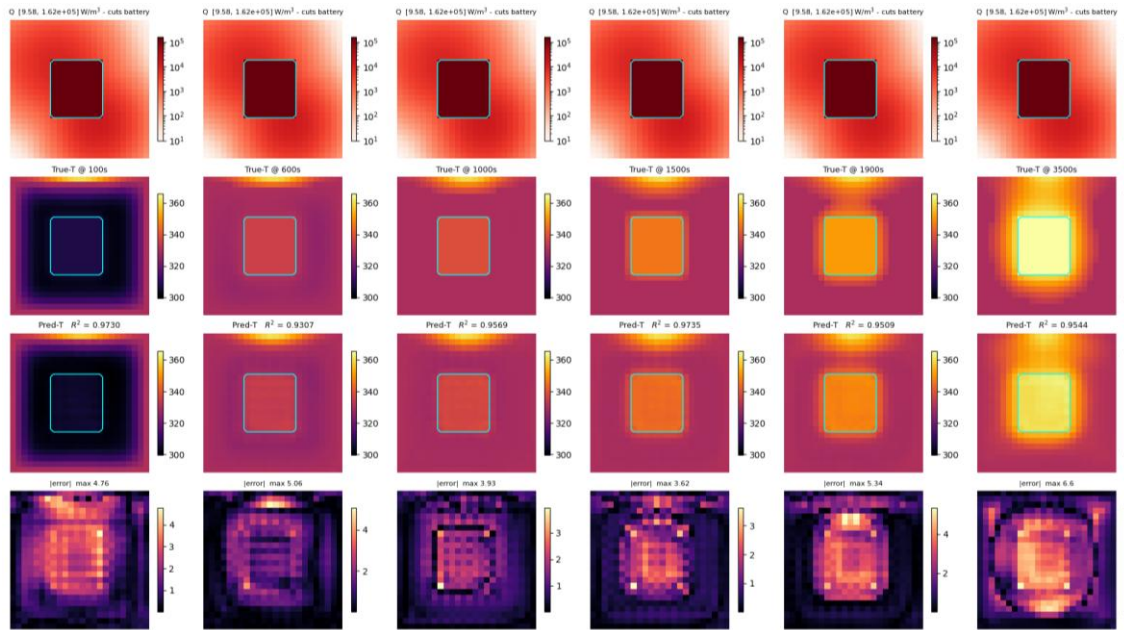


Fig. 42(e): Reference and predicted three-dimensional slice-based temperature fields for y-normal plane 9 (xz-plane), with $R^2 = 0.9752$ over the full space–time field of the plane.

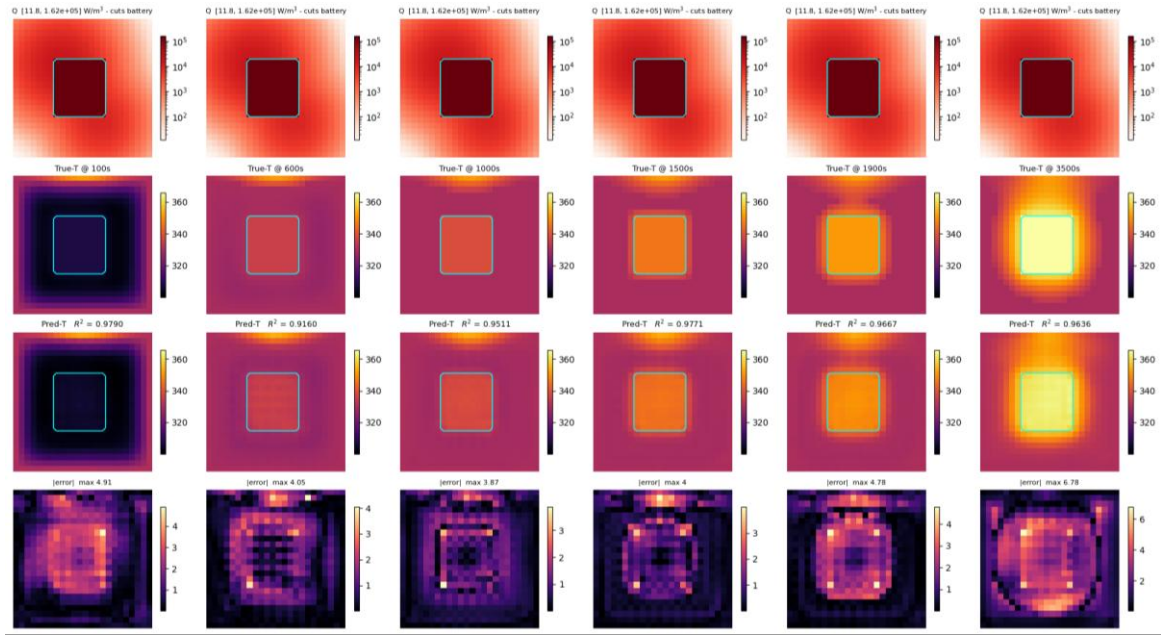


Fig. 42(f): Reference and predicted three-dimensional slice-based temperature fields for y-normal plane 10 (xz-plane), with $R^2 = 0.9792$ over the full space–time field of the plane.

Figure 43 compares the reference and predicted temperature fields on the six selected planes normal to the z-axis. In each panel the first row gives the volumetric heat-source values on the plane with the battery outline superposed, the second and third rows the reference and predicted temperature fields, and the fourth the absolute error, at eight instants between 100 s and 3500 s.

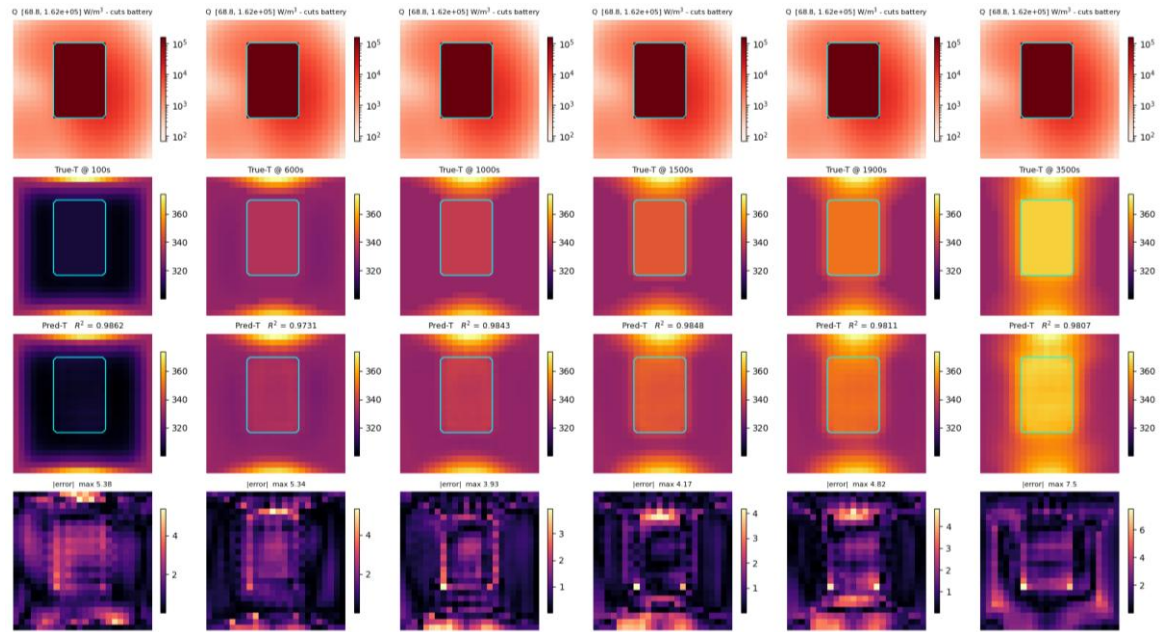


Fig. 43(a): Reference and predicted three-dimensional slice-based temperature fields for z-normal plane 4 (xy-plane), with $R^2 = 0.9879$ over the full space–time field of the plane.

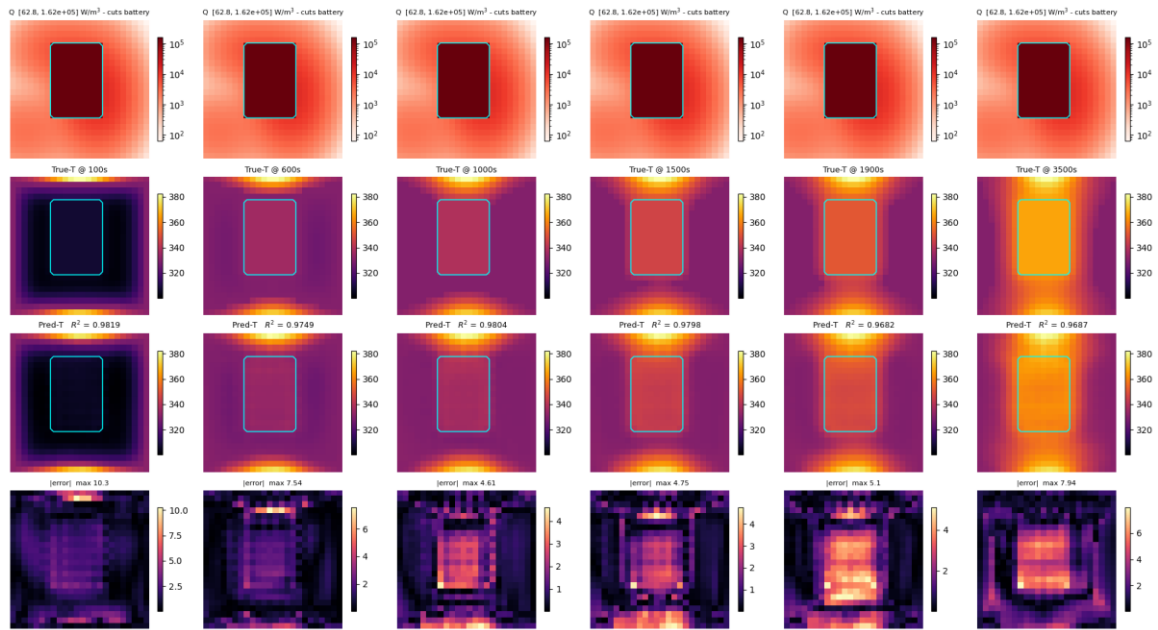


Fig. 43(b): Reference and predicted three-dimensional slice-based temperature fields for z-normal plane 5 (xy-plane), with $R^2 = 0.9824$ over the full space–time field of the plane.

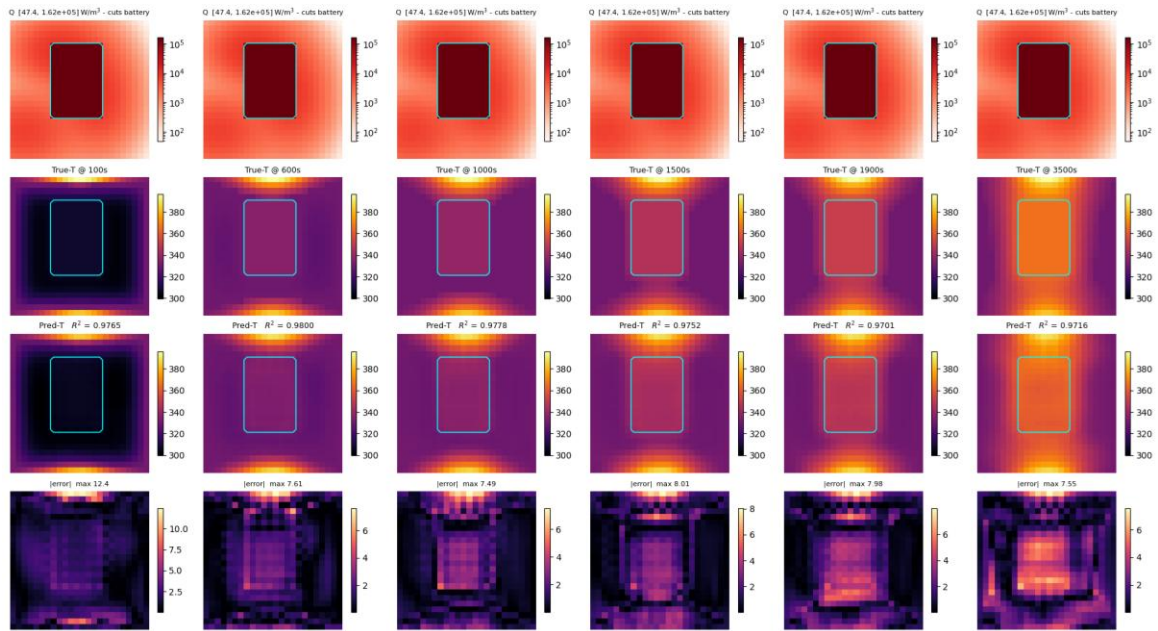


Fig. 43(c): Reference and predicted three-dimensional slice-based temperature fields for z-normal plane 6 (xy-plane), with $R^2 = 0.9810$ over the full space–time field of the plane.

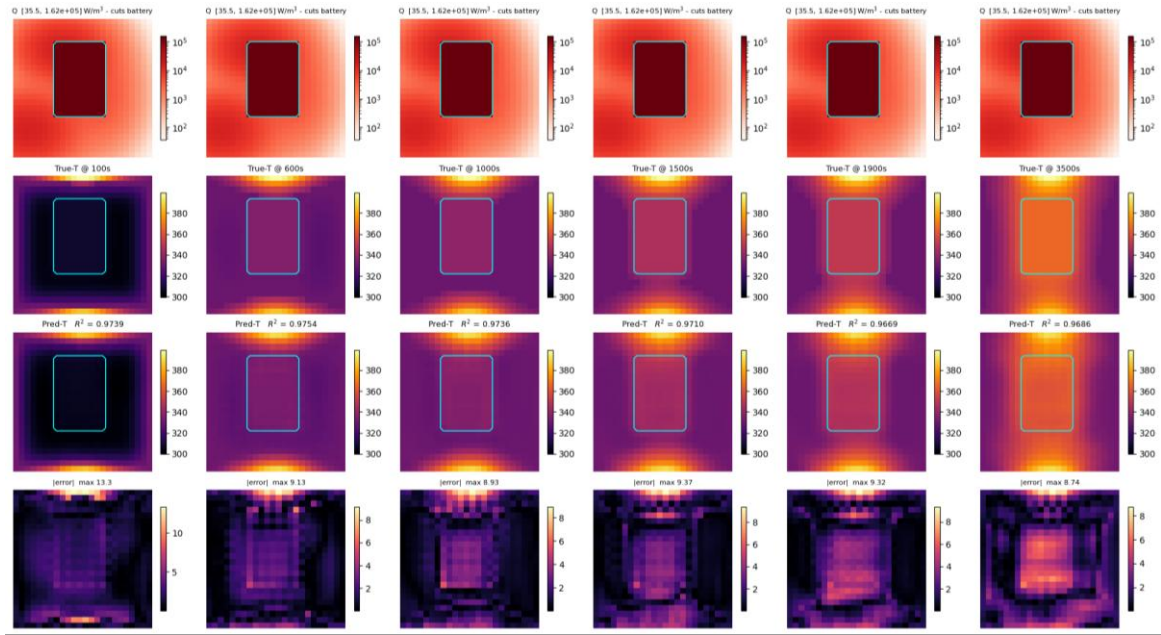


Fig. 43(d): Reference and predicted three-dimensional slice-based temperature fields for z-normal plane 7 (xy-plane), with $R^2 = 0.9781$ over the full space-time field of the plane.

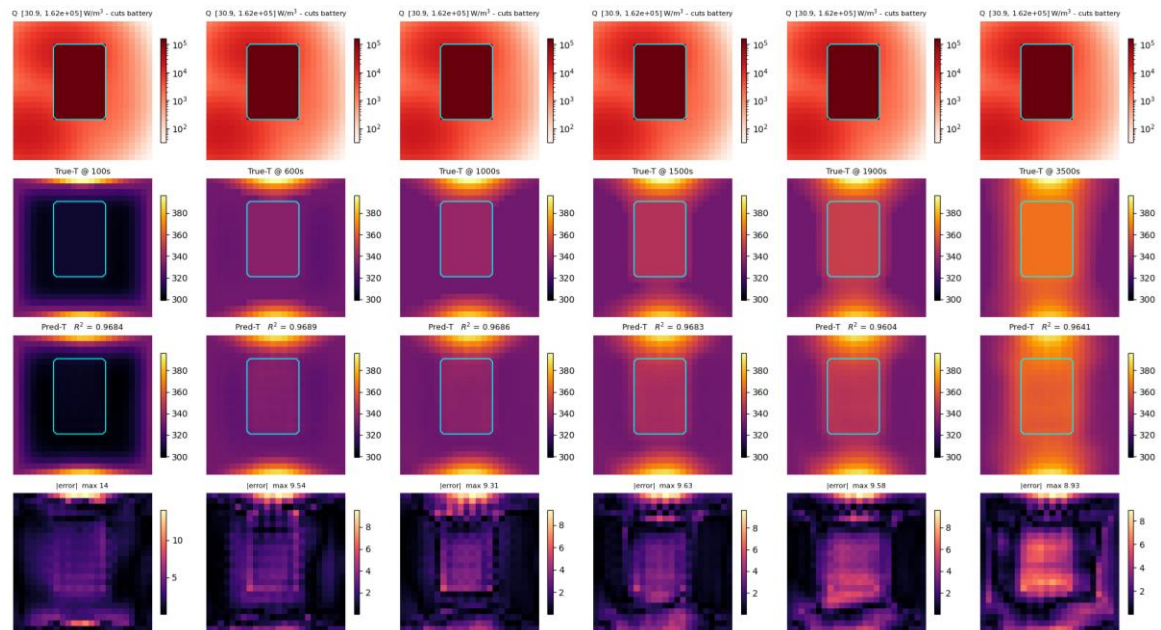


Fig. 43(e): Reference and predicted three-dimensional slice-based temperature fields for z-normal plane 8 (xy-plane), with $R^2 = 0.9749$ over the full space-time field of the plane.

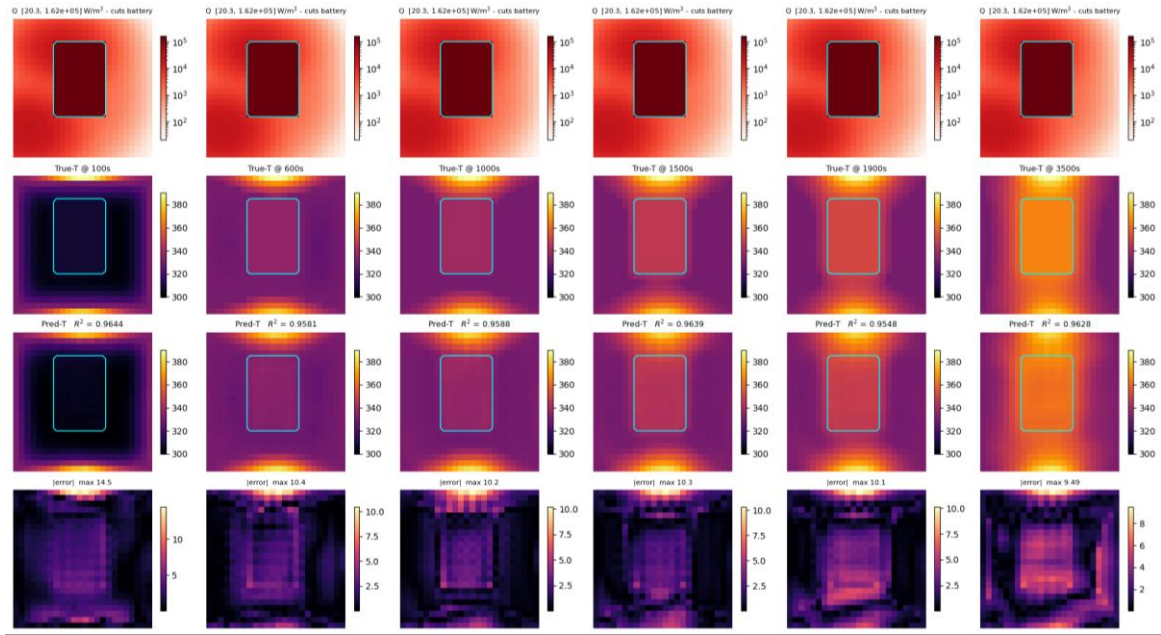


Fig. 43(f): Reference and predicted three-dimensional slice-based temperature fields for z-normal plane 9 (xy-plane), with $R^2 = 0.9712$ over the full space–time field of the plane.

The predictions follow the reference solution closely on every selected plane. At the earliest instant the material is still near its initial temperature except in thin layers at the heated faces, and the battery cross-section is visible as a cold region because the lumped source has not yet accumulated appreciable energy. The battery region warms progressively and, from about 800 s onward, is the warmest part of each plane that intersects it, with heat spreading outward into the surrounding PCM and interacting with the boundary layers at the heated faces. The surrogate reproduces both features and the transition between them.

The error maps show a consistent structure. The largest deviations trace the perimeter of the battery cross-section, where the imposed generation field is discontinuous, together with a band along the heated faces at early times. Interior errors away from these features remain small, and the maximum pointwise deviation on the planes shown is of the order of 6 to 17 K against a field spanning roughly 300 K to 390 K, without growing systematically with time.

Table 7: Slice-wise temperature prediction accuracy of the slice-based three-dimensional U-DeepONet for the prismatic battery test case. All planes listed intersect the battery.

Plane	x-normal R^2	x-normal RMSE (K)	y-normal R^2	y-normal RMSE (K)	z-normal R^2	z-normal RMSE (K)
1	0.9672	2.218	0.9651	1.854	0.9879	1.346
2	0.9586	2.724	0.9772	1.574	0.9824	1.772

Plane	x-normal R^2	x-normal RMSE (K)	y-normal R^2	y-normal RMSE (K)	z-normal R^2	z-normal RMSE (K)
3	0.9562	3.190	0.9758	1.640	0.9810	2.103
4	0.9573	3.235	0.9731	1.715	0.9781	2.323
5	0.9574	3.139	0.9752	1.616	0.9749	2.420
6	0.9517	3.157	0.9792	1.409	0.9712	2.454

Temperature R^2 lies between 0.9517 and 0.9672 on the x-normal planes, between 0.9651 and 0.9792 on the y-normal planes and between 0.9712 and 0.9879 on the z-normal planes, with root-mean-square errors of 1.3 K to 3.2 K. The z-normal planes are the most accurate and the x-normal planes the least, which follows from the boundary configuration: the two faces normal to x are the uniform ones, so the x-normal planes carry the least in-plane structure and the strongest out-of-plane gradients, and it is precisely the out-of-plane transport that the slice-window fusion must infer rather than observe directly.

4.4.3 Slice-Based Liquid-Fraction Prediction for the Prismatic Battery

Figure 44 presents the corresponding liquid-fraction comparison for the planes normal to the x-axis, with rows arranged as before.

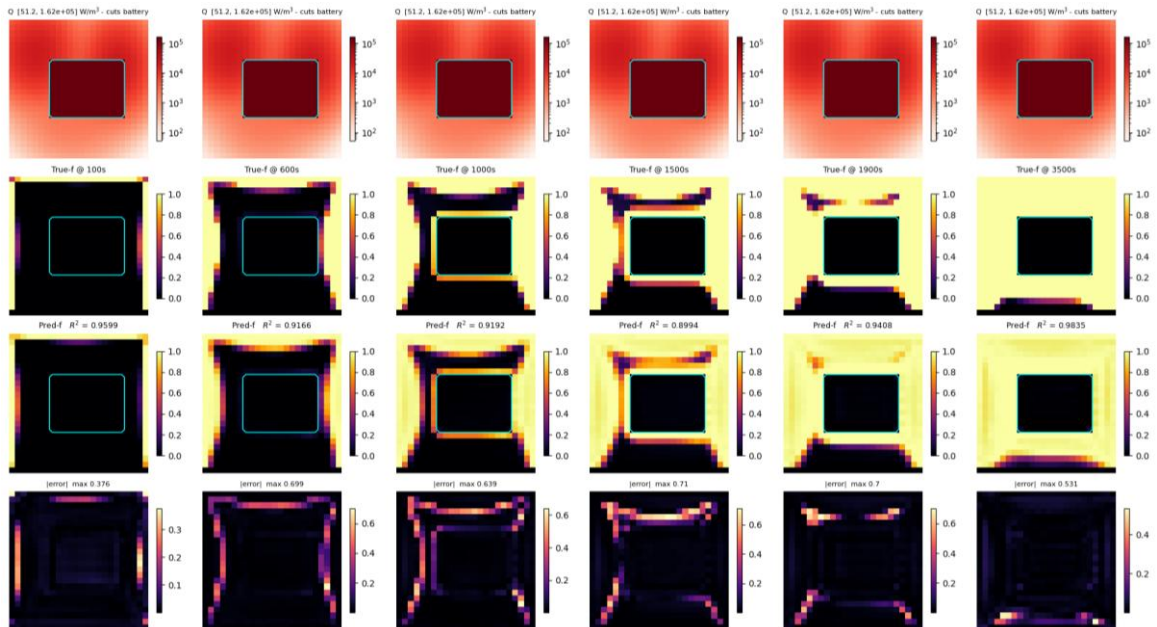


Fig. 44(a): Reference and predicted three-dimensional slice-based liquid-fraction fields for x-normal plane 4 (yz-plane), with $R^2 = 0.9384$ over the full space–time field of the plane.

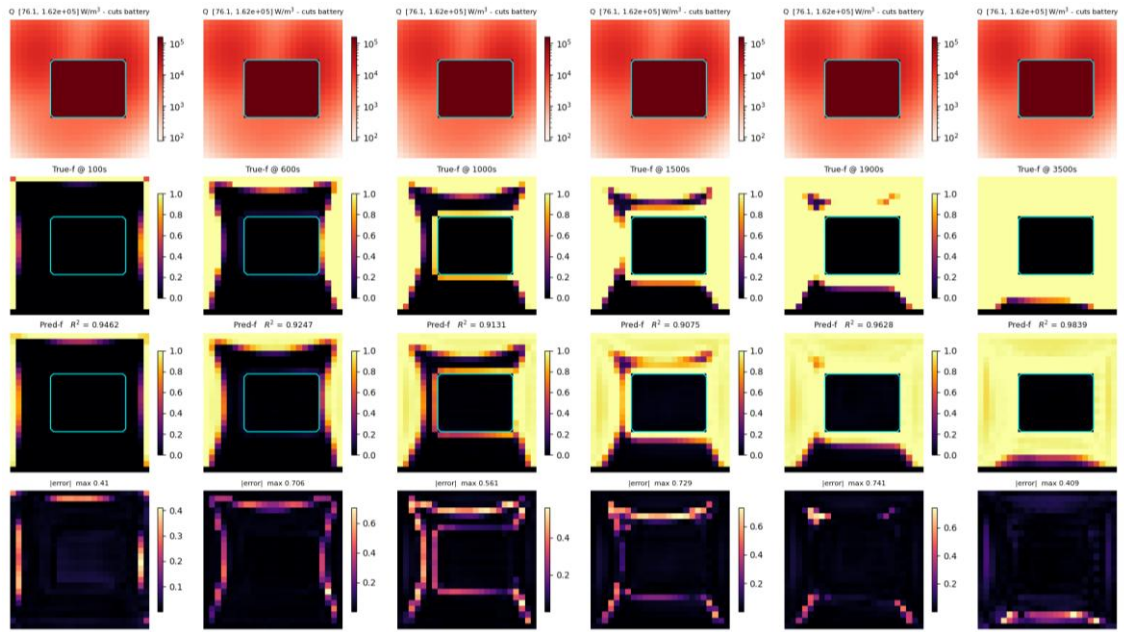


Fig. 44(b): Reference and predicted three-dimensional slice-based liquid-fraction fields for x-normal plane 5 (yz-plane), with $R^2 = 0.9444$ over the full space–time field of the plane.

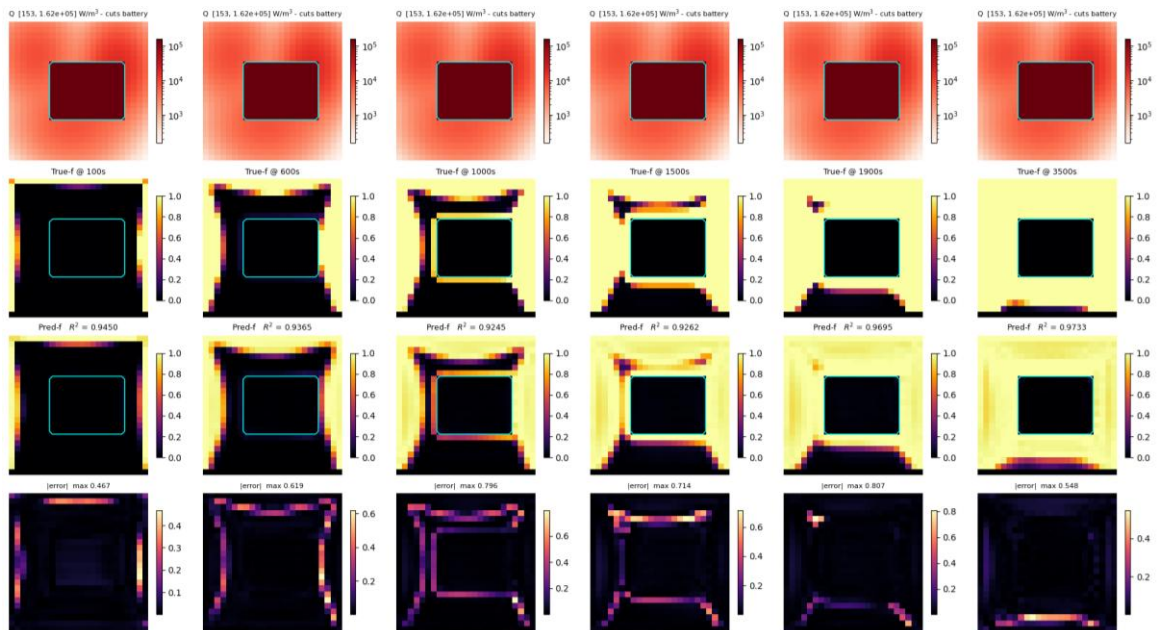


Fig. 44(c): Reference and predicted three-dimensional slice-based liquid-fraction fields for x-normal plane 6 (yz-plane), with $R^2 = 0.9528$ over the full space–time field of the plane.

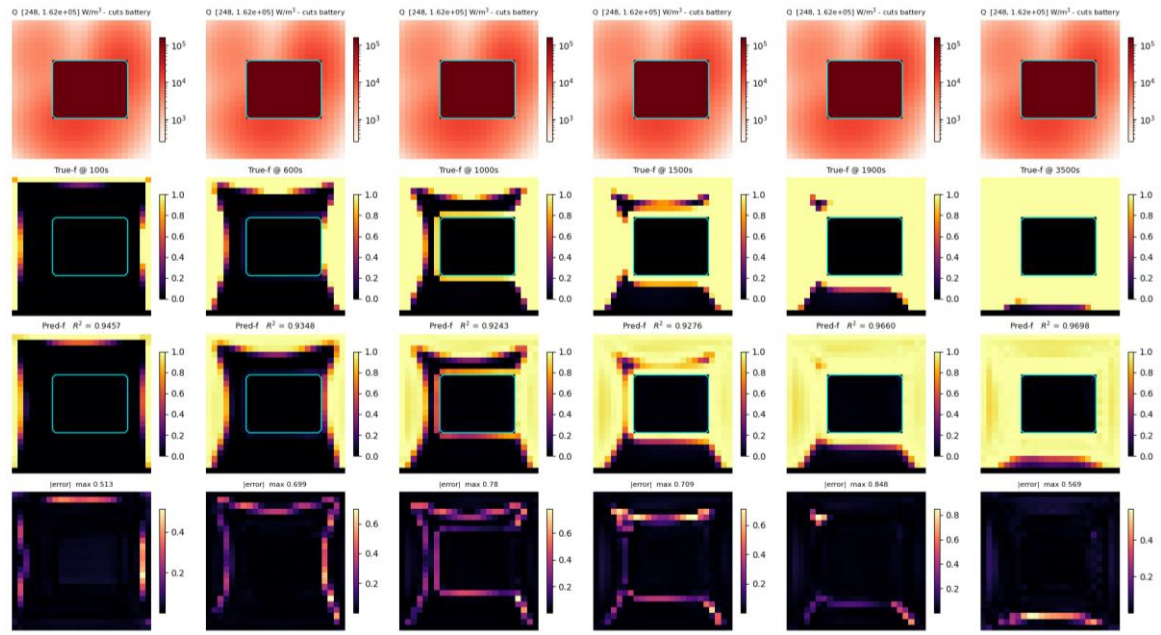


Fig. 44(d): Reference and predicted three-dimensional slice-based liquid-fraction fields for x-normal plane 7 (yz-plane), with $R^2 = 0.9519$ over the full space–time field of the plane.

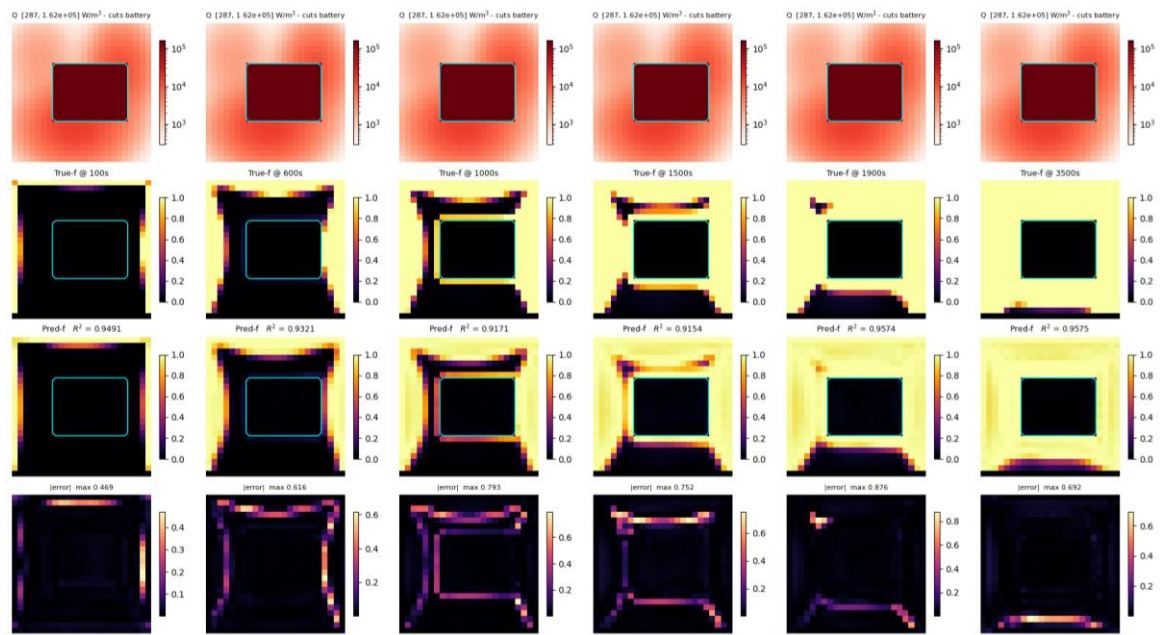


Fig. 44(e): Reference and predicted three-dimensional slice-based liquid-fraction fields for x-normal plane 8 (yz-plane), with $R^2 = 0.9459$ over the full space–time field of the plane.

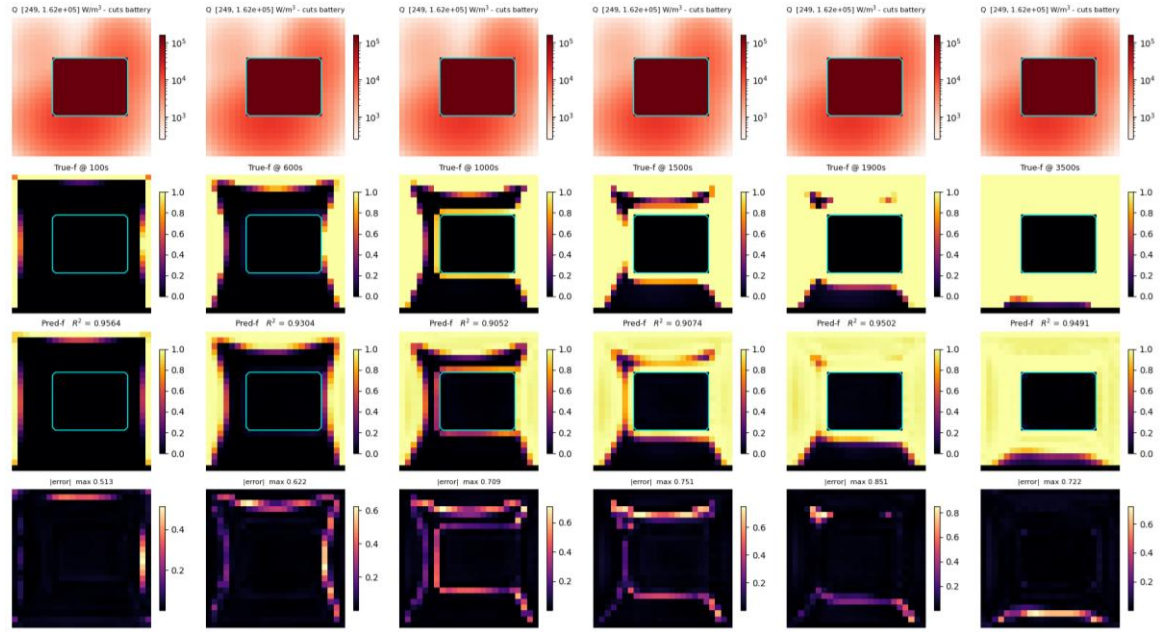


Fig. 44(f): Reference and predicted three-dimensional slice-based liquid-fraction fields for x-normal plane 9 (yz-plane), with $R^2 = 0.9407$ over the full space–time field of the plane.

Figure 45 presents the corresponding liquid-fraction comparison for the planes normal to the y-axis, with rows arranged as before.

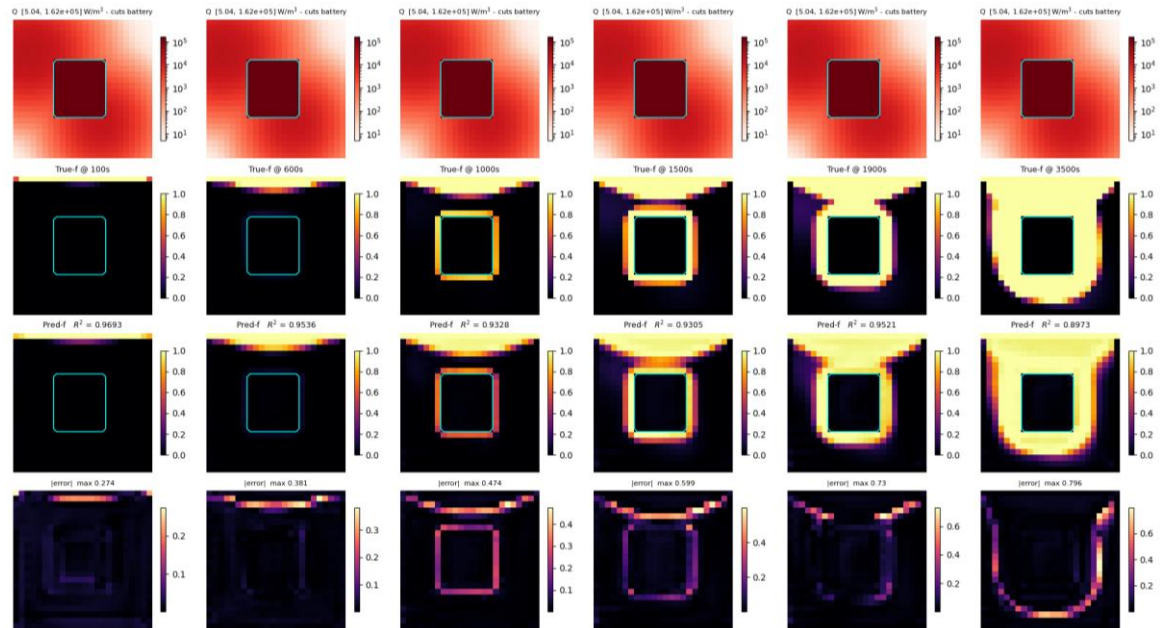


Fig. 45(a): Reference and predicted three-dimensional slice-based liquid-fraction fields for y-normal plane 5 (xz-plane), with $R^2 = 0.9469$ over the full space–time field of the plane.

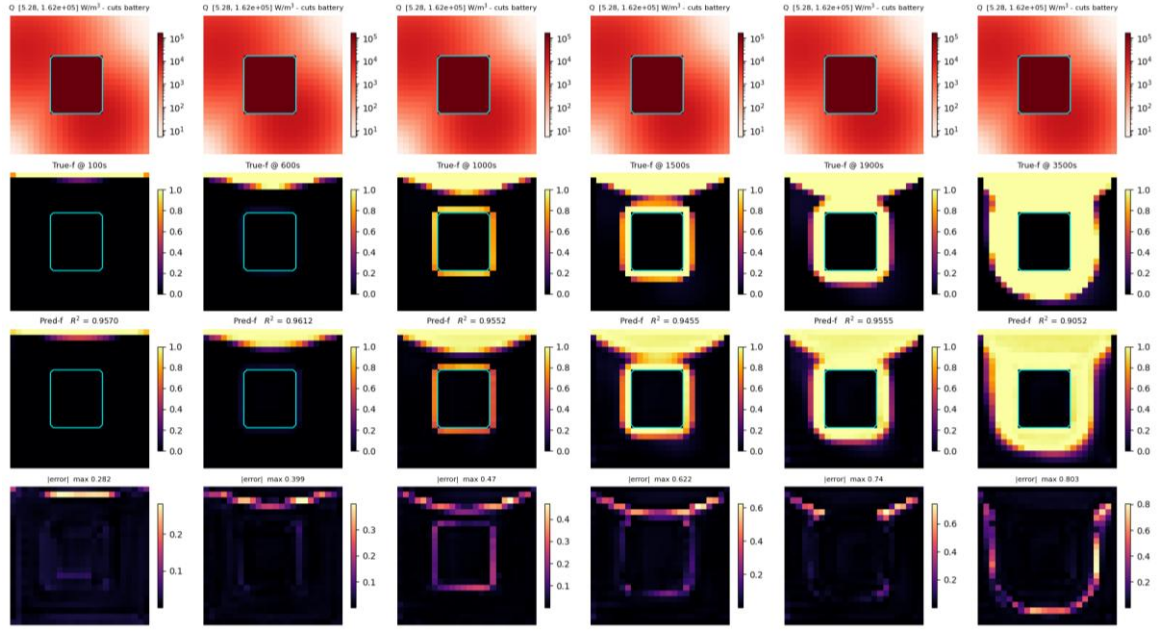


Fig. 45(b): Reference and predicted three-dimensional slice-based liquid-fraction fields for y-normal plane 6 (xz-plane), with $R^2 = 0.9552$ over the full space–time field of the plane.

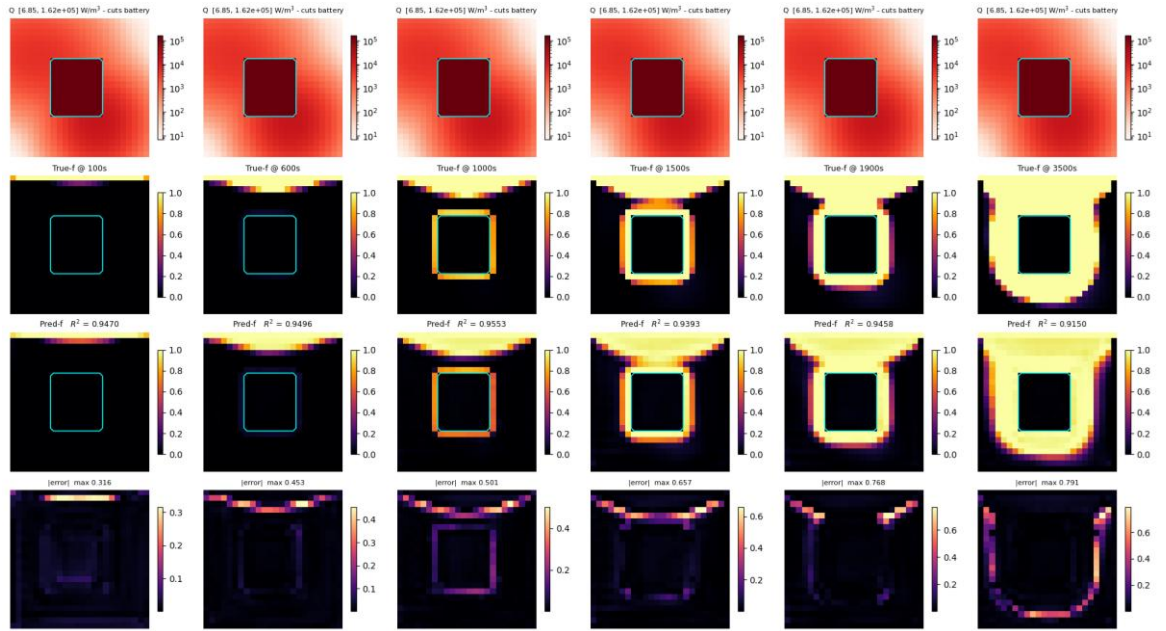


Fig. 45(c): Reference and predicted three-dimensional slice-based liquid-fraction fields for y-normal plane 7 (xz-plane), with $R^2 = 0.9500$ over the full space–time field of the plane.

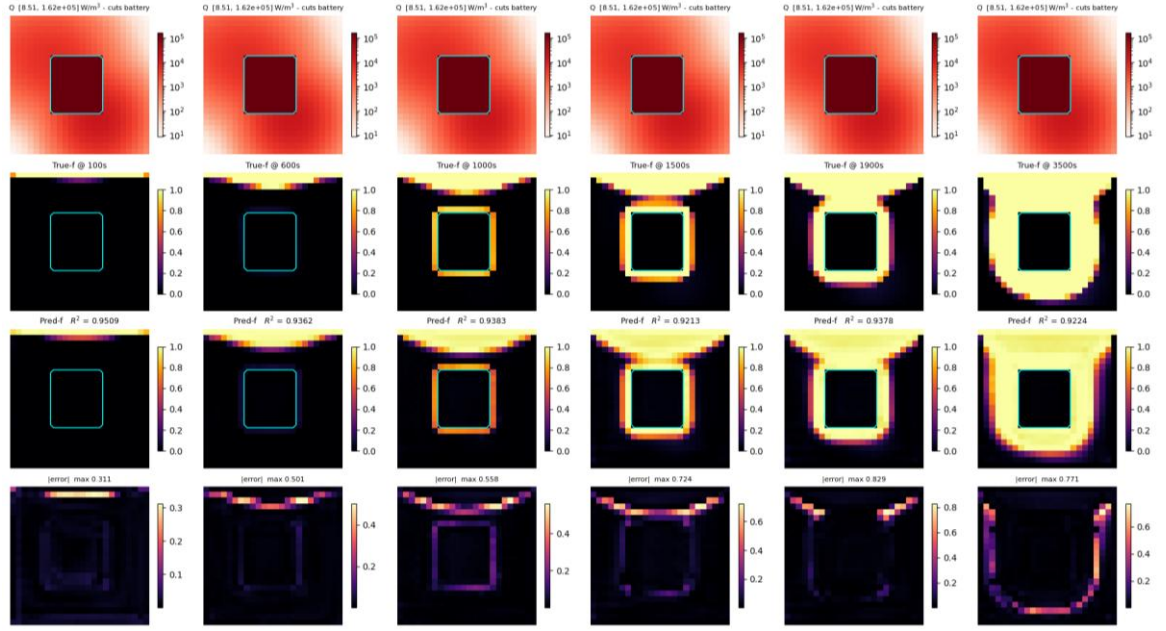


Fig. 45(d): Reference and predicted three-dimensional slice-based liquid-fraction fields for y-normal plane 8 (xz-plane), with $R^2 = 0.9407$ over the full space–time field of the plane.

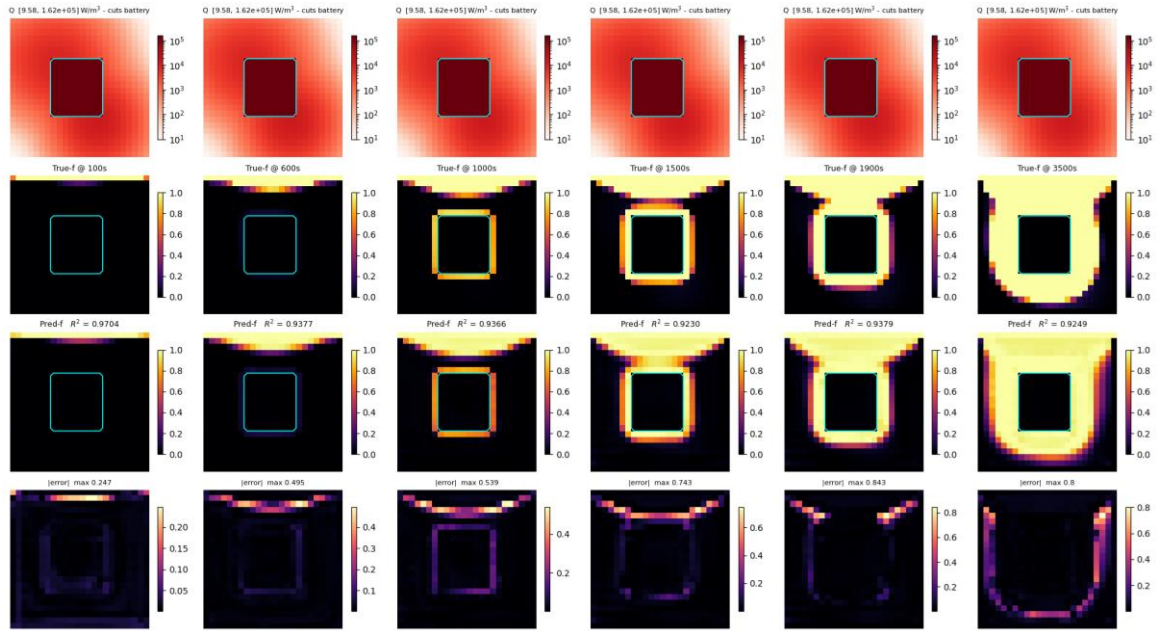


Fig. 45(e): Reference and predicted three-dimensional slice-based liquid-fraction fields for y-normal plane 9 (xz-plane), with $R^2 = 0.9415$ over the full space–time field of the plane.

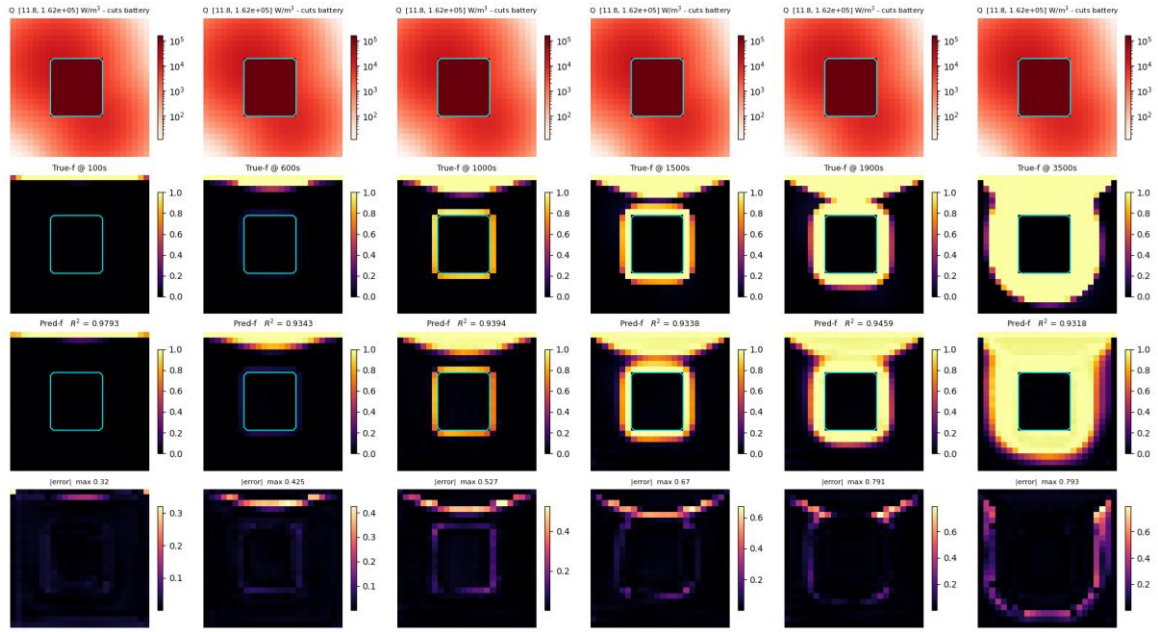


Fig. 45(f): Reference and predicted three-dimensional slice-based liquid-fraction fields for y-normal plane 10 (xz-plane), with $R^2 = 0.9490$ over the full space–time field of the plane.

Figure 46 presents the corresponding liquid-fraction comparison for the planes normal to the z-axis, with rows arranged as before.

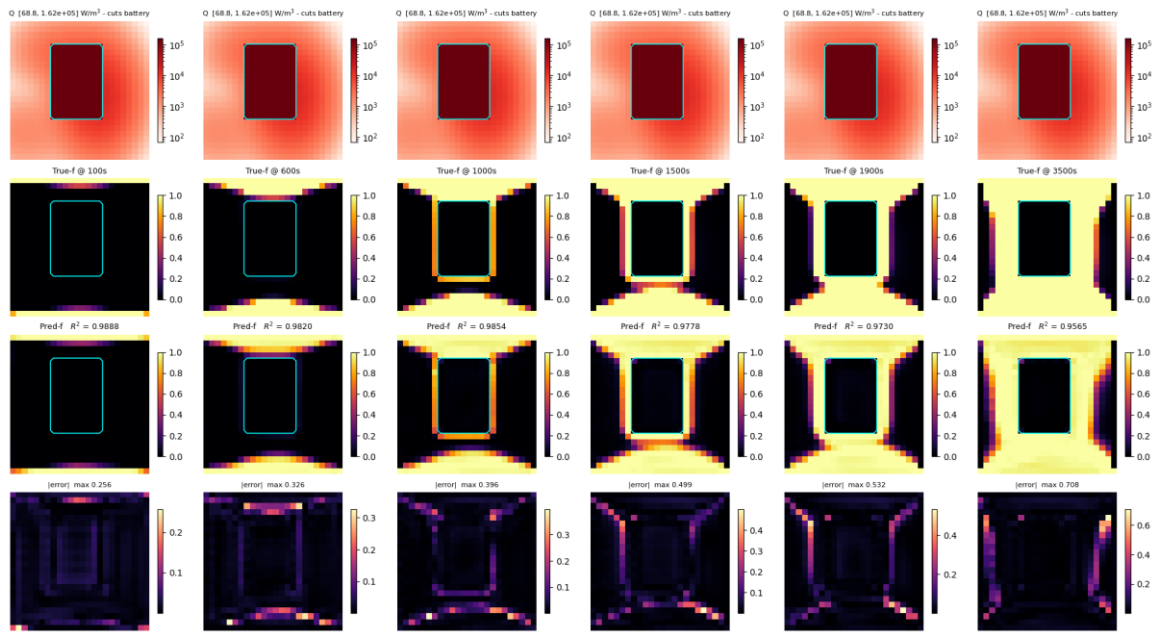


Fig. 46(a): Reference and predicted three-dimensional slice-based liquid-fraction fields for z-normal plane 4 (xy-plane), with $R^2 = 0.9779$ over the full space–time field of the plane.

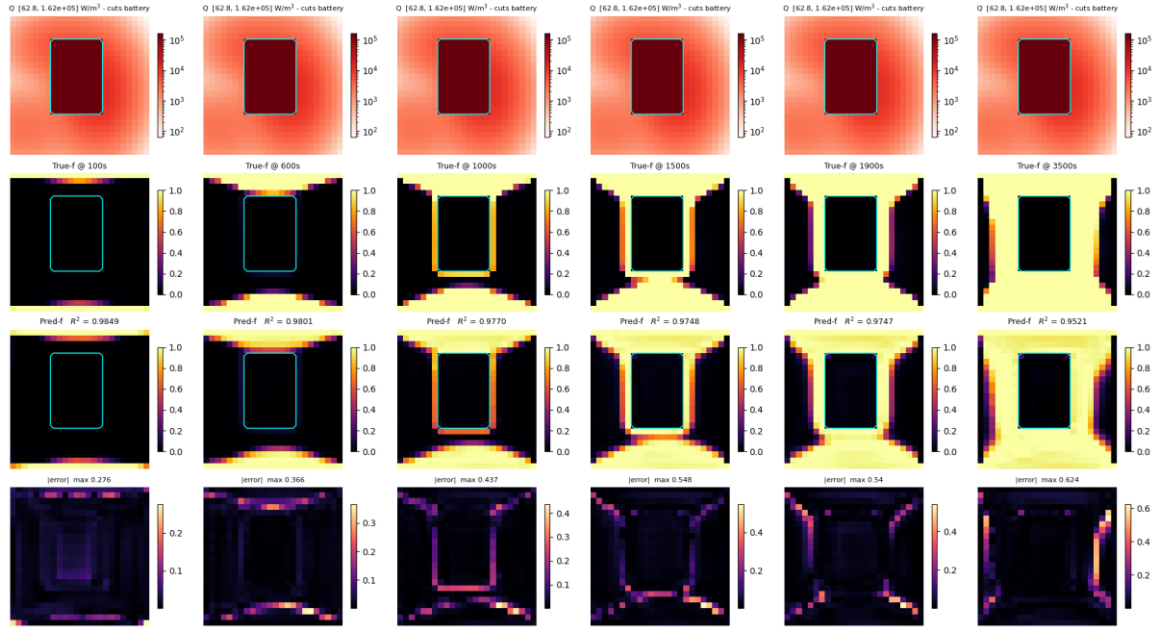


Fig. 46(b): Reference and predicted three-dimensional slice-based liquid-fraction fields for z-normal plane 5 (xy-plane), with $R^2 = 0.9775$ over the full space–time field of the plane.

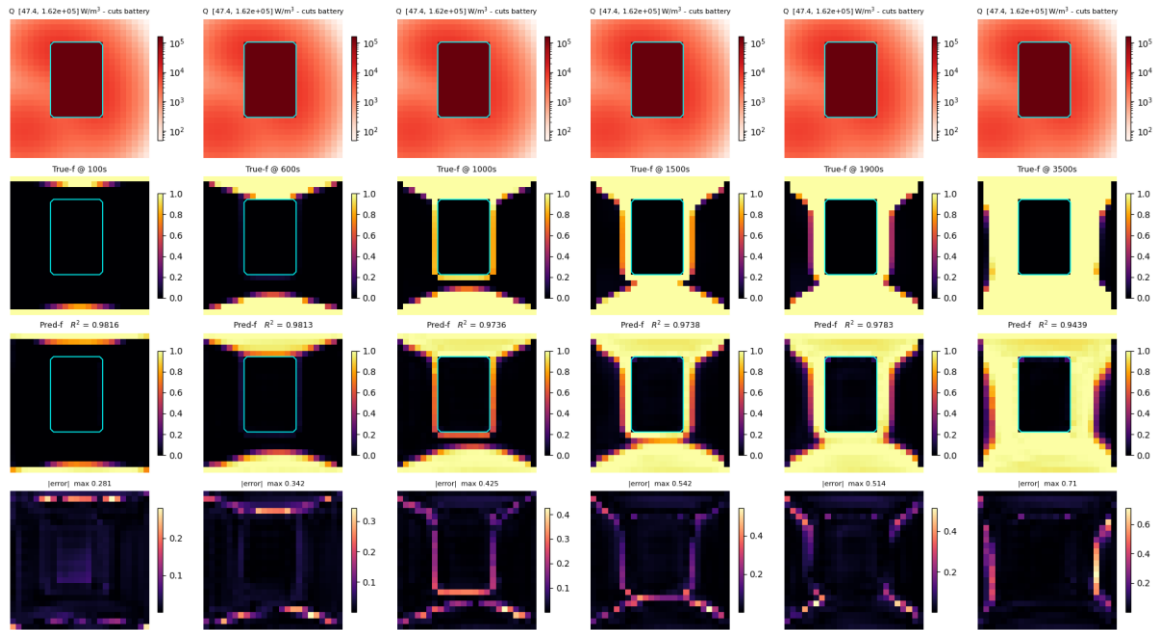


Fig. 46(c): Reference and predicted three-dimensional slice-based liquid-fraction fields for z-normal plane 6 (xy-plane), with $R^2 = 0.9778$ over the full space–time field of the plane.

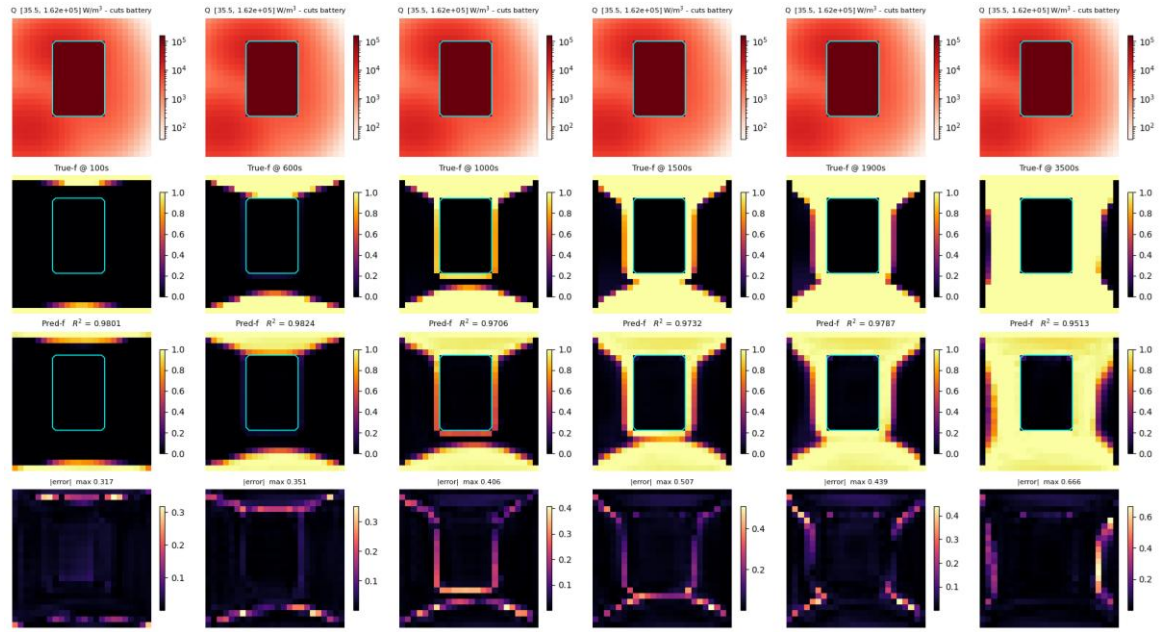


Fig. 46(d): Reference and predicted three-dimensional slice-based liquid-fraction fields for z-normal plane 7 (xy-plane), with $R^2 = 0.9776$ over the full space–time field of the plane.

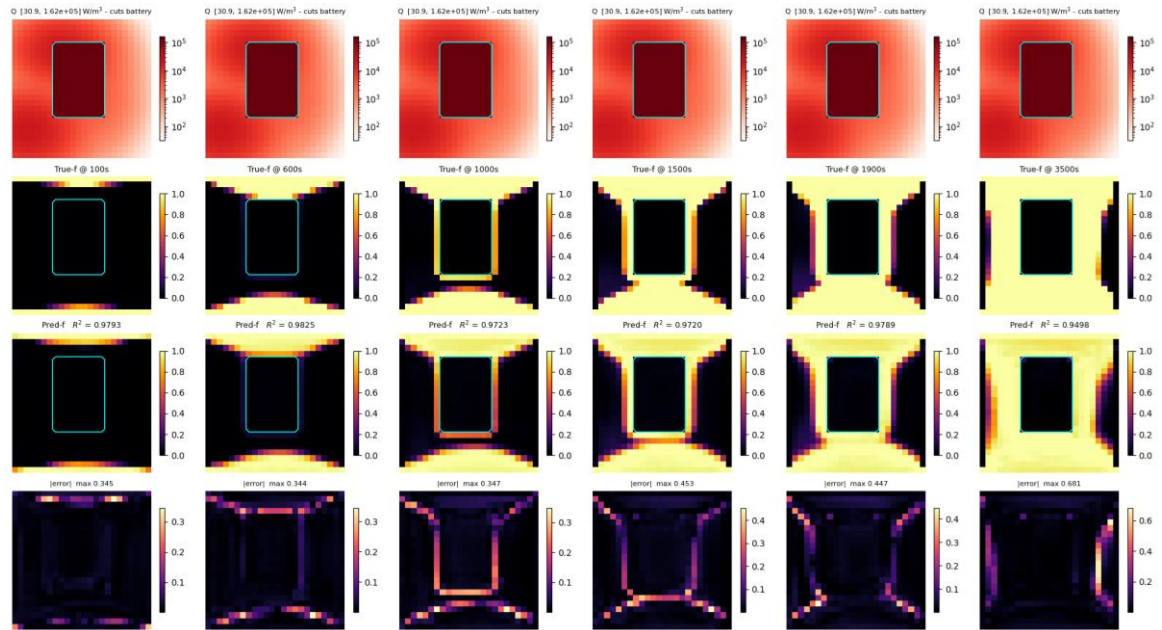


Fig. 46(e): Reference and predicted three-dimensional slice-based liquid-fraction fields for z-normal plane 8 (xy-plane), with $R^2 = 0.9772$ over the full space–time field of the plane.

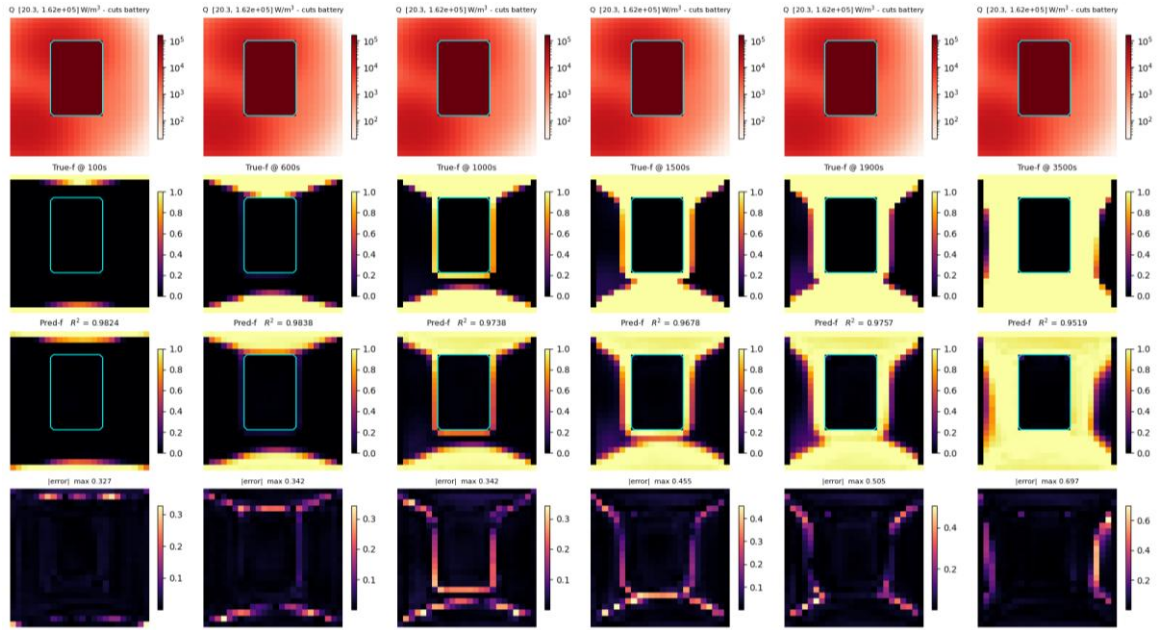


Fig. 46(f): Reference and predicted three-dimensional slice-based liquid-fraction fields for z-normal plane 9 (xy-plane), with $R^2 = 0.9766$ over the full space–time field of the plane.

Melting begins as narrow layers at the hot faces and as an annular region around the battery, and the two regions subsequently expand and merge. The battery body itself remains at zero liquid fraction throughout, as it must, since its cells are excluded from the phase-change relation by the lumped-capacitance treatment of Section 3.5.5; the surrogate reproduces this correctly and does not melt the source region. The predicted fronts follow the reference fronts in both position and shape, and the residual error is again confined to thin bands tracking the instantaneous interface and to the battery perimeter.

Table 8: Slice-wise liquid-fraction prediction accuracy of the slice-based three-dimensional U-DeepONet for the same planes. Liquid-fraction errors are dimensionless.

Plane	x-normal R^2	x-normal RMSE	y-normal R^2	y-normal RMSE	z-normal R^2	z-normal RMSE
1	0.9384	0.1183	0.9469	0.0899	0.9779	0.0676
2	0.9444	0.1132	0.9552	0.0841	0.9775	0.0692
3	0.9528	0.1048	0.9500	0.0892	0.9778	0.0695
4	0.9519	0.1058	0.9407	0.0969	0.9776	0.0699
5	0.9459	0.1119	0.9415	0.0959	0.9772	0.0704
6	0.9407	0.1169	0.9490	0.0882	0.9766	0.0709

Liquid-fraction R^2 lies between 0.9384 and 0.9528 on the x-normal planes, between 0.9407 and 0.9552 on the y-normal planes and between 0.9766 and 0.9779 on the z-normal planes. The ordering of the three orientations is the same as for the temperature, and the absolute values are lower, which is expected: the liquid fraction varies sharply across the interface, so a displacement of the predicted front by a single grid cell saturates the pointwise error there, whereas the temperature field is smooth and forgiving of the same displacement.

Considering the full set of fifteen planes on each axis rather than the six shown, accuracy is highest on the planes that intersect the battery and falls away on the outermost planes at either end of each axis. This is a property of the metric rather than a failure of the model: the coefficient of determination normalises by the variance of the reference field, and the outermost planes lie close to the bounding faces where the field is nearly uniform, so a small absolute error is divided by a small variance. The root-mean-square errors confirm this reading, the outermost x-normal plane carrying the lowest temperature R^2 of the set at 0.7950 while also carrying one of the lowest root-mean-square errors at 0.92 K. The surrogate is therefore most reliable precisely where the thermal field is most structured, which is where a prediction is actually wanted.

4.5 Cross-Geometry Generalisation

A surrogate is of practical value only if it remains reliable for inputs unlike those it was trained on. The configurations examined so far all belong to the distribution from which the training data were drawn. The study reported here steps outside it deliberately, using the two training configurations described in Section 3.6: a model trained exclusively on prismatic battery sources, and a model trained on the geometrically generalised dataset containing prismatic, circular and elliptical sources in equal proportion.

The circular test configuration comprises three circular sources of increasing radius arranged along a diagonal, with a peak generation rate of $1.11 \times 10^6 \text{ W m}^{-2}$ at a domain-averaged value of $1.37 \times 10^5 \text{ W m}^{-2}$. The elliptical configuration comprises three ellipses of differing aspect ratio and orientation, peaking at $1.12 \times 10^6 \text{ W m}^{-2}$ at a mean of $1.22 \times 10^5 \text{ W m}^{-2}$. The two cases therefore impose comparable thermal loading and differ principally in the curvature of the source boundaries, which is the feature the prismatic-trained model has never encountered.

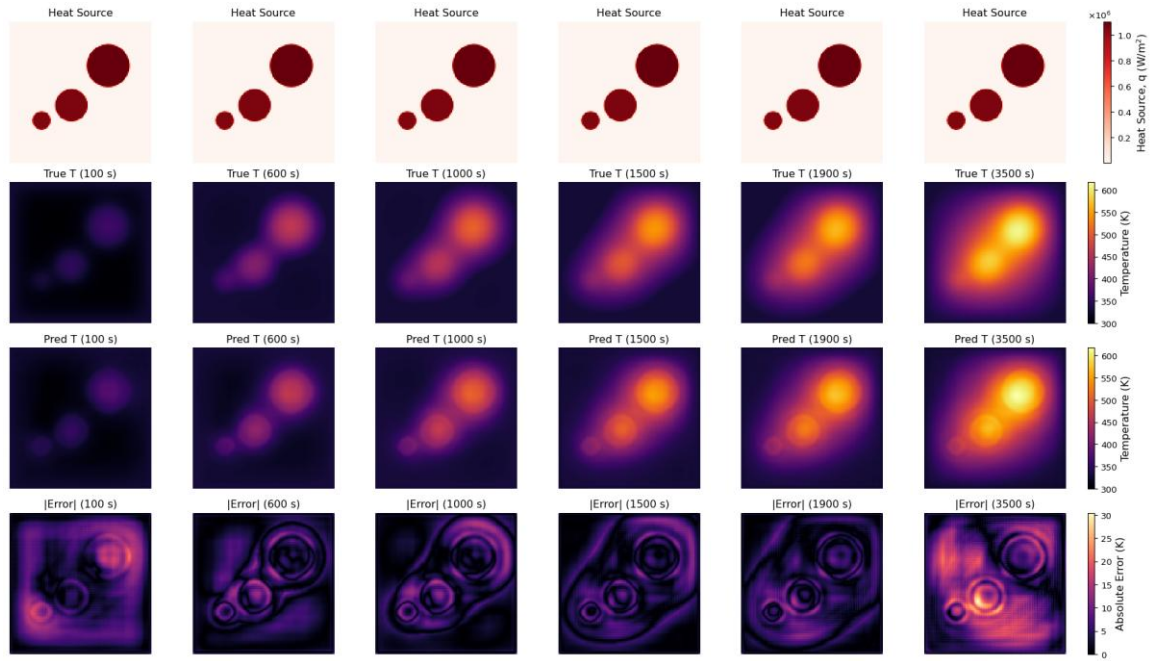


Fig. 47(a): Cross-geometry generalisation to unseen circular heat sources, predicted by the model trained exclusively on prismatic sources. Imposed heat-source field with the reference and predicted temperature fields and the corresponding absolute-error maps, at six instants between 100 and 3500 s.

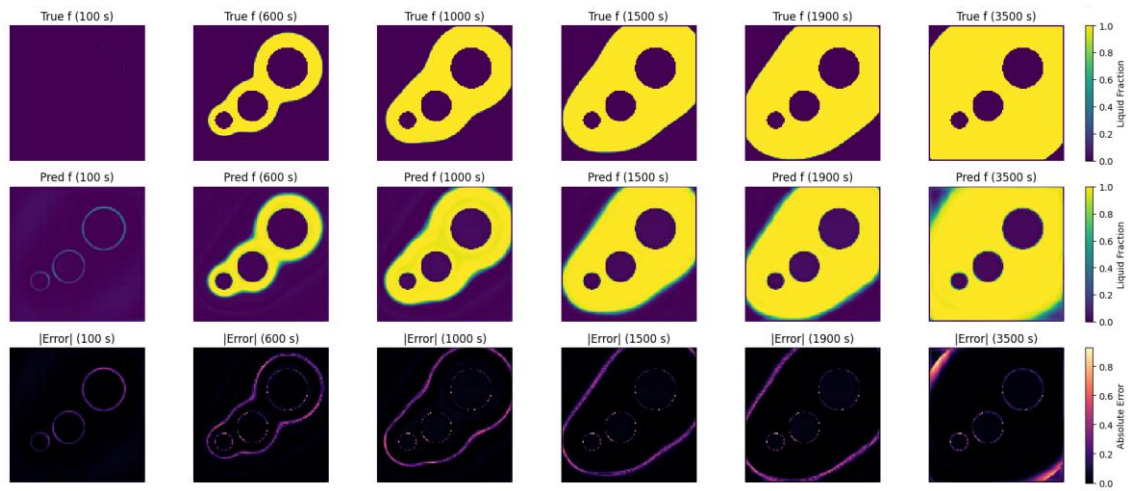


Fig. 47(b): Cross-geometry generalisation to unseen circular heat sources, the same comparison for the liquid-fraction field.

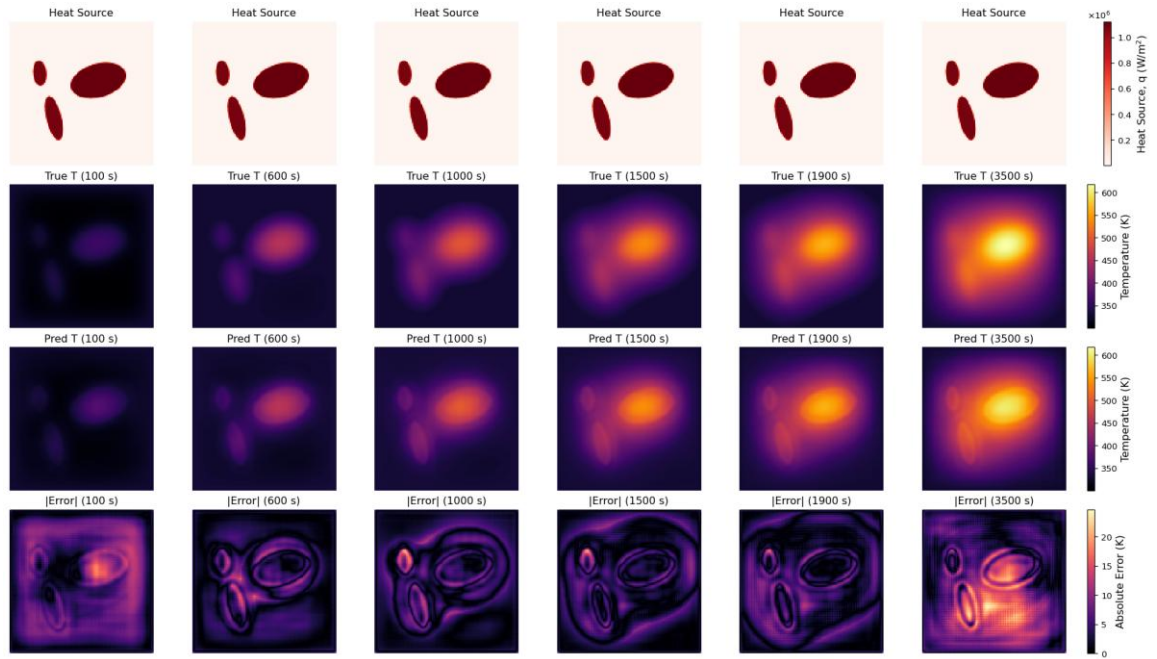


Fig. 48(a): Cross-geometry generalisation to unseen elliptical heat sources of differing aspect ratio and orientation, predicted by the model trained exclusively on prismatic sources. Imposed heat-source field with the reference and predicted temperature fields and the corresponding absolute-error maps.

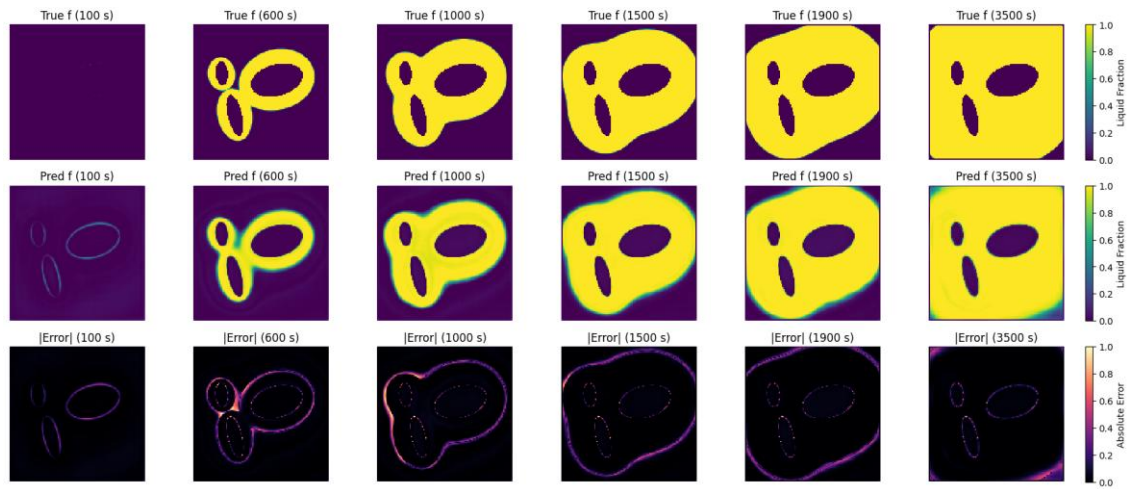


Fig. 48(b): Cross-geometry generalisation to unseen elliptical heat sources, the same comparison for the liquid-fraction field.

For both geometries the surrogate recovers the position, extent and relative intensity of the thermal field surrounding each source, its growth into a merged high-temperature region reaching approximately 620 K, and the annular melting that initiates around each source before expanding and coalescing into a single connected liquid region, while correctly retaining the unmelted source bodies. The error maps localise the discrepancy to the source perimeters, where the imposed field is discontinuous, and to the advancing melt front,

rather than distributing it over the domain. At the latest instant a lagging region persists in the corner furthest from the sources, where the prediction retains an intermediate liquid fraction while the reference solution is fully melted.

Table 9: Cross-geometry generalisation performance of the U-DeepONet for unseen circular and elliptical heat-source configurations, comparing a model trained only on prismatic sources with one trained on a geometrically generalised dataset. Errors are evaluated over the full space–time field of each case; liquid-fraction errors are dimensionless.

Test geometry	Training dataset	T MAE (K)	T RMSE (K)	T R ²	f MAE	f RMSE	f R ²
Circular	Prismatic-only	3.95	5.30	0.9918	0.0283	0.0837	0.9714
Circular	Generalised	2.96	4.06	0.9952	0.0334	0.0992	0.9598
Elliptical	Prismatic-only	3.48	4.62	0.9927	0.0301	0.0914	0.9659
Elliptical	Generalised	2.82	3.97	0.9946	0.0365	0.1060	0.9542

The first result is that transfer occurs at all. The prismatic-trained model predicts both curved geometries with liquid-fraction mean absolute errors of 0.0283 and 0.0301 at coefficients of determination of 0.9714 and 0.9659, and temperature errors of 3.95 K and 3.48 K at R² values of 0.9918 and 0.9927, despite neither shape appearing anywhere in its training data. This indicates that the operator has not memorised the specific source shapes present in that data, but has instead learned a largely local mapping from the imposed generation field to the thermal and phase-change response. A model that had memorised rectangles would have no means of responding correctly to a circle.

The second result is that training on the geometrically diverse dataset produces a consistent but field-dependent effect. The temperature prediction improves for both geometries, the mean absolute error falling from 3.95 K to 2.96 K for the circular case and from 3.48 K to 2.82 K for the elliptical case, with R² rising in both instances. The liquid-fraction prediction degrades by a comparable relative margin, the mean absolute error rising from 0.0283 to 0.0334 and from 0.0301 to 0.0365, with R² falling from 0.9714 to 0.9598 and from 0.9659 to 0.9542. Since the direction of both effects is the same for the two test geometries, the behaviour is attributable to the change in training set rather than to the particular configuration examined.

A plausible explanation is that the two fields are limited by different things. The temperature field is smooth and benefits from the broader coverage of source arrangements that the generalised dataset provides. The liquid-fraction error, by contrast, is dominated

by the accurate placement of a thin melt front, and is therefore more sensitive to the reduced number of training examples per geometry that follows from distributing a fixed case budget across three source shapes. Establishing this would require a controlled comparison in which the number of cases per geometry and the training schedule are matched, which is left to future work.

The practical implication is worth stating plainly. If the quantity of interest is the temperature field, diversifying the training geometries is worthwhile. If it is the liquid fraction, and the geometry of the application is known in advance, concentrating the simulation budget on that geometry appears to be the better use of it, and the prismatic-trained model transfers well enough that the loss from doing so is small.

4.6 Computational Cost

The accuracy reported above is useful only if the surrogate is substantially cheaper than the solver it replaces. For the two-dimensional problem the numerical solver requires approximately 42 to 48 s per case to advance the coupled enthalpy–temperature formulation on the 120×120 grid and produce the twenty-four stored snapshots. The U-DeepONet returns the complete transient field in a single forward pass, in a median time of 0.077 s and within 0.069 to 0.086 s for the majority of runs performed. This corresponds to a typical speed-up of approximately 545 to 620 times, or about 3.2 ms per predicted snapshot against roughly 1.9 s per snapshot for the solver.

For the three-dimensional problem the relevant comparison is per case rather than per plane, since the surrogate returns all forty-five planes at all twenty-four snapshots in one batched evaluation. The measured costs are collected in Table 10.

Table 10: Evaluation cost of the numerical solver and of the U-DeepONet surrogate in two and three dimensions. Three-dimensional timings are for one complete test case on a CPU using fourteen threads.

Quantity	Temperature	Liquid fraction
2D solver, per case	42–48 s	42–48 s
2D U-DeepONet inference, per case	0.069–0.086 s	0.069–0.086 s
2D median inference	0.077 s	0.077 s
2D typical speed-up	545–620 ×	545–620 ×
3D planes per case	45	45

Quantity	Temperature	Liquid fraction
3D snapshots per plane	24	24
3D predicted images per case	1080	1080
3D total inference time per case	0.51–0.59 s	0.91–0.97 s
3D time per predicted image	≈ 0.5 ms	≈ 0.9 ms
3D time per plane	11–13 ms	17–22 ms
Trainable parameters, 3D model	1,930,689	1,930,689

The complete three-dimensional thermal history of a case is therefore returned in roughly half a second for the temperature field and under one second for the liquid fraction, against the tens of seconds required by the volumetric solver on the same hardware. Because the temporal trunk is evaluated at arbitrary query times, the full history is obtained without marching through intermediate time steps, so the cost of a prediction is essentially independent of the number of snapshots requested. The liquid-fraction model is the slower of the two despite having an identical architecture, which reflects run-to-run variation in thread scheduling rather than any structural difference between the networks.

4.7 General Discussion

The results indicate that the proposed U-DeepONet surrogate model is able to capture the essential thermal and phase-change dynamics of the PCM systems under various heating scenarios. For volumetric, boundary-driven, and spatially varying Gaussian heat sources $Q(x, y)$, the model is able to capture the development of hotspots, heat diffusion, and the melting process. The predicted temperature fields are in close agreement with the numerical solver solutions in terms of both spatial variation and magnitude. Although a slight smoothing effect is noticed in regions with sharp gradients, which is a common characteristic of neural network surrogates, the physical thermal patterns are preserved. Likewise, the predicted liquid fraction fields are able to capture the melting initiation regions and the propagating melt fronts accurately without generating physically spurious results. This can be said for 1D, 2D and 3D datasets and PCM behaviour.

The data efficiency analysis has confirmed that the model is data-efficient, with the accuracy increasing steadily with the number of training data and converging beyond a certain number of training data. Moreover, the grid-to-grid formulation has resulted in a substantial reduction in computational cost, allowing for extremely fast training and inference times.

Thus, we can say the U-DeepONet framework is successful in learning the operator mapping from the heat-source configuration to the thermo-physical response. The predictions made by the framework are accurate and require only a fraction of the computational cost of the conventional solvers.

CHAPTER 5:

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This research applied operator-learning-based surrogate modeling to the transient thermal and phase change processes in a PCM-systems. The research started with a simplified one-dimensional model, where the DeepONet was first validated for the spatial heat-source function to temperature field mapping. The successful 1D model implementation allowed the researchers to be confident that operator learning can represent diffusion-dominated thermal processes in a stable and consistent way.

The model was later generalized to two-dimensional transient simulations of the PCM with an enthalpy-based numerical solver for dataset generation. A diverse dataset of high-fidelity simulations was created, which included variations in volumetric heat-source intensity, boundary temperature profiles, and spatially correlated Gaussian-process heat-source fields.

The initial 2D tests were conducted using a pooled pointwise DeepONet framework, where each space-time point was considered an independent training data point. Although the results were acceptable, the process was computationally expensive and did not leverage the spatial characteristics of the problem effectively. To overcome these issues, a grid-to-grid U-DeepONet model was designed, combining a U-Net-inspired spatial encoder with a temporal trunk network. In this setup, each PCM simulation was considered a single structured training example, as opposed to millions of randomly distributed points.

The proposed U-DeepONet model ensured stable convergence, high predictive capability, and improved computational performance. It successfully simulated the temperature dynamics and liquid fraction development under various heating modes, accurately modeling the development of hotspots, melt fronts moving towards the center, and irregular melting patterns.

Although a mild smoothening effect was noticed near the sharp interfaces, the results remained physically sound and consistent with the major thermal and PCM dynamics.

A systematic data sensitivity analysis further revealed that the model is data-efficient, requiring only a few hundred simulations to achieve a reliable level of performance and improving gradually before saturating. This further validates that the convolution-based spatial encoding scheme is an effective way to endow the model with a strong inductive bias towards learning diffusion-driven physics.

The framework was expanded to a slice-based three-dimensional U-DeepONet formulation for predicting transient 3D PCM melting. Instead of learning directly on costly volumetric tensors, we broke down the complete three-dimensional thermal fields into several orthogonal two-dimensional slices along the x , y , and z directions. Each slice was represented by a multi-channel tensor that included heat-source distributions, coordinate encodings, boundary-condition maps, mask channels, and temporal information.

The slice-based 3D U-DeepONet learned the transient three-dimensional thermal evolution while working fully with two-dimensional slice representations. The model predicted temperature fields and liquid-fraction evolution across various slice orientations and time snapshots. The predicted slices matched well with the numerical solutions, effectively capturing thermal gradients, hotspot propagation, and melt-front dynamics with low prediction error. The results showed that this slicing strategy significantly cuts down the computational cost linked to full Conv3D architectures while still maintaining key three-dimensional thermal behaviour.

More importantly, the trained surrogate model enables near-instantaneous predictions of temperature and liquid fraction fields. Compared to the performance of conventional numerical solvers, this represents a substantial reduction in computational cost while retaining acceptable levels of accuracy.

The data-sensitivity study was subsequently extended over a wider and more finely spaced range of training-set sizes on the generalised dataset. The error decreased steeply between 80 and roughly 300 training simulations and then flattened, the improvement obtained by going from 300 to 500 simulations being small in comparison with the additional solver time required to produce those cases. The validation error tracked the training error closely throughout, with no separation between the two curves, which indicates that the network is limited by the amount of data rather than by its capacity and that it does not overfit at the dataset sizes considered. A training budget of the order of three hundred simulations is therefore sufficient for this class of problems, and this is a useful result for practice, because the cost of building a surrogate is dominated by the cost of generating its training data.

The ability of the surrogate to generalise across heat-source geometries was then examined directly, by testing models on source shapes they had never been trained on. A model trained exclusively on prismatic sources predicted the circular and elliptical test cases with temperature coefficients of determination of 0.9918 and 0.9927 respectively, showing that the operator learns the thermal response to a localised source rather than memorising a particular geometry. Adding the unseen geometries to the training set raised these values to 0.9952 and 0.9946 and reduced the temperature root-mean-square error from 5.30 K to 4.06 K and from 4.62 K to 3.97 K. The liquid fraction behaved in the opposite manner: the generalised model was slightly less accurate than the prismatic-only model on both test geometries, its coefficient of determination falling from 0.9714 to 0.9598 and from 0.9659 to 0.9542. Spreading a fixed training budget across several source shapes therefore improves the smooth temperature field but dilutes the resolution of the sharper melt-front field, and the two objectives are in tension.

The three-dimensional framework was applied to a prismatic battery-shaped heat source embedded in the PCM domain, a configuration that represents a battery module surrounded by a phase-change thermal buffer. The slice-based U-DeepONet reconstructed the full transient three-dimensional temperature field from two-dimensional slice representations alone, with slice-wise coefficients of determination between 0.9517 and 0.9879 and root-mean-square errors between 1.35 K and 3.24 K over the planes that cut the battery. The corresponding liquid-fraction predictions reached coefficients of determination between

0.9384 and 0.9779 with root-mean-square errors between 0.067 and 0.118. Accuracy was highest on the planes normal to the shortest dimension of the source, where the field varies most smoothly, and lowest on the planes that cut the source across its longest dimension, where the thermal gradients at the source boundary are steepest; the melt front is reproduced in the correct position at every instant, with a mild smoothing across the interface of the same character as that already observed in two dimensions.

The computational argument for the surrogate is unchanged by the move to three dimensions and is in fact strengthened by it. The trained three-dimensional model reconstructs all forty-five planes at all twenty-four stored instants, a total of 1080 field images per case, in 0.51 to 0.59 s for temperature and 0.91 to 0.97 s for liquid fraction, against a solver time of several seconds for the same case. The model carries 1,930,689 trainable parameters, which is modest by the standards of three-dimensional surrogates and is a direct consequence of the slice-based formulation, since the network operates on two-dimensional slices rather than on a volumetric tensor. The framework is therefore usable inside a design loop in which many geometric and operating configurations must be evaluated, which is precisely the situation in which a direct solver becomes prohibitive.

In conclusion, this study has demonstrated that operator learning, from pointwise DeepONet to grid-based U-DeepONet, can be an effective and efficient surrogate modelling framework for PCM- thermal energy storage systems.

5.2 Future Scope

- Extend the surrogate modelling to 3D PCM-metal foam geometries for higher realism in design studies.
- To include convection formulation in the data generation analysis to make the data more generalized.
- To improve and lower the training time of the model, enabling it to converge faster.
- Explore foam design optimization (porosity, pore size, material selection) using the DeepONet surrogate to balance latent storage capacity and thermal response.

- Couple the slice-based three-dimensional surrogate with an electrochemical battery model so that the heat generation rate follows the duty cycle of the cell rather than being prescribed.
- Train separate output heads, or apply a weighted loss, so that the loss of liquid-fraction accuracy observed when a single model is trained across several source geometries is recovered.
- Extend the cross-geometry study from the shape of the heat source to the shape of the enclosure itself, and to source positions away from the centre of the domain.
- Embed the surrogate in a design-optimisation loop for battery thermal management, using it to size the phase-change buffer and to place the cells for a prescribed peak-temperature limit.
- Incorporate the governing energy equation as a residual penalty during training, so that the predicted fields satisfy the enthalpy balance and the smoothing at the melt front is reduced.

REFERENCES

- [1] Dinesh, B.V.S., & Bhattacharya, A. (2019). Effect of foam geometry on heat absorption characteristics of PCM–metal foam composite thermal energy storage systems. *International Journal of Heat and Mass Transfer*, 134, 866–883. <https://doi.org/10.1016/j.ijheatmasstransfer.2019.01.095>
- [2] Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G.E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3, 218–229.
- [3] Raissi, M., Perdikaris, P., & Karniadakis, G.E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs. *Journal of Computational Physics*, 378, 686–707.
- [4] Li, Z., Kovachki, N., Azizzadenesheli, K., et al. (2021). Fourier Neural Operator for Parametric PDEs. *International Conference on Learning Representations (ICLR)*.
- [5] Krishnan, S., Garimella, S.V., & Kang, S. (2005). A two-temperature model for metal-foam heat exchangers. *Journal of Heat Transfer*, 127, 995–1004.
- [6] Yang, C., & Garimella, S.V. (2010). Melting of phase change materials in metal foams. *International Journal of Heat and Mass Transfer*, 53(19–20), 4279–4288.
- [7] Ren, Q., Chan, C.L., & Ding, Y. (2017). Pore-scale modeling of PCM melting in open-cell metal foams. *Applied Energy*, 203, 357–369.
- [8] Abishek, C., Verma, P., & Singh, R. (2018). Effect of foam surface area on PCM melting. *Energy Conversion and Management*, 171, 1414–1425.
- [9] Prasad, K., et al. (2021). Predicting phase change using convolutional neural networks. *Applied Thermal Engineering*, 197, 117398.
- [10] Zhou, Y., et al. (2022). Autoencoder–LSTM surrogate model for PCM heating. *Energy and AI*, 7, 100144.
- [11] Subhayan, P.T., Brewick, P. (2024). A bi-fidelity DeepONet approach for modeling hysteretic systems under uncertainty. *Applied Mathematical Modelling*, 135, 708–728.
- [12] Choubey, S., Rahia, S., Pal, B., Agrawal, M. (2023). A novel data generation scheme for surrogate modelling with deep operator networks. *Applied Mathematical Modelling*, 116, 675–693.
- [13] Williams, C.K.I., & Rasmussen, C.E. (2006). *Gaussian Processes for Machine Learning*. MIT Press.
- [14] Chibani, A., Haddad, M.A.N., Boucetta, C., Merouani, S., Badji, R., Rashid, F.L., & Mohammed, H.I. (2025). Simultaneous evaluation of PCM–metal foam (polyethylene glycol 1500–AlSi10Mg) composite with porous structures fabricated via additive manufacturing: The influence of porosity and pore size. *Applied Thermal Engineering*, 126988. <https://doi.org/10.1016/j.applthermaleng.2025.126988>

- [15] Diab, W., & Al Kobaisi, M. (2023). U-DeepONet: U-Net enhanced Deep Operator Network for geologic carbon sequestration. *Computers & Geosciences*.
- [16] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 234–241.
- [17] Koric, S., et al. (2023). DeepONet-based surrogate modeling for heat conduction with parametric heat sources. *International Journal of Heat and Mass Transfer*, 205, 123000.
- [18] Wang, S., et al. (2020). Physics-informed neural networks for solving moving interface Stefan problems. *Computational Physics*.
- [19] Li, Y., Bazant, M.Z., & Zhu, Y. (2023). Phase-field DeepONet for gradient-flow dynamics. *Computer Methods in Applied Mechanics and Engineering*, 404, 115807.
- [20] Ciesielski, R., et al. (2024). DeepONet surrogate modeling of phase-field solidification microstructure evolution. *Computational Materials Science*, 240, 112345.
- [21] Mao, X., et al. (2025). Predicting melting of gallium in nickel porous media using Fourier neural operators. *ASME Journal of Heat Transfer*.
- [22] Chen, T., & Chen, H. (1995). Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Transactions on Neural Networks*, 6, 911–917.
- [23] Lu, L., Jin, P., & Karniadakis, G.E. (2019). DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *arXiv preprint arXiv:1910.03193*.
- [24] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2020). Neural operator: Graph kernel network for partial differential equations. *International Conference on Learning Representations*.
- [25] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Stuart, A., & Anandkumar, A. (2020). Multipole graph neural operator for parametric partial differential equations. *Advances in Neural Information Processing Systems*.
- [26] Kovachki, N., Lanthaler, S., & Mishra, S. (2021). Neural operator: Learning maps between function spaces. *Journal of Machine Learning Research*, 22, 1–97.
- [27] Deng, B., Shin, Y., Lu, L., & Karniadakis, G.E. (2022). On the approximation and generalization of DeepONets for learning operators arising from advection–diffusion equations. *Neural Networks*, 153, 411–426.
- [28] Wang, S., Wang, H., & Perdikaris, P. (2021). Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Science Advances*, 7, eabi8605.
- [29] Li, Z., Huang, D., Liu, B., & Anandkumar, A. (2022). Physics-informed neural operator for learning partial differential equations. *International Conference on Learning Representations*.
- [30] Lu, L., Pestourie, R., Yao, W., Wang, Z., Verdugo, F., Johnson, S.G., & Karniadakis, G.E. (2022). Multifidelity DeepONet for learning the solution operators of parametric partial differential equations. *Physical Review Research*, 4, 023210.

- [31] Garg, A., Gupta, H., & Chakraborty, S. (2023). VB-DeepONet: A Bayesian operator learning framework for uncertainty quantification. *Engineering Applications of Artificial Intelligence*, 125, 105685.
- [32] Qiu, T., Bridges, R.A., & Chen, M. (2024). Derivative-enhanced deep operator network. *Advances in Neural Information Processing Systems*, 22013–22035.
- [33] Mora, C., Yousefpour, A., Hosseinmardi, S., Owhadi, H., & Bostanabad, R. (2025). Operator learning with Gaussian processes. *Computer Methods in Applied Mechanics and Engineering*, 434, 117581.
- [34] Lanthaler, S. (2023). Error estimates for PCA-Net and reduced basis neural operators. *Journal of Machine Learning Research*, 24, 1–67.
- [35] Chen, W., He, Y., & Song, Z. (2026). A novel mesoscopic multi-head PI-DeepONet framework for generalized phase change prediction of porous composite PCMs in multi-dimensional parameter spaces. *Energy*, 302, 131665.
- [36] Li, Z., Kovachki, N., Choy, C., Li, B., Kossaifi, J., Otta, S., Nabian, M.A., Azizzadenesheli, K., & Anandkumar, A. (2023). Geometry-informed neural operator for large-scale 3D PDEs. *Advances in Neural Information Processing Systems*, 35790–35805.
- [37] Zhu, Y., et al. (2023). Reliable extrapolation of deep neural operators informed by physics or sparse observations. *Computer Methods in Applied Mechanics and Engineering*, 412, 116093.